

part_1

October 25, 2025



1 CNC Milling Machine - Tool Wear Detection

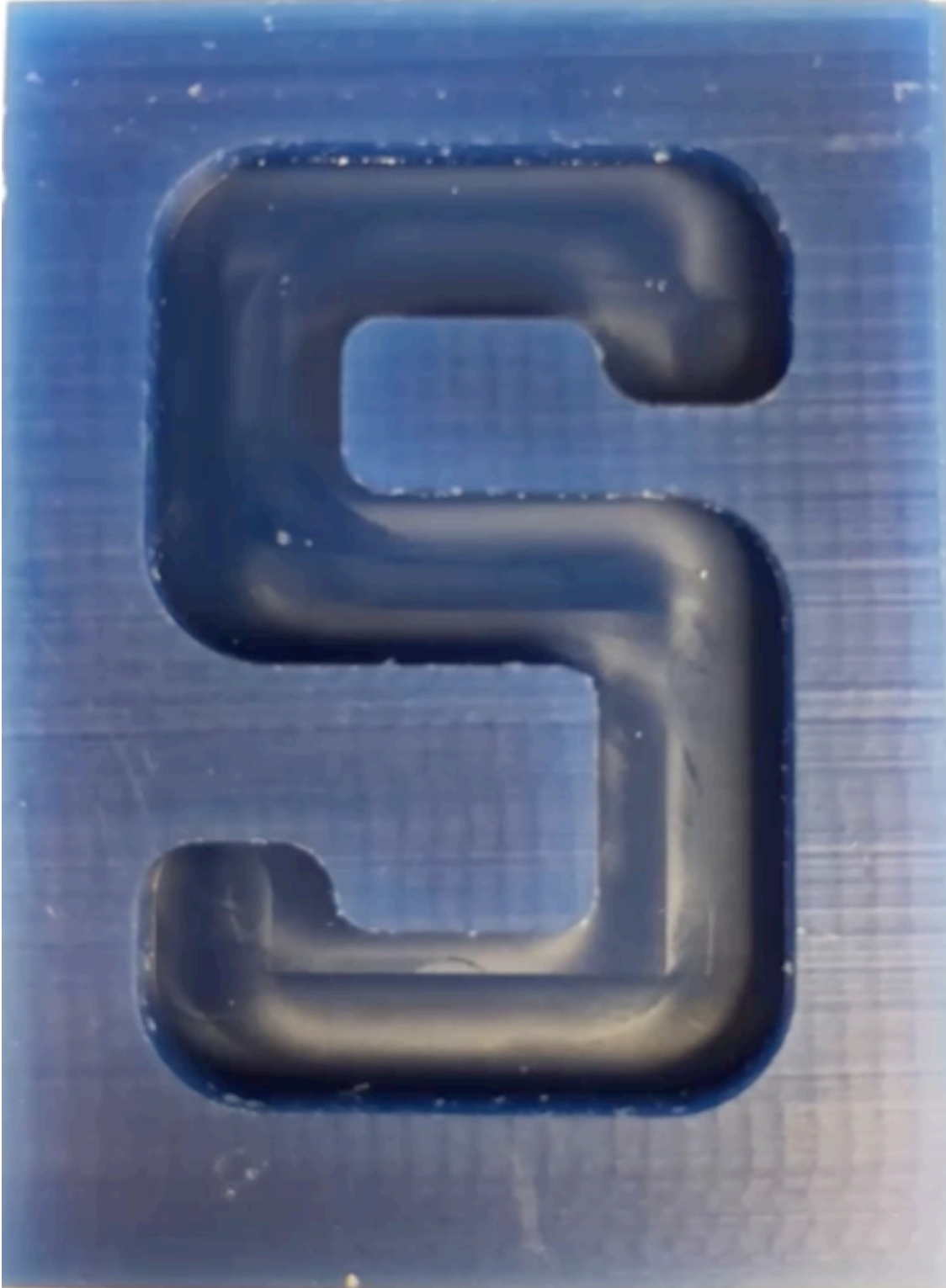
Data description

Time series data was collected from 18 experiments with a sampling rate of 100ms and are separately reported in files experiment_01.csv to experiment_18.csv. Each file has measurements from the 4

motors in the CNC(X, Y, Z axes and S (spindle))

Problem Statement A series of machining experiments were run on 2” x 2” x 1.5” wax blocks in a CNC milling machine in the System-level Manufacturing and Automation Research Testbed (SMART) at the University of Michigan. Machining data was collected from a CNC machine for variations of tool condition, feed rate, and clamping pressure. Each experiment produced a finished wax part with an “S” shape - S for smart manufacturing - carved into the top face, as shown in test_artifact.jpg

2 x 2 x 1,5
(SMART) . S- (S
,), , test_artifact.jpg.



Goals

The goal of our project is to predict the wear of parts on a CNC milling machine

Features

X-axis variables

Variable	Description
X1_ActualPosition	Actual x position of part (mm)
X1_ActualVelocity	Actual x velocity of part (mm/s)
X1_ActualAcceleration	Actual x acceleration of part (mm/s/s)
X1_CommandPosition	Reference x position of part (mm)
X1_CommandVelocity	Reference x velocity of part (mm/s)
X1_CommandAcceleration	Reference x acceleration of part (mm/s/s)
X1_CurrentFeedback	Current (A)
X1_DCBusVoltage	Voltage (V)
X1_OutputCurrent	Current (A)
X1_OutputVoltage	Voltage (V)
X1_OutputPower	Power (kW)

Y-axis variables

Variable	Description
Y1_ActualPosition	Actual y position of part (mm)
Y1_ActualVelocity	Actual y velocity of part (mm/s)
Y1_ActualAcceleration	Actual y acceleration of part (mm/s/s)
Y1_CommandPosition	Reference y position of part (mm)
Y1_CommandVelocity	Reference y velocity of part (mm/s)
Y1_CommandAcceleration	Reference y acceleration of part (mm/s/s)
Y1_CurrentFeedback	Current (A)
Y1_DCBusVoltage	Voltage (V)
Y1_OutputCurrent	Current (A)
Y1_OutputVoltage	Voltage (V)
Y1_OutputPower	Power (kW)

Z-axis variables

Feature	Description
Z1_ActualPosition	Actual z position of part (mm)
Z1_ActualVelocity	Actual z velocity of part (mm/s)
Z1_ActualAcceleration	Actual z acceleration of part (mm/s/s)
Z1_CommandPosition	Reference z position of part (mm)
Z1_CommandVelocity	Reference z velocity of part (mm/s)
Z1_CommandAcceleration	Reference z acceleration of part (mm/s/s)
Z1_CurrentFeedback	Current (A)
Z1_DCBusVoltage	Voltage (V)
Z1_OutputCurrent	Current (A)
Z1_OutputVoltage	Voltage (V)

S - Spindle

Feature	Description
S1_ActualPosition	Actual position of spindle (mm)
S1_ActualVelocity	Actual velocity of spindle (mm/s)
S1_ActualAcceleration	Actual acceleration of spindle (mm/s/s)
S1_CommandPosition	Reference position of spindle (mm)
S1_CommandVelocity	Reference velocity of spindle (mm/s)
S1_CommandAcceleration	Reference acceleration of spindle (mm/s/s)
S1_CurrentFeedback	Current (A)
S1_DCBusVoltage	Voltage (V)
S1_OutputCurrent	Current (A)
S1_OutputVoltage	Voltage (V)
S1_OutputPower	Power (A)
S1_SystemInertia	Torque inertia (kg*m ²)
M1_CURRENT_PROGRAM_NUMBER	Number the program is listed under on the CNC
M1_sequence_number	Line of G-code being executed
M1_CURRENT_FEEDRATE	Instantaneous feed rate of spindle

1.1 1. Import libraries

```
[1]: import pandas as pd
import numpy as np
# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

# System
import os
import glob
import warnings

# Separate to train test split
from sklearn.model_selection import train_test_split, cross_val_score,
    ↪cross_val_predict

# Models
from sklearn.linear_model import SGDClassifier
from sklearn.ensemble import RandomForestClassifier,
    ↪GradientBoostingClassifier, AdaBoostClassifier, BaggingClassifier,
    ↪ExtraTreesClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC, LinearSVC
from sklearn.naive_bayes import GaussianNB
```

```

# Metrics
from sklearn.metrics import roc_auc_score, confusion_matrix, accuracy_score, \
    ↪classification_report, recall_score, f1_score, precision_score

# for search best parameters, build pipeline, cross_validation
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.pipeline import Pipeline

# Imputers
from feature_engine.imputation import RandomSampleImputer, CategoricalImputer, \
    ↪EndTailImputer
from sklearn.impute import KNNImputer, SimpleImputer
# Feature Magnitude
from sklearn.preprocessing import StandardScaler, Normalizer, MinMaxScaler, \
    ↪RobustScaler

# Feature Encoding
from feature_engine.encoding import CountFrequencyEncoder, OneHotEncoder, \
    ↪OrdinalEncoder

# Outliers
from feature_engine.outliers import Winsorizer

# Column Transformer
from sklearn.compose import ColumnTransformer

# For feature selection
from sklearn.feature_selection import VarianceThreshold
from feature_engine.selection import DropConstantFeatures, \
    ↪DropDuplicateFeatures, SmartCorrelatedSelection, DropCorrelatedFeatures

#Discretization
from feature_engine.discretisation import DecisionTreeDiscretiser

# For save models
import pickle
# Early stopping
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, \
    ↪ReduceLROnPlateau, LearningRateScheduler

# ANN library
import tensorflow as tf
from scikeras.wrappers import KerasRegressor
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout, BatchNormalization
from keras.optimizers import Adam, SGD, Nadam, RMSprop, Adadelta

```

```

from keras.callbacks import EarlyStopping
from scikeras.wrappers import KerasClassifier
from tensorflow.keras.regularizers import l2
from tensorflow.keras import regularizers

```

```

[2]: def diagnostic_plots(dataset, variable):

    fig = plt.figure()
    plt.figure(figsize = (16, 4))

    plt.subplot(1, 3, 1)
    sns.histplot(dataset[variable], bins = 30)
    plt.title('Histogram')

    plt.subplot(1, 3, 2)
    stats.probplot(dataset[variable], dist = 'norm', plot = plt)
    plt.ylabel('Variable Quantile')

    plt.subplot(1, 3, 3)
    sns.boxplot(y = dataset[variable])
    plt.title('BoxPlot')

    plt.show()

```

Diagnostic plots for show as histogram, probplot and boxplot

1.2 2. Load and Collecting the dataset

```

[3]: main_df = pd.read_csv('../input/train.csv')
main_df.head(5)

```

```

[4]: files = list()

for i in range(1, 19):
    exp_number = '0' + str(i) if i < 10 else str(i)
    file = pd.read_csv('../input/experiment_{}.csv'.format(exp_number))

    row = main_df[main_df['No'] == i]

    file['feedrate'] = row.iloc[0]['feedrate']
    file['clamp_pressure'] = row.iloc[0]['clamp_pressure']
    file['machining_finalized'] = row.iloc[0]['machining_finalized']
    file['passed_visual_inspection'] = row.iloc[0]['passed_visual_inspection']
    file['target'] = 1 if row.iloc[0]['tool_condition'] == 'worn' else 0

    files.append(file)

```

```
[5]: files
```

```
[6]: df = pd.concat(files, ignore_index = False)
df.shape
```

1.3 3. Exploratory Data Analysis (EDA)

```
[7]: # Check missing values
df.isnull().mean()
```

```
[8]: # Information about dataset
# From the analysis we see that all variable have numerical values except one
# variable
df.info()
```

```
[9]: # Describe variables
df.describe()
```

1.4 3.1 There are suspicions that some variables have empty values. They should be removed. Next we will check and delete

```
[10]: # Look at correlation between variables
corr = df.corr(numeric_only = True)
corr
```

```
[11]: # Find constant feat
constant_features = [col for col in df.columns if len(df[col].unique()) == 1]
print(constant_features)
```

```
[12]: # Drop constant feat
df.drop(['Z1_CurrentFeedback', 'Z1_DCBusVoltage', 'Z1_OutputCurrent',
        'Z1_OutputVoltage', 'S1_SystemInertia'],
        axis = 1, inplace = True)
```

```
[13]: df.shape
```

```
[14]: # Categorical variables
categorical_features = [
    var for var in df.columns if df[var].dtype == 'O'
]
categorical_features
```

```
[15]: # Numerical variables
numerical_features = [
    var for var in df.columns if df[var].dtype != 'O'
]
numerical_features
```

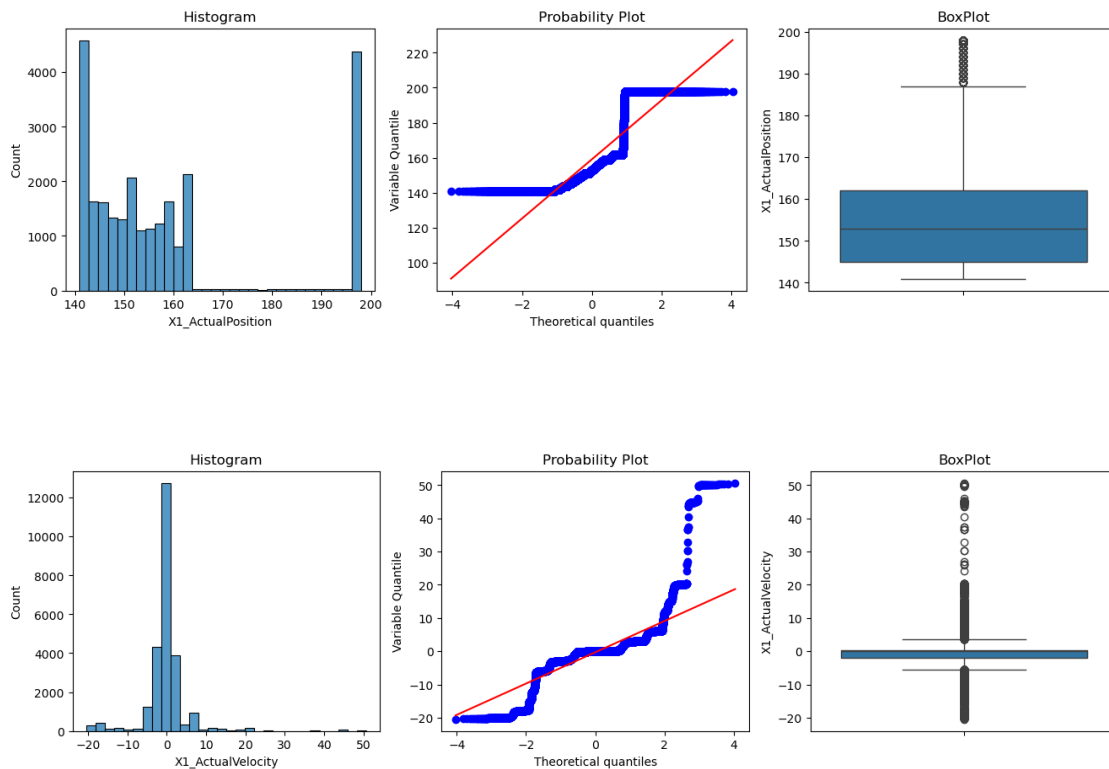
Temporarily code category variables using OneHotEncoder method from Feature-Engine for data analysis

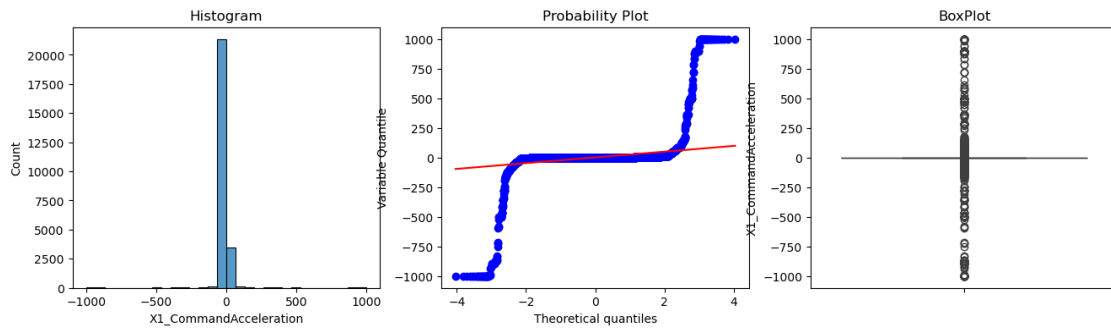
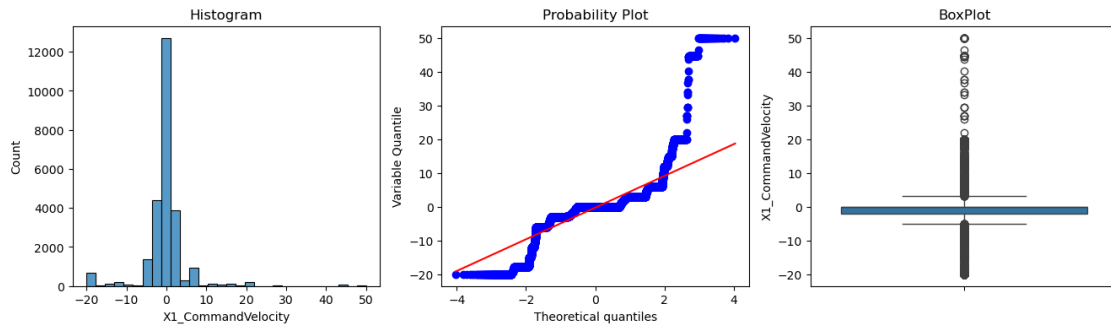
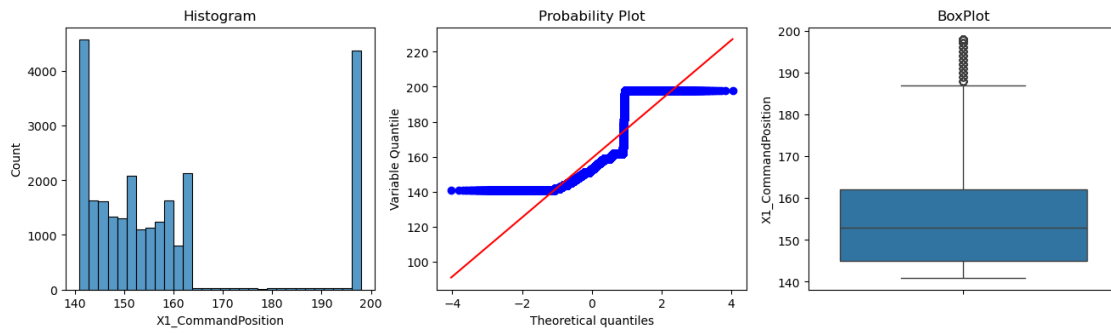
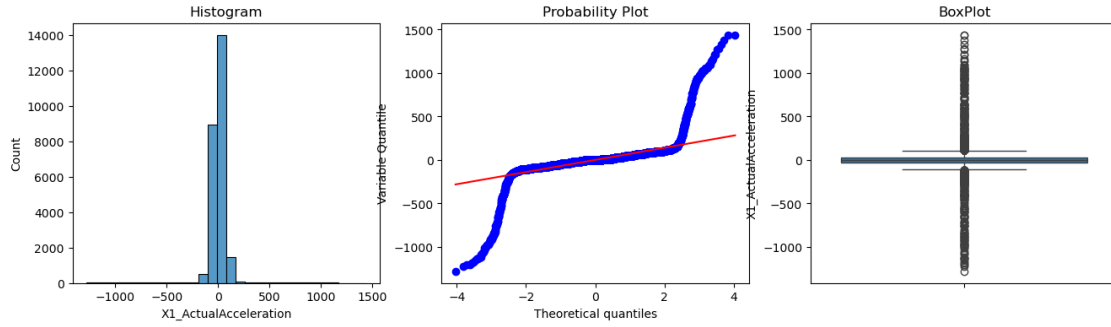
```
[16]: ohe_enc = OneHotEncoder(
        variables = ['Machining_Process', 'machining_finalized',
        ↪ 'passed_visual_inspection'],
        drop_last = True
    )
    ohe_enc.fit(df.fillna('Missing'))
```

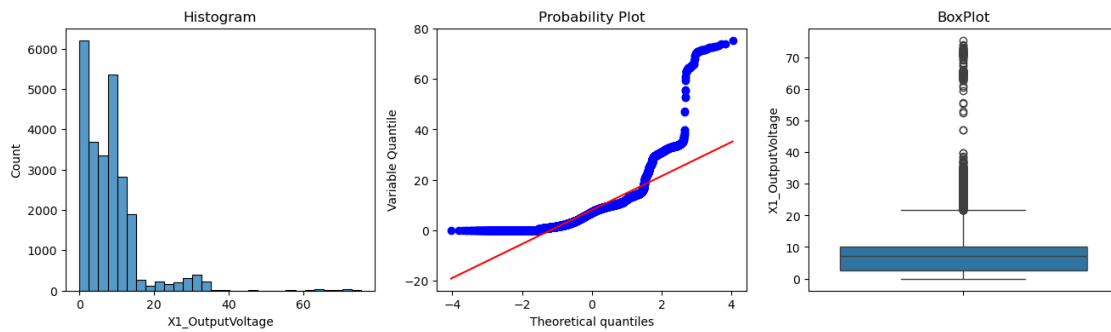
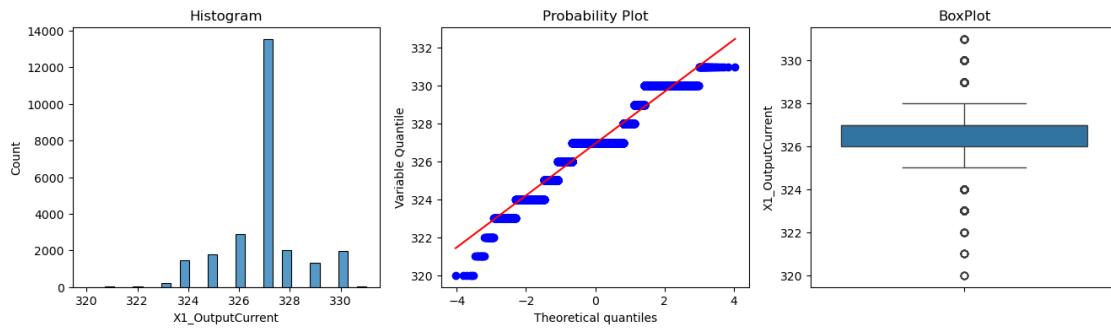
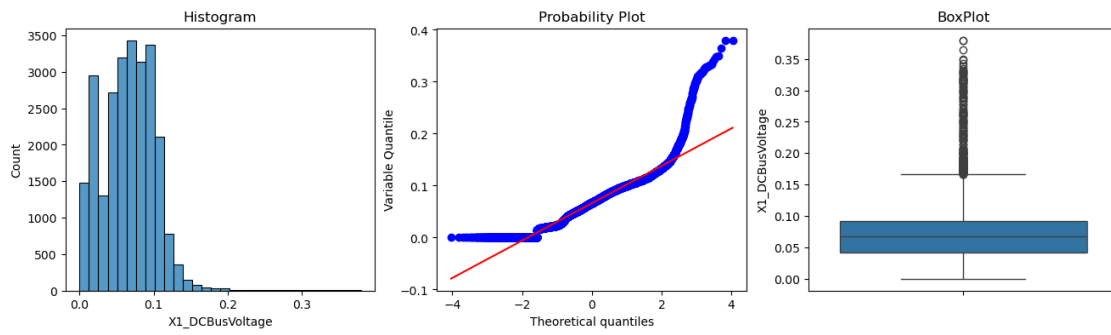
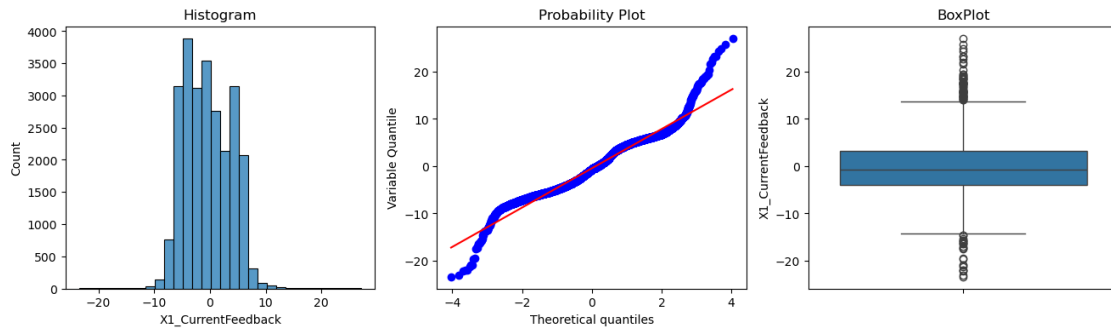
```
[17]: tmp = ohe_enc.transform(df.fillna('Missing'))
    tmp.shape
```

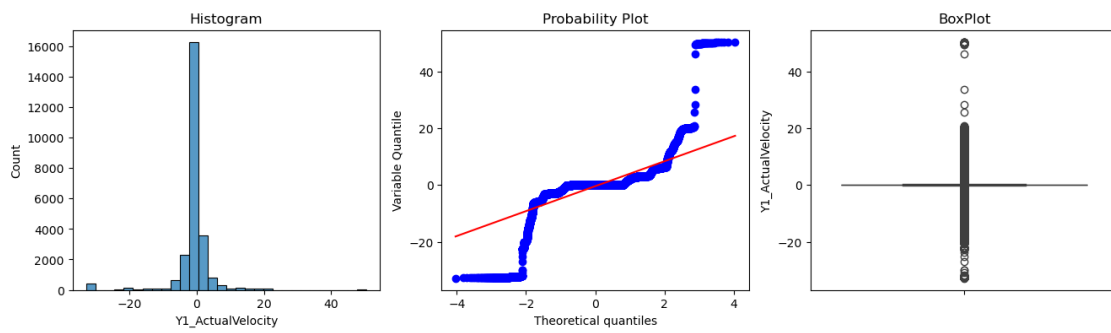
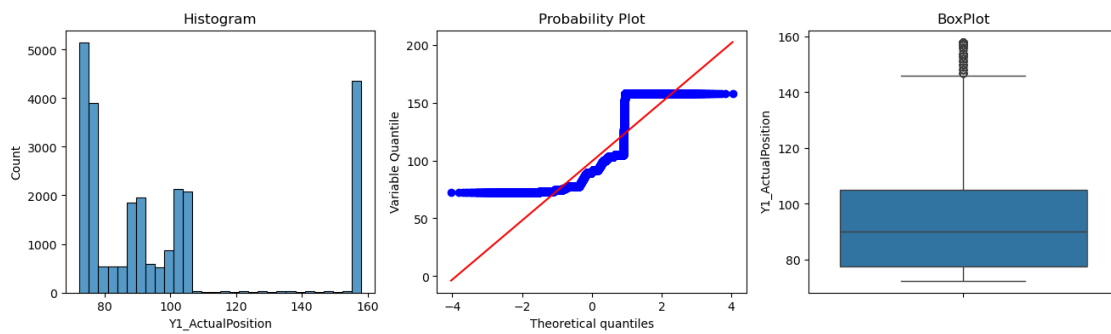
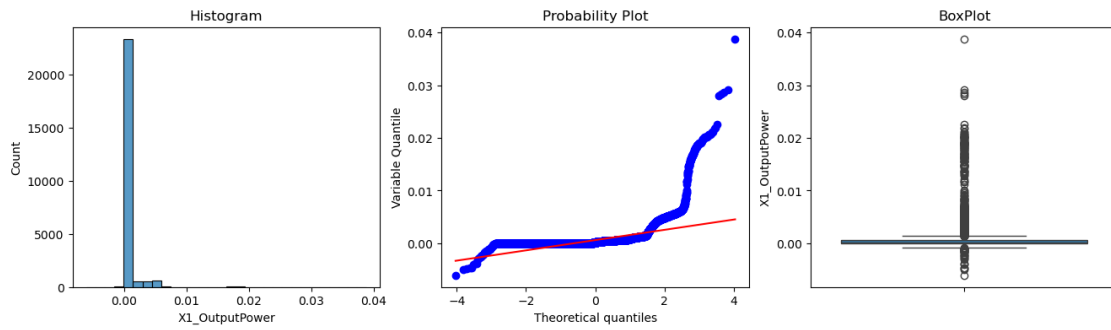
Distribution all Variables

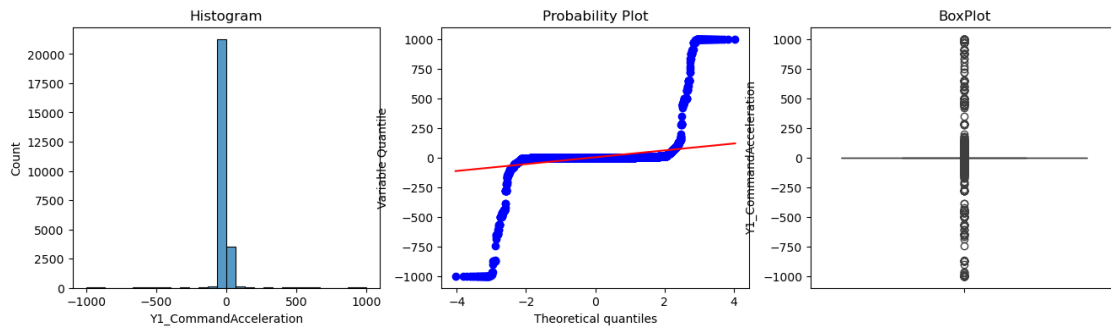
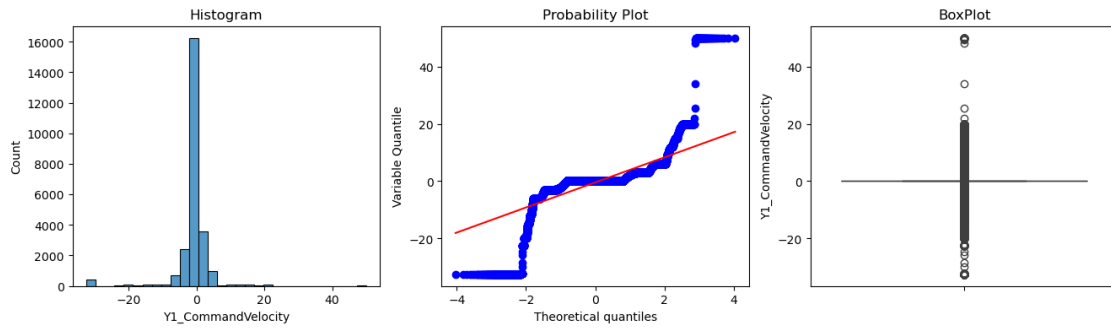
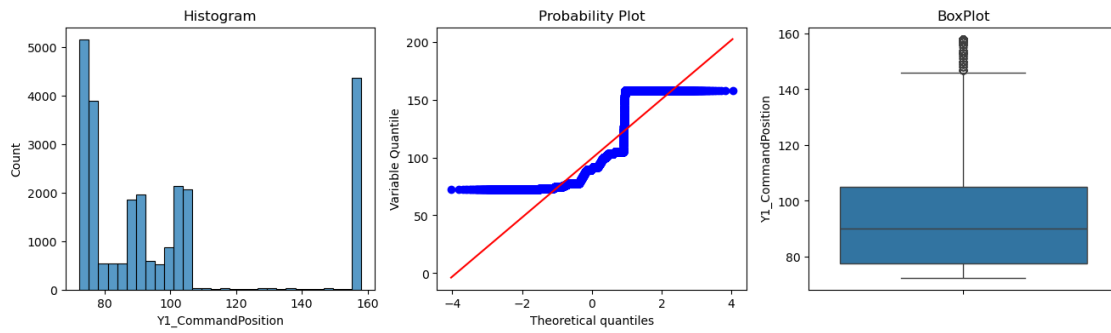
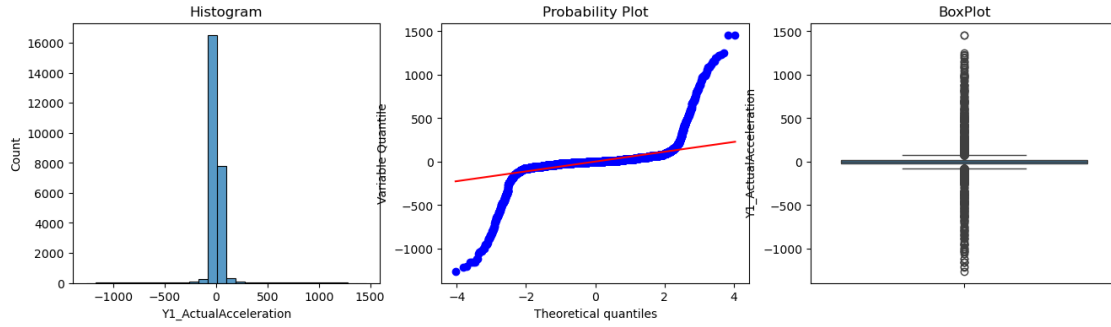
```
[18]: for var in tmp.columns:
        diagnostic_plots(tmp, var)
```

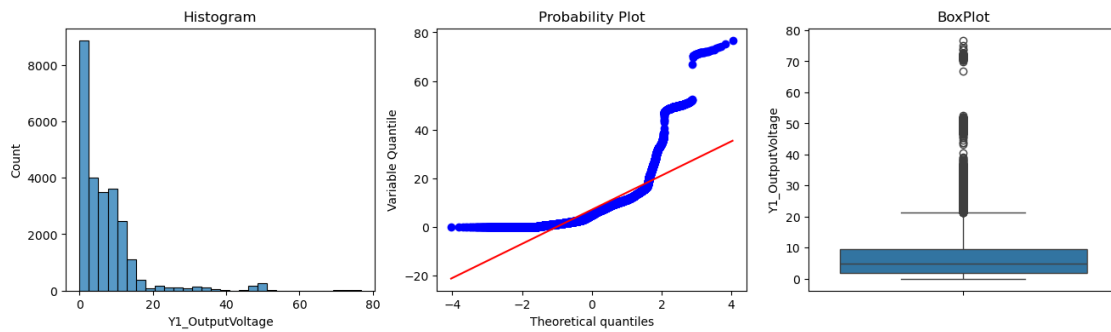
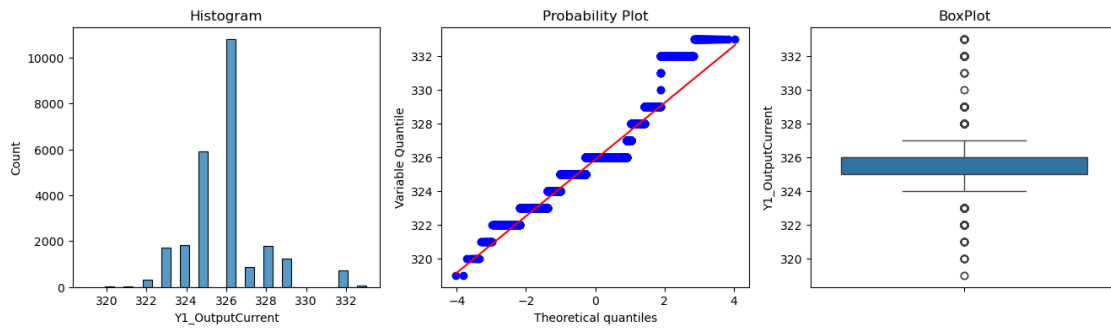
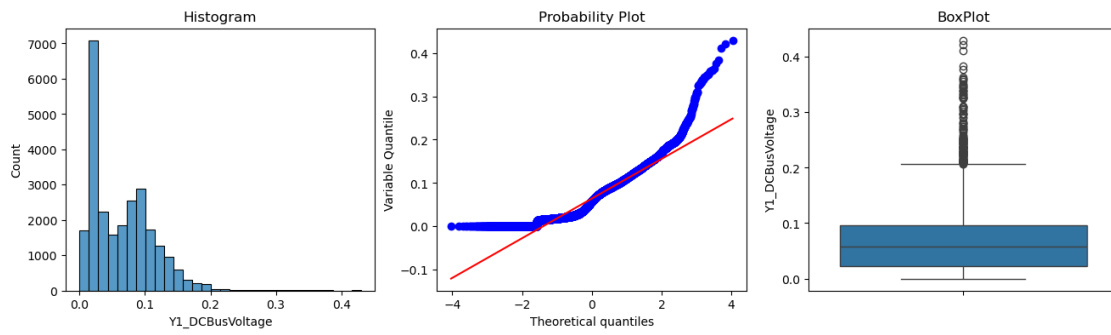
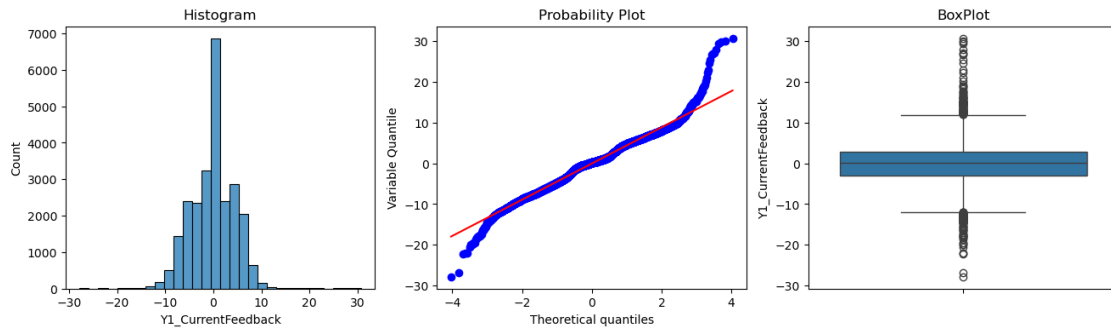


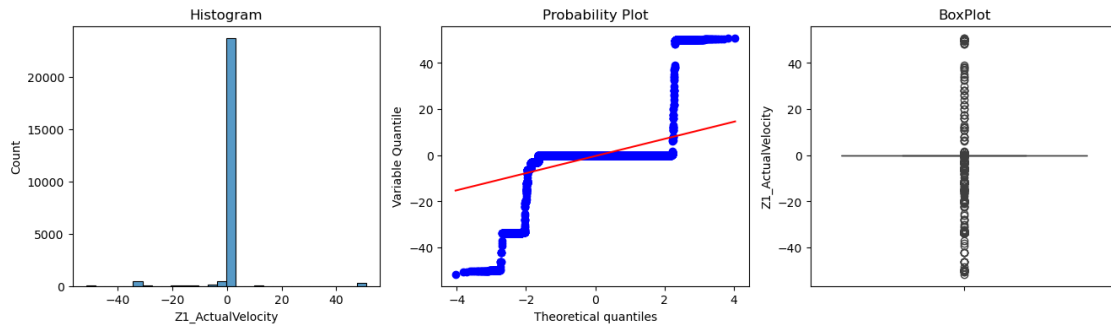
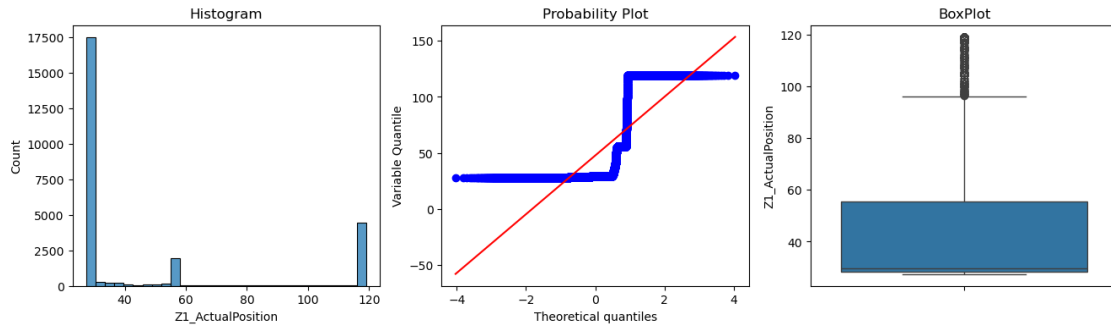
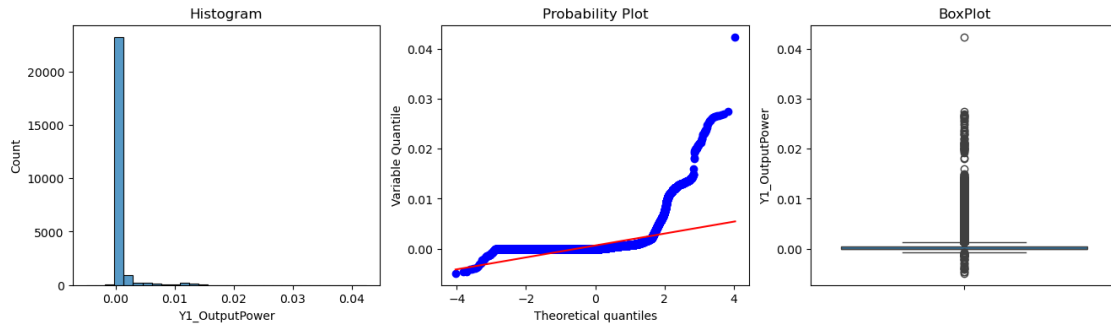


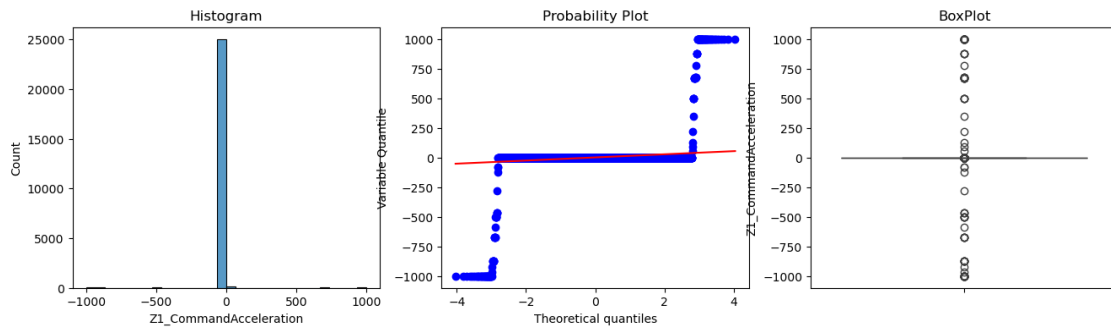
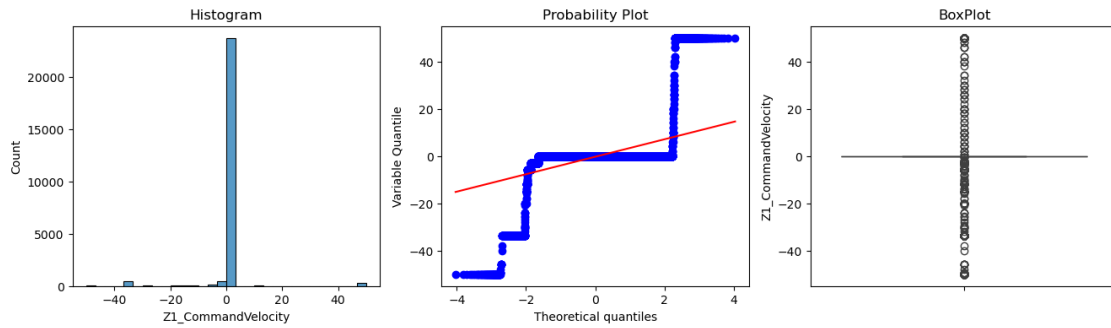
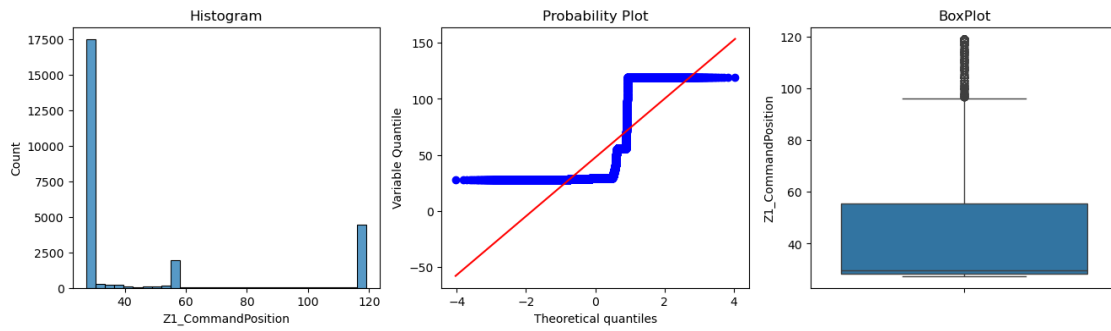
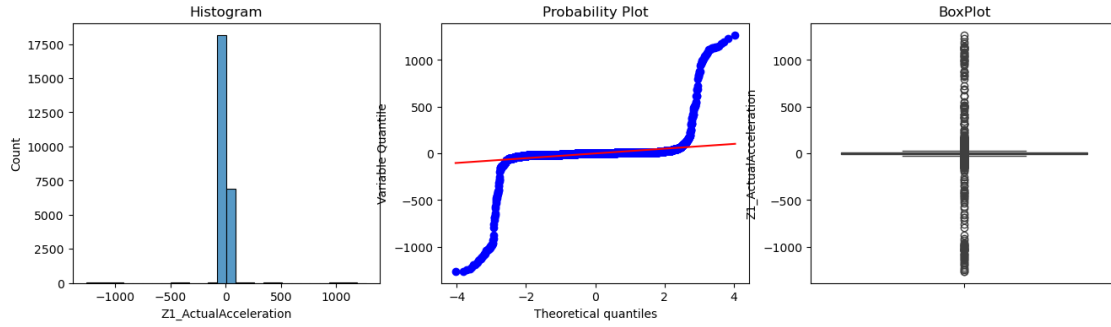


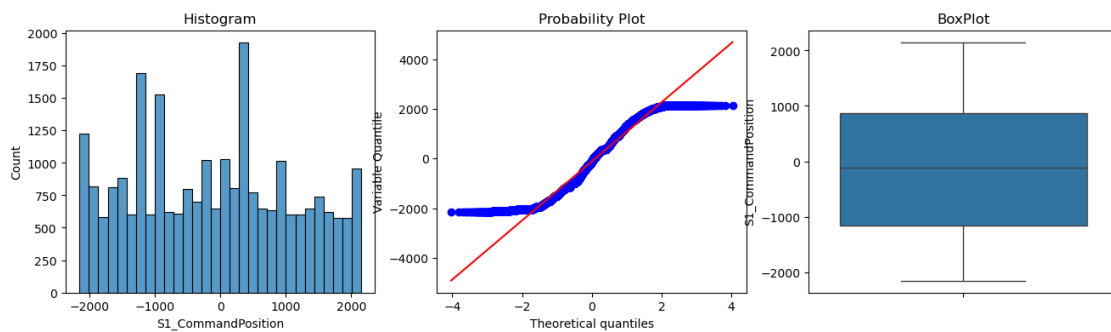
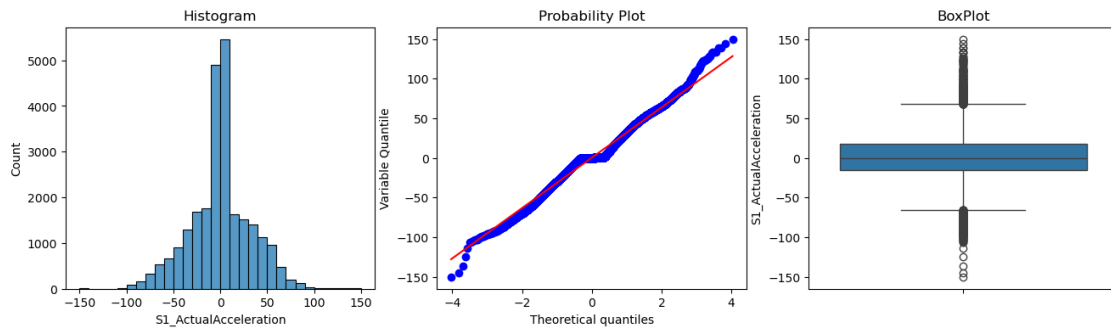
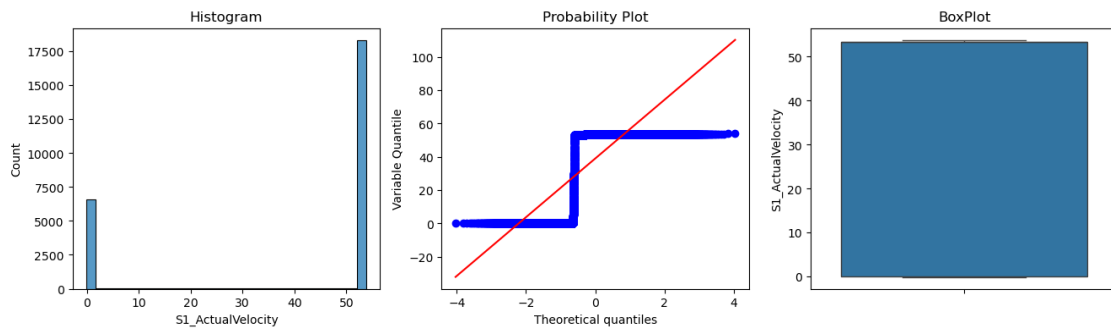
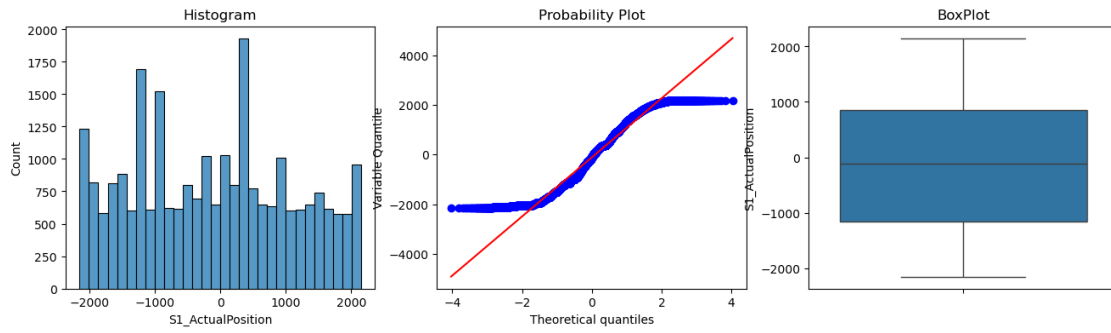


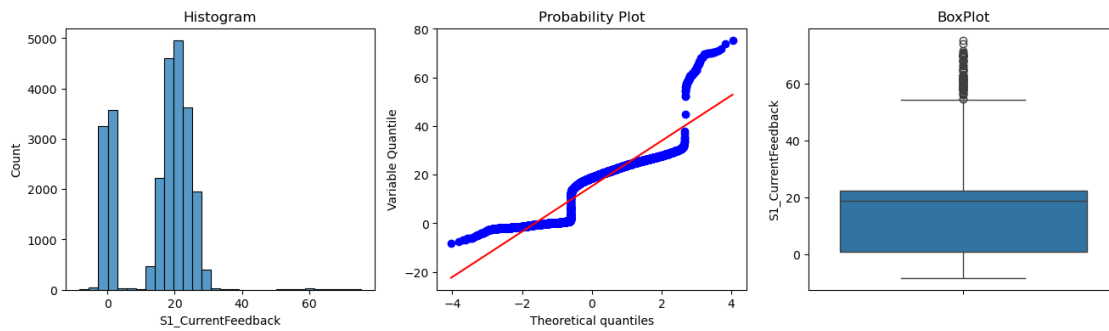
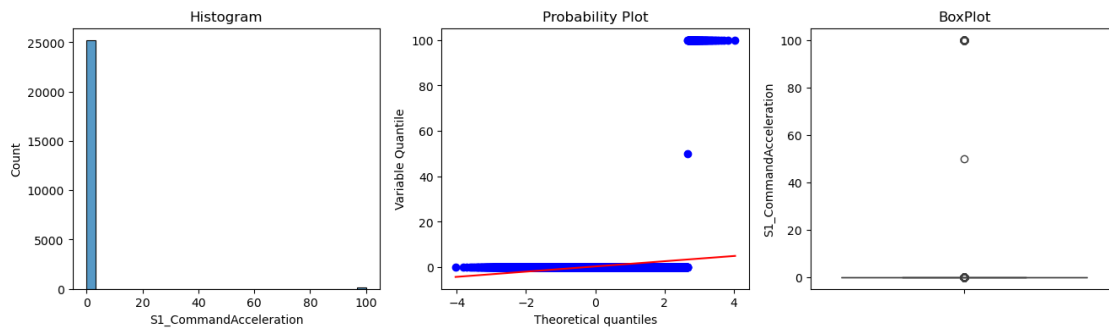
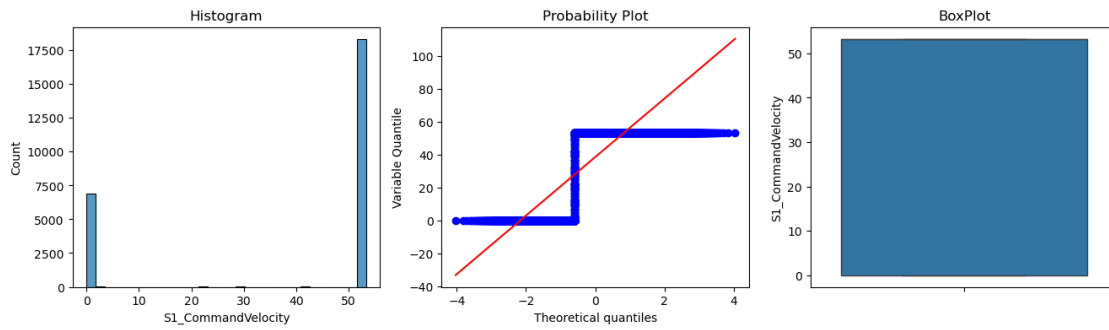


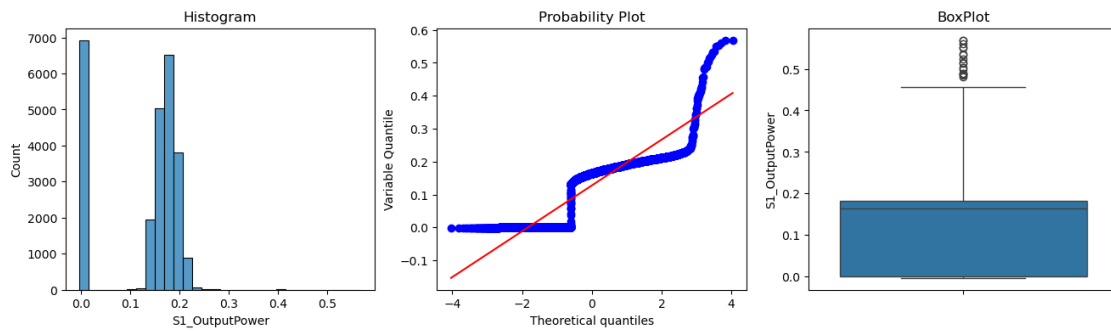
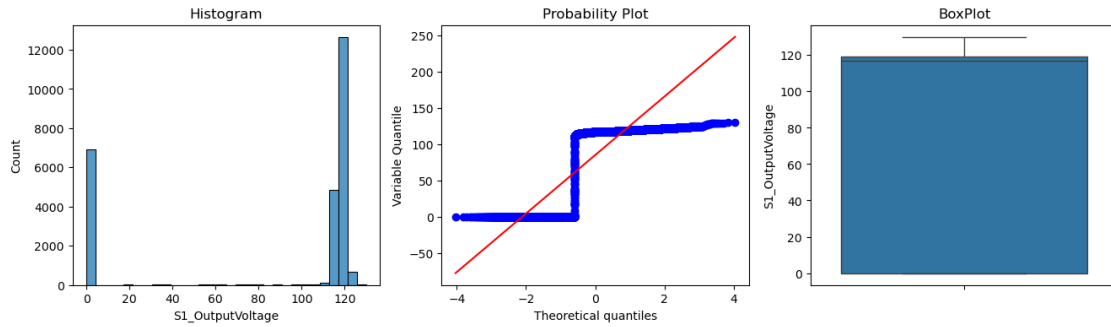
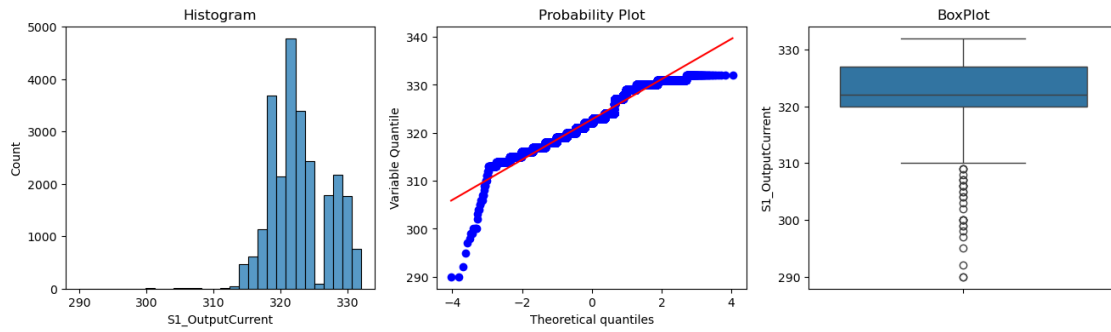
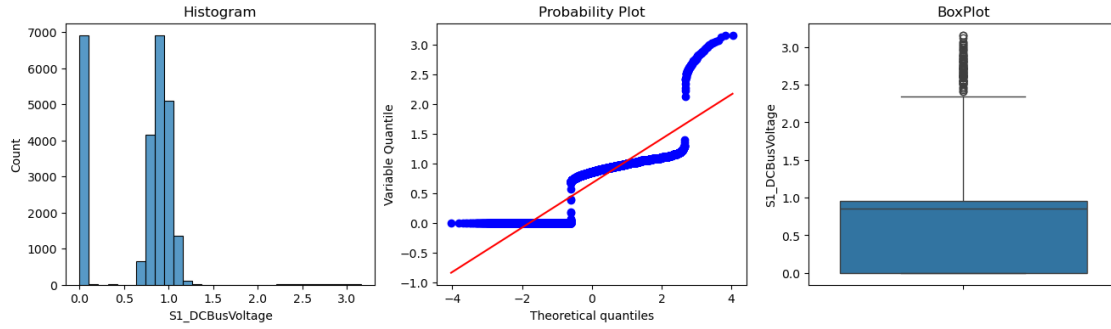


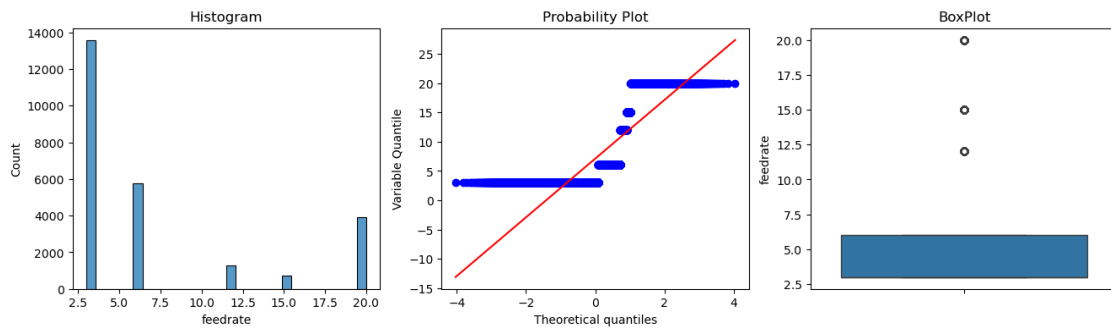
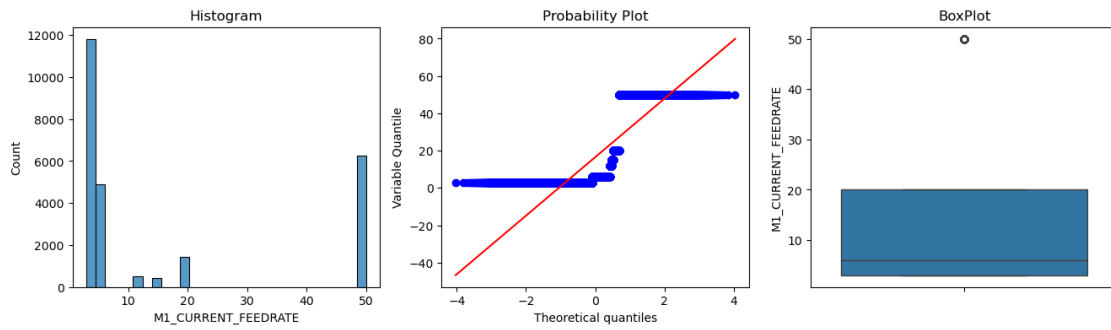
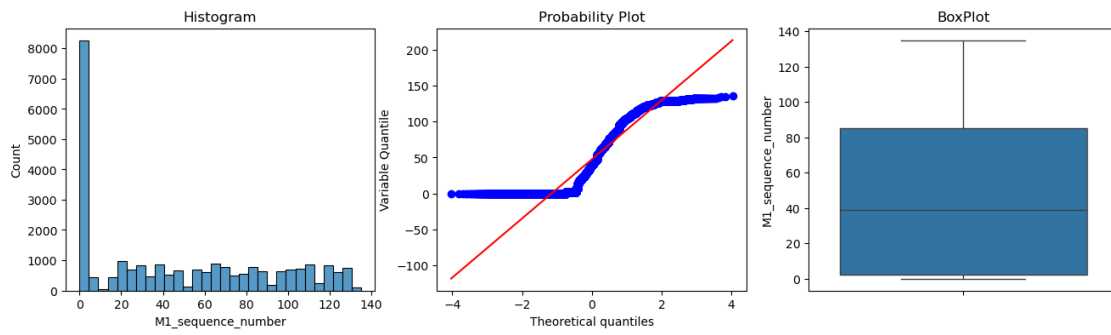
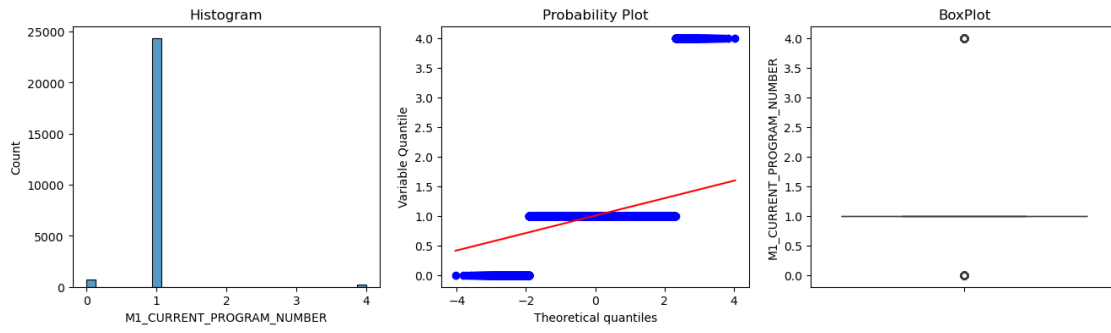


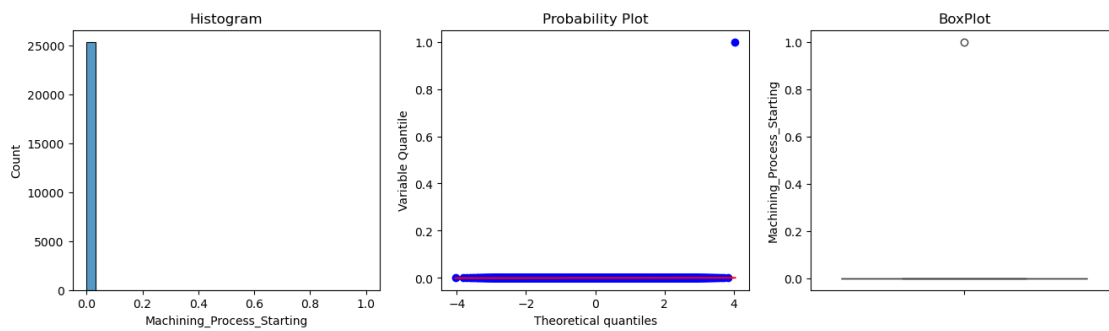
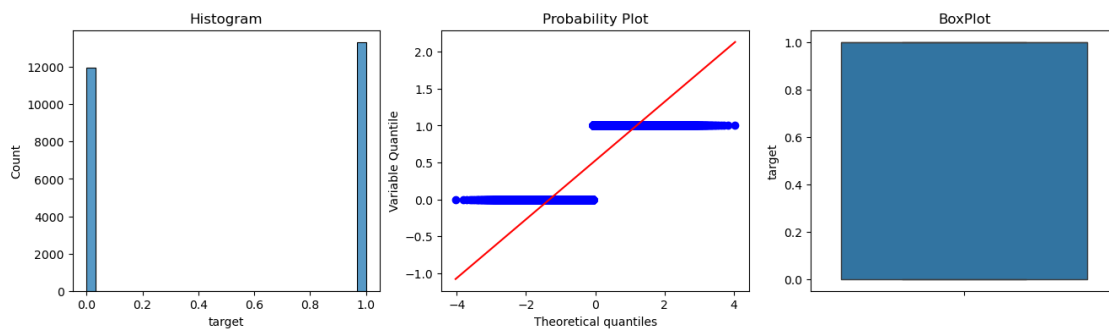
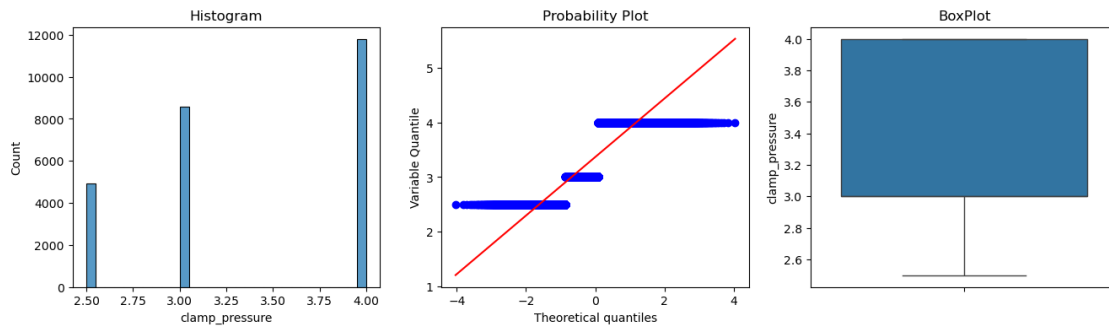


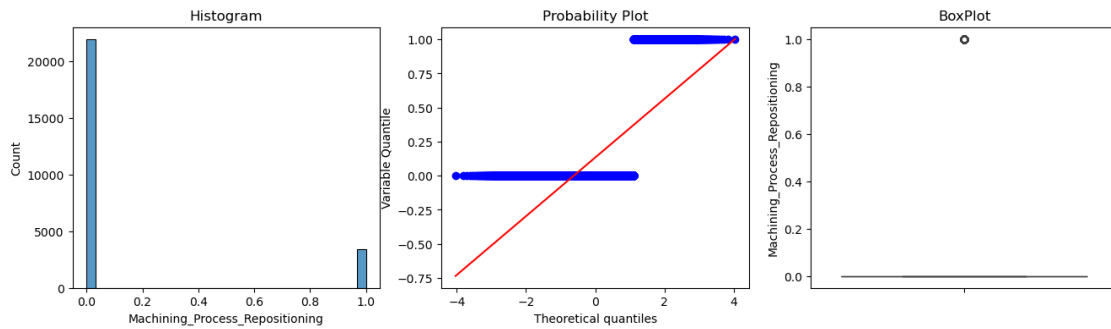
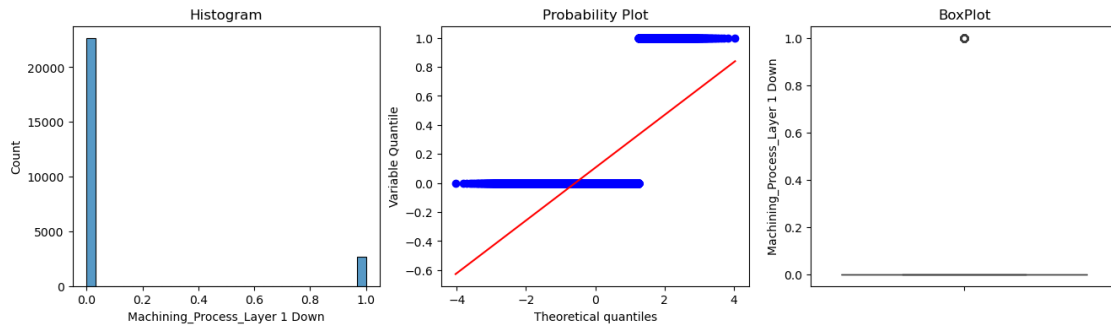
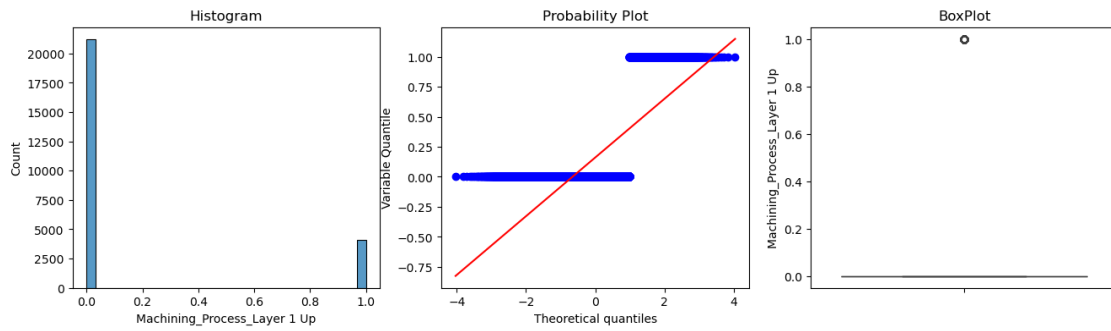
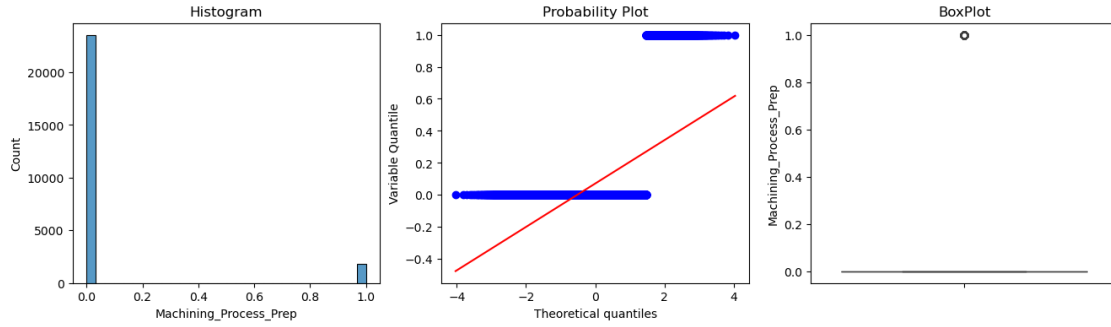


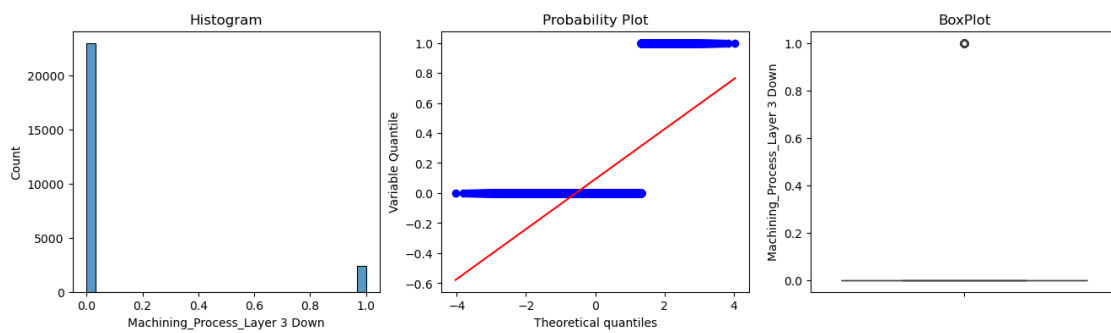
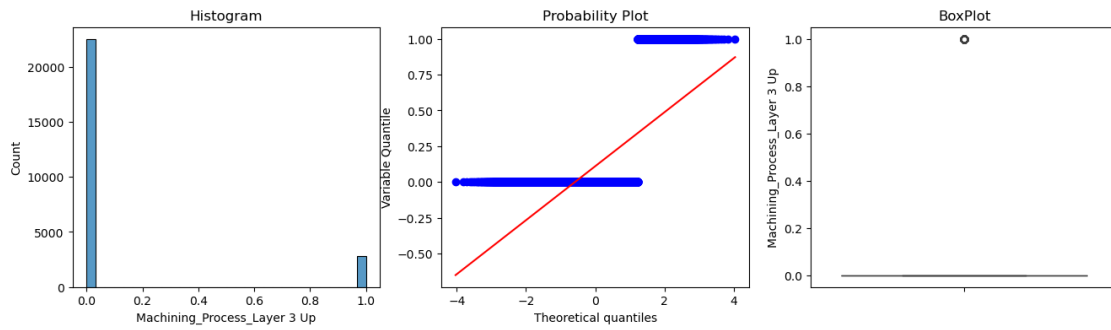
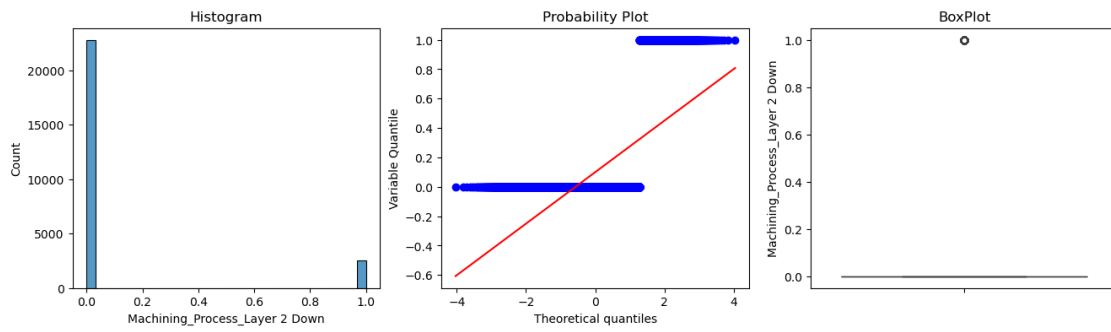
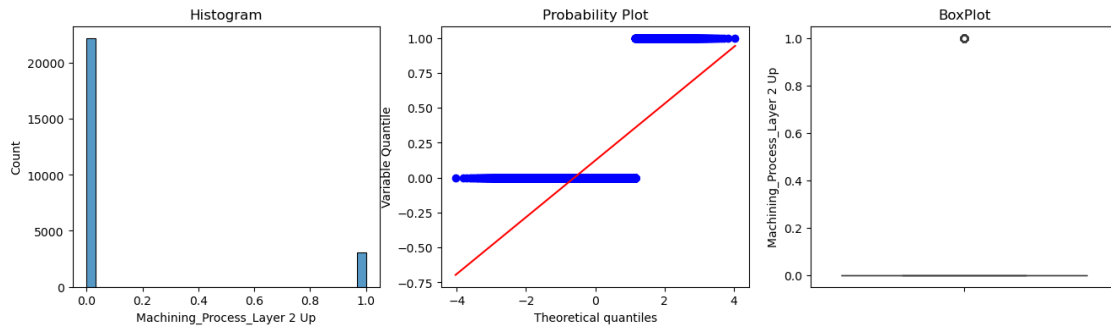


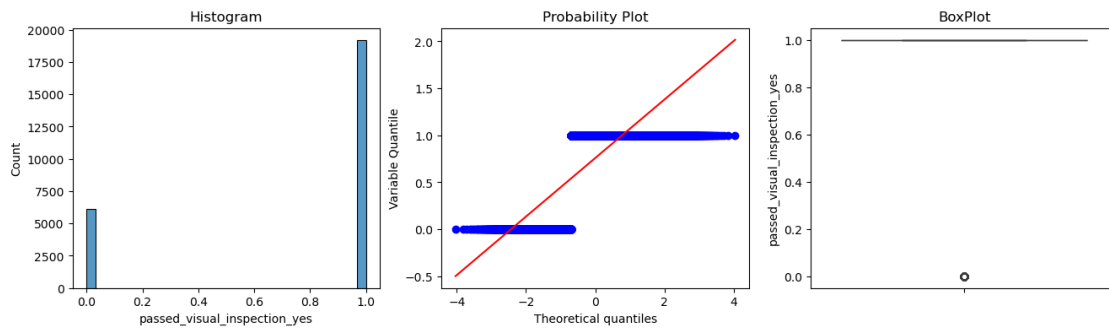
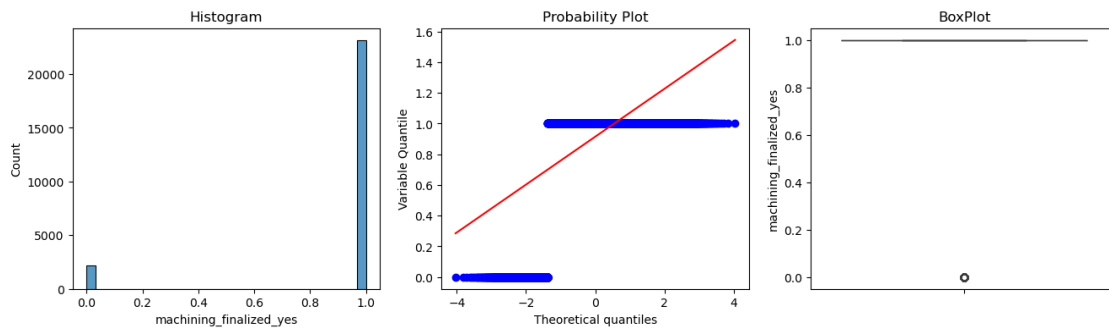
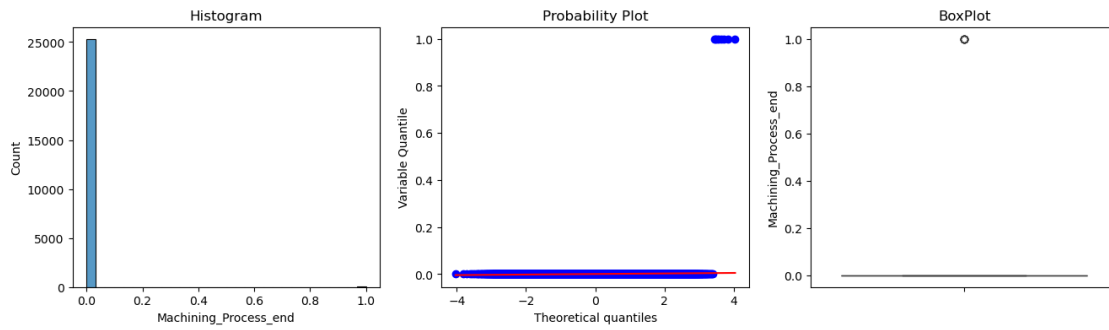


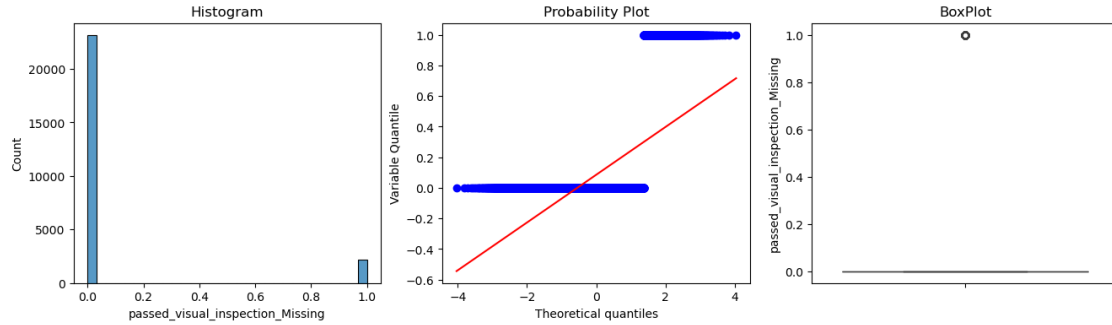






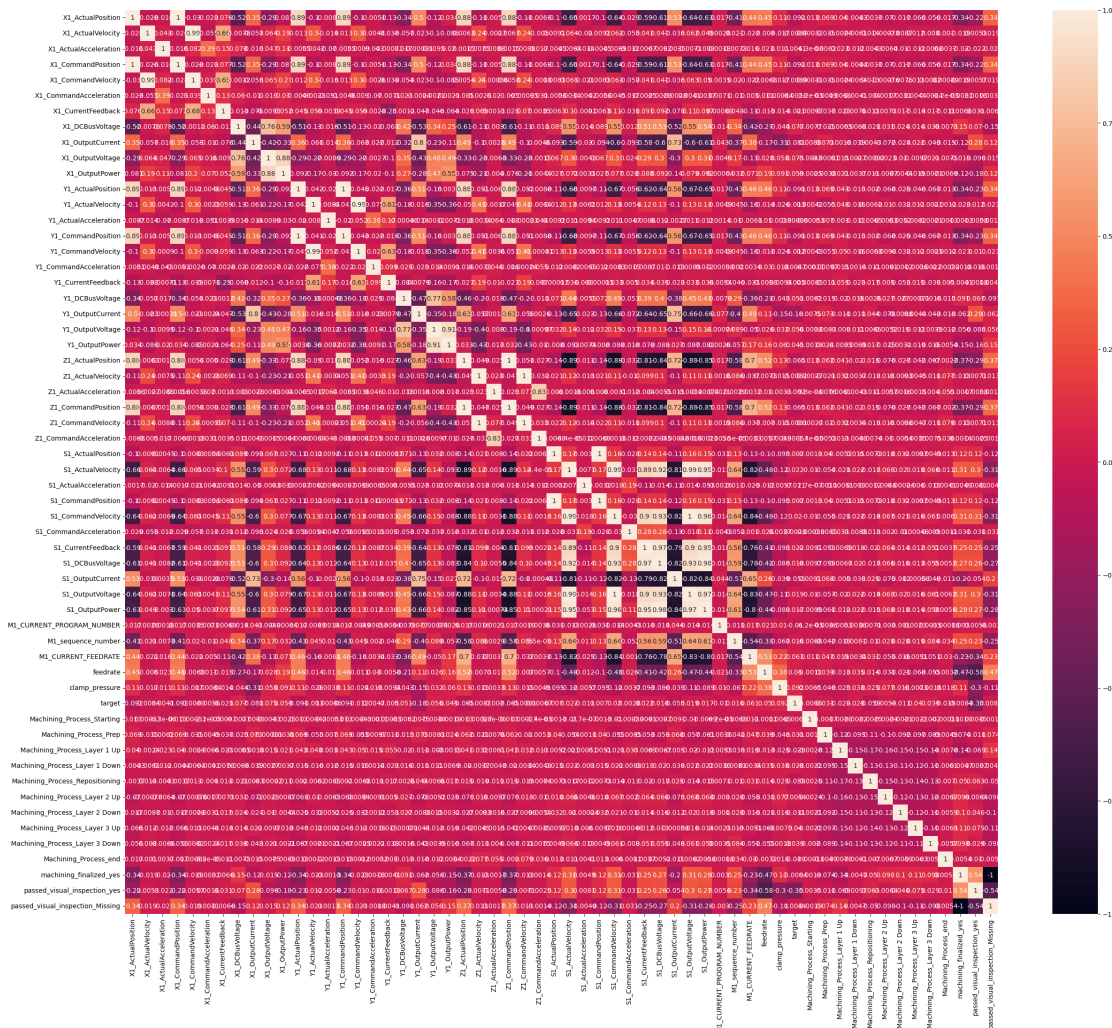






1.5 3.2 Correlation Matrix

```
[19]: corr_matrix = tmp.corr
plt.figure(figsize = (30, 25))
p = sns.heatmap(corr_matrix(), annot = True)
```



Build the function for plot dependences on Target

```
[22]: def plot_tool_condition_counts(df, target='target', max_unique=20):
    """
        target.
        : stacked barplot.
        : boxplot violinplot.
    """

    columns_to_group = [col for col in df.columns if col != target]

    for column in columns_to_group:
        plt.figure(figsize=(8, 4))
        plt.tight_layout()

        unique_count = df[column].nunique()

        if unique_count <= max_unique:
            grouped_counts = df.groupby([column, target]).size().
            ↪unstack(fill_value=0)
            grouped_counts.plot(
                kind='bar',
                stacked=True,
                color=['#1f77b4', '#ff7f0e'],
                alpha=0.8,
                figsize=(8, 4)
            )
            plt.title(f'{column} grouped by {target}')
            plt.xlabel(column)
            plt.ylabel('Count')
            plt.legend(title='ToolCondition', labels=['unworn', 'worn'])
            plt.xticks(rotation=45, ha='right')

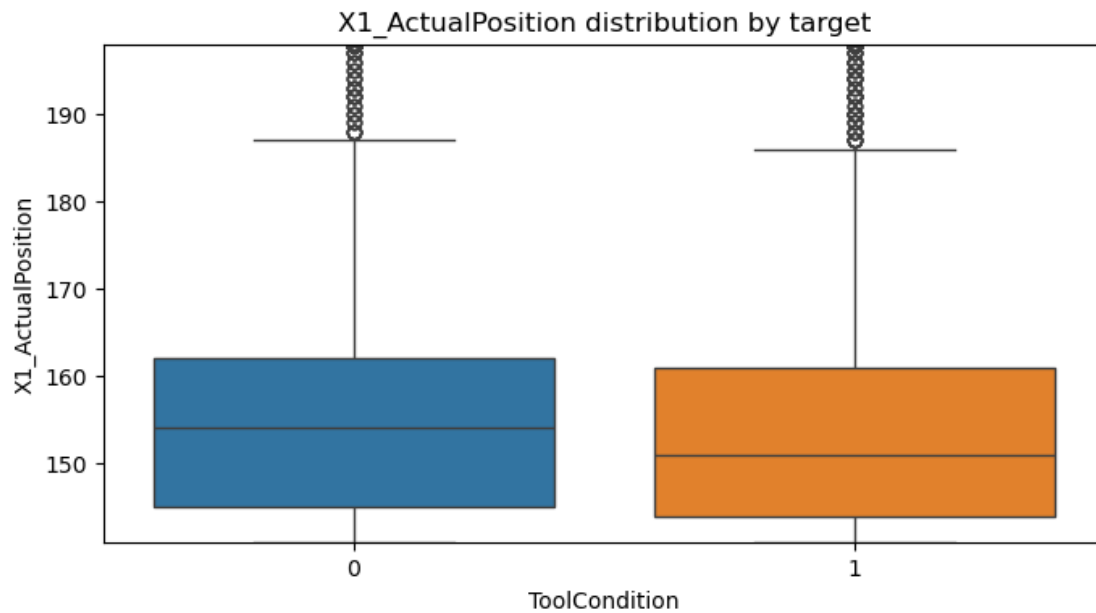
        else:
            sns.boxplot(
                x=target, y=column, data=df,
                hue=target, dodge=False,
                palette=['#1f77b4', '#ff7f0e'],
                legend=False
            )
            plt.title(f'{column} distribution by {target}')
            plt.xlabel('ToolCondition')
            plt.ylabel(column)
```

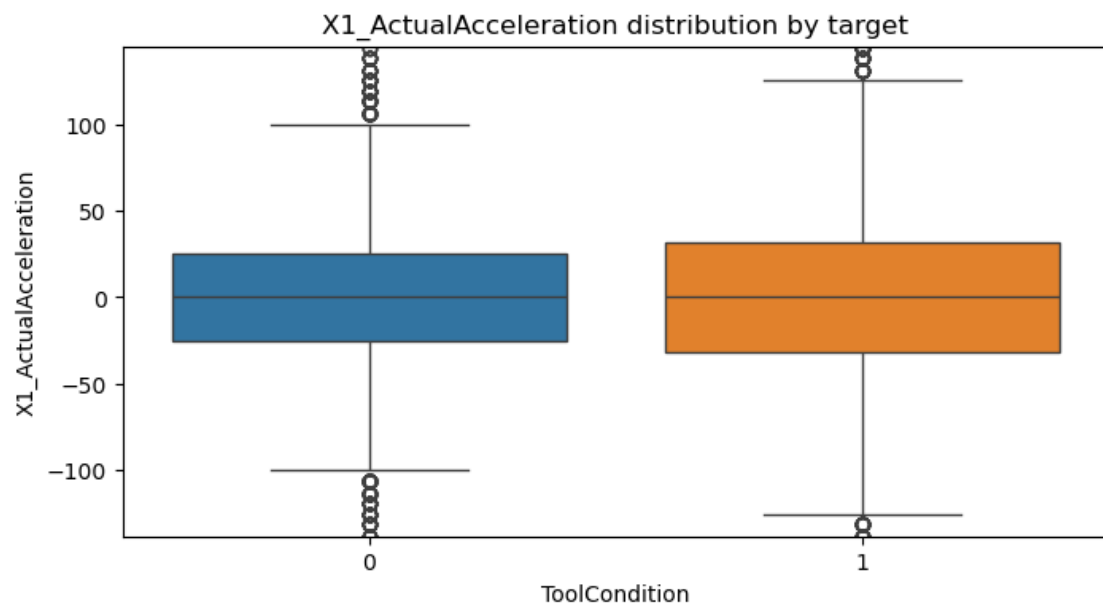
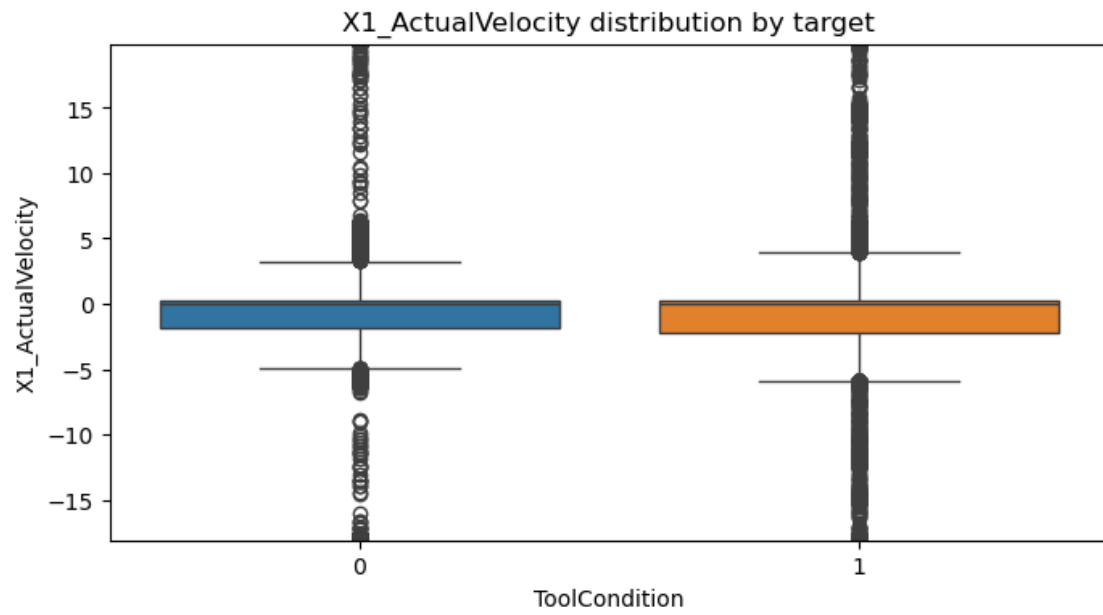
```
q_low, q_high = df[column].quantile([0.01, 0.99])
plt.ylim(q_low, q_high)
plt.legend([], [], frameon=False)

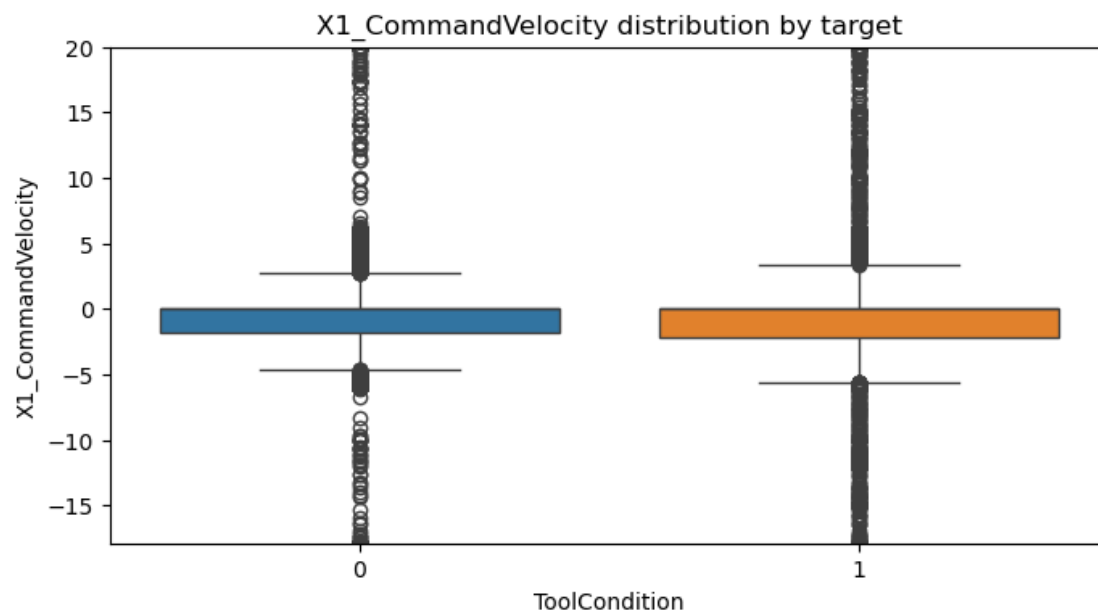
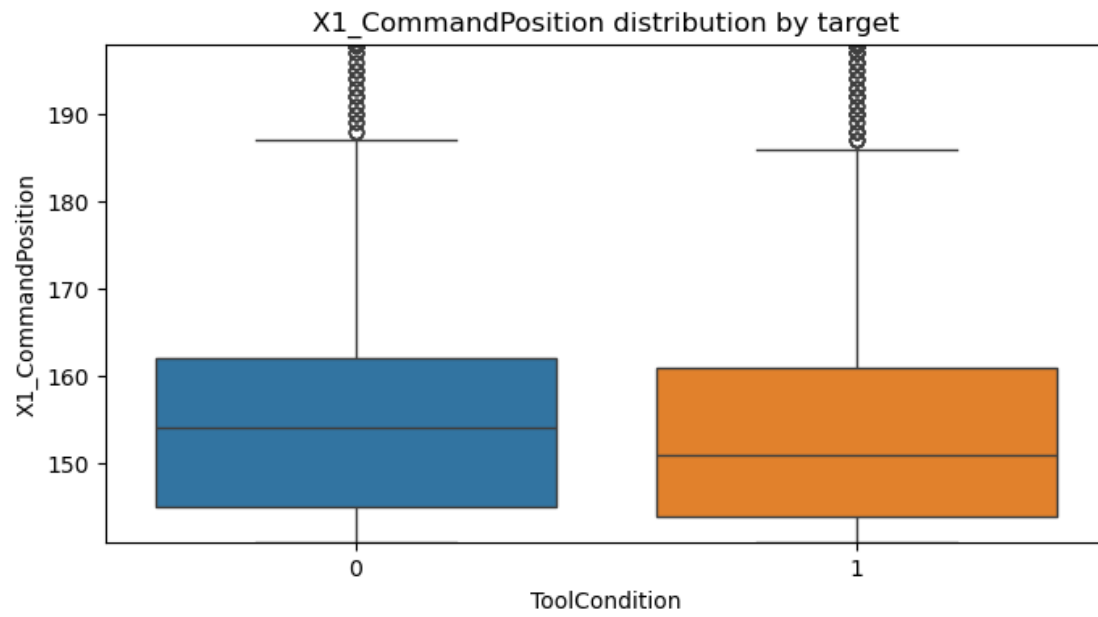
plt.show()
```

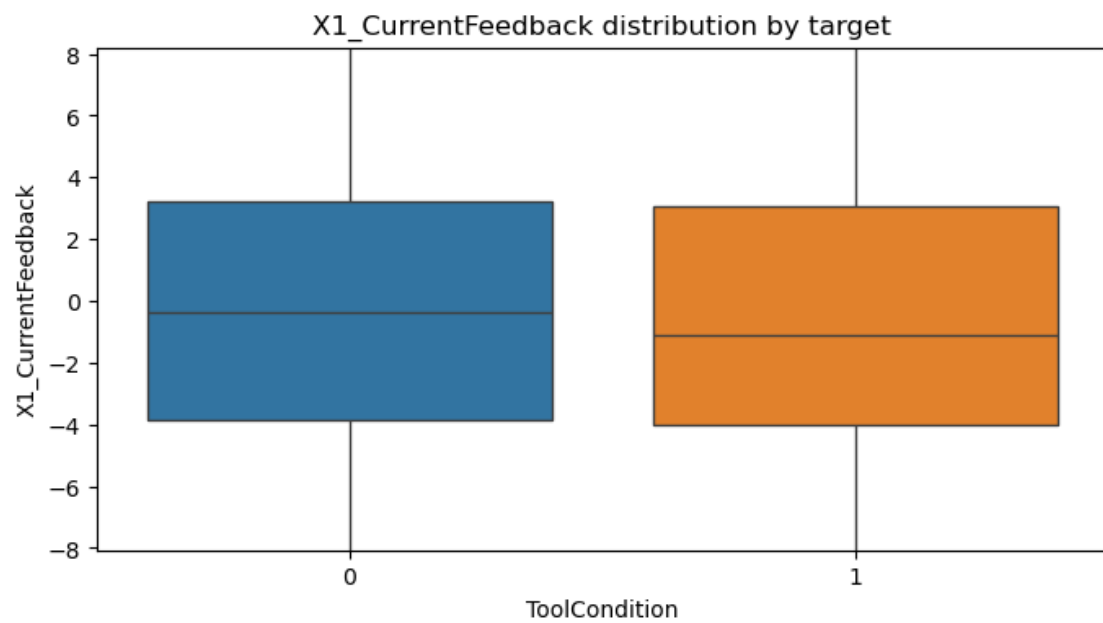
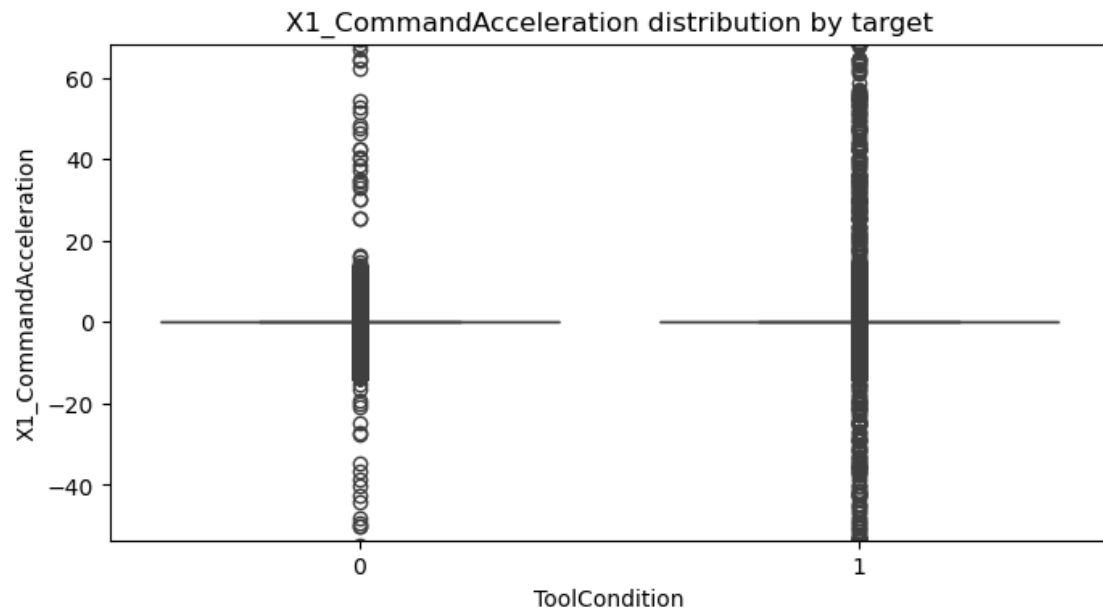
1.6 3.3 We look at the dependence of all variables on our Target

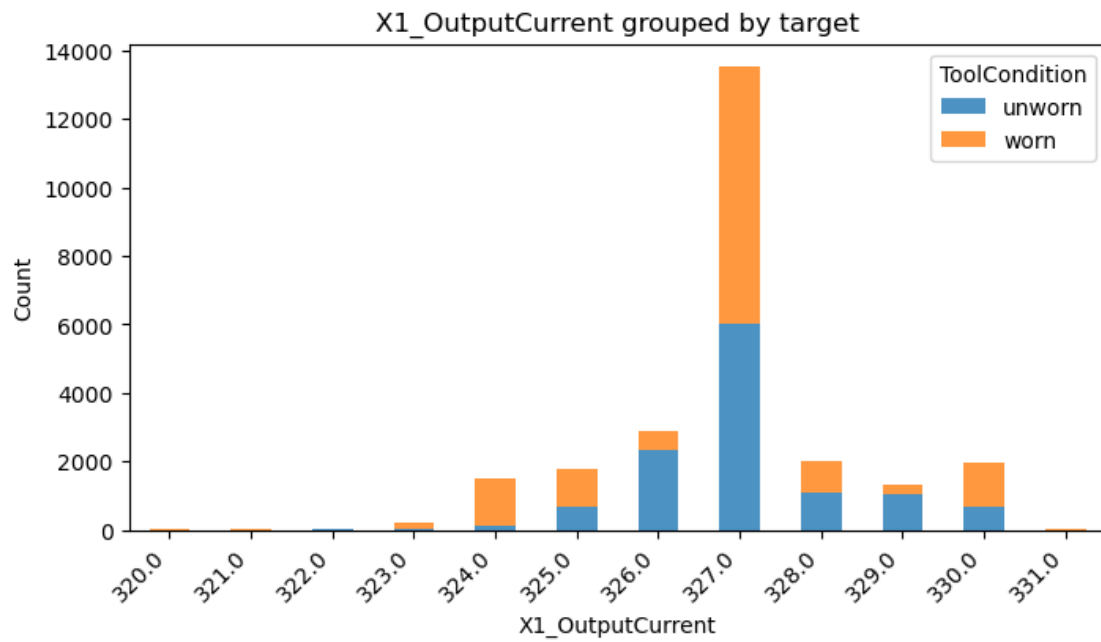
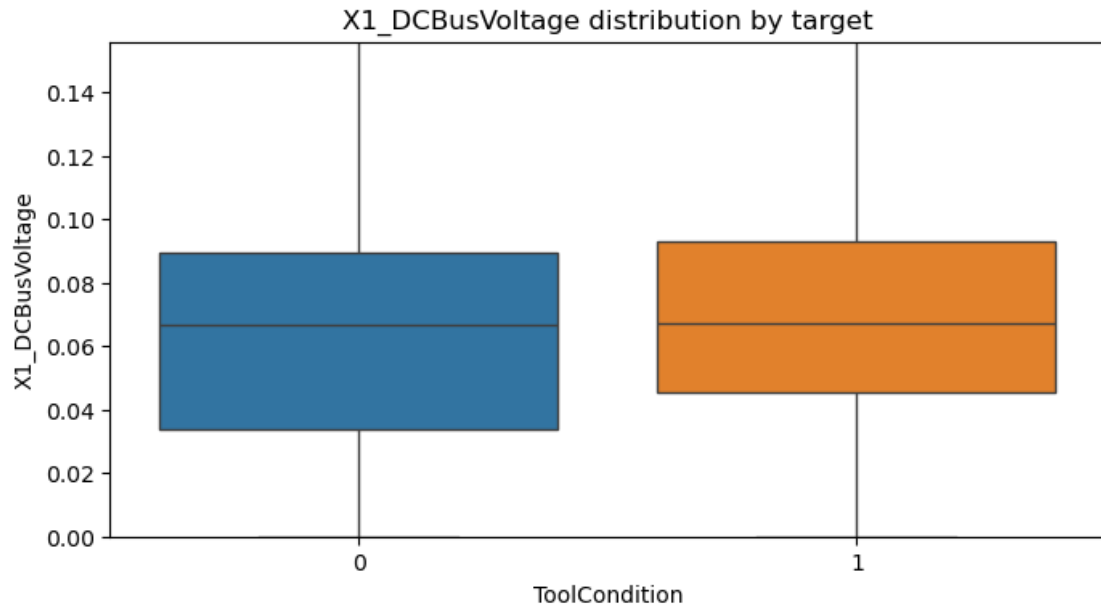
```
[23]: plot_tool_condition_counts(tmp)
```

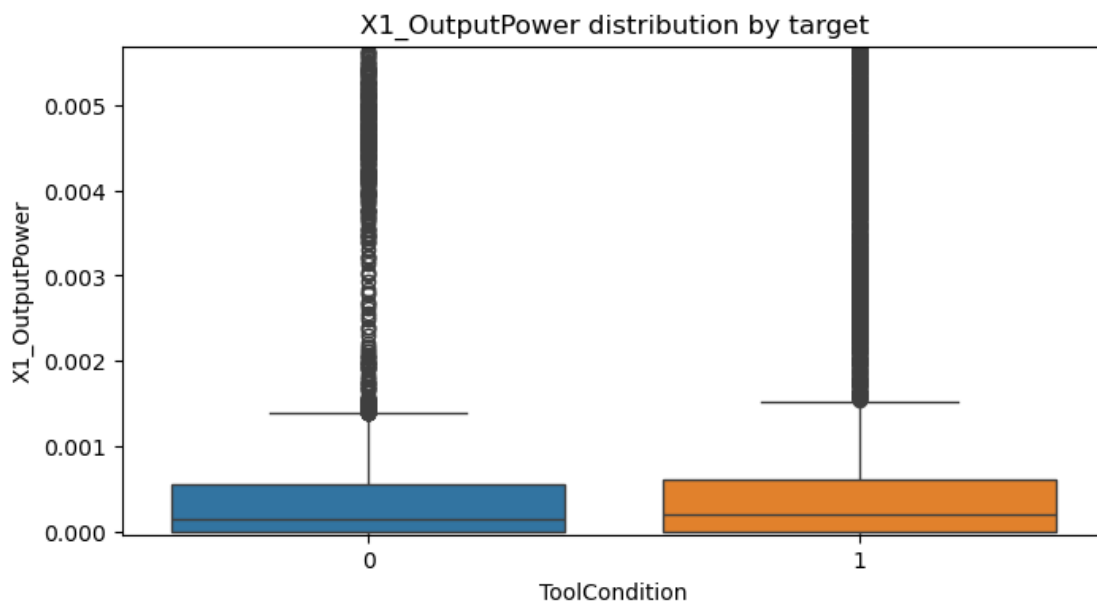
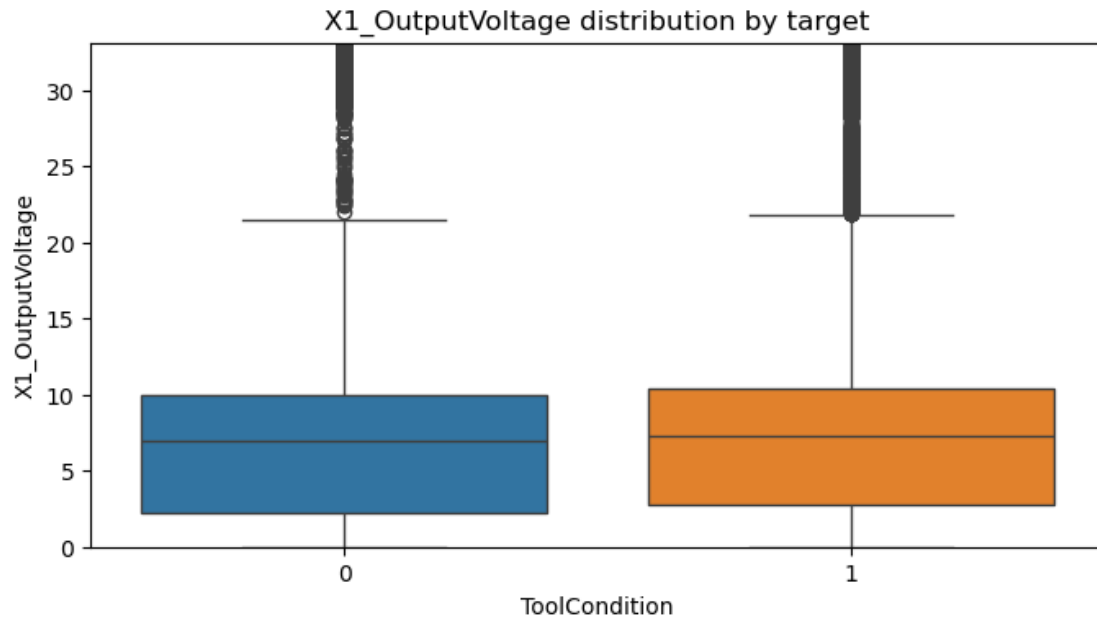


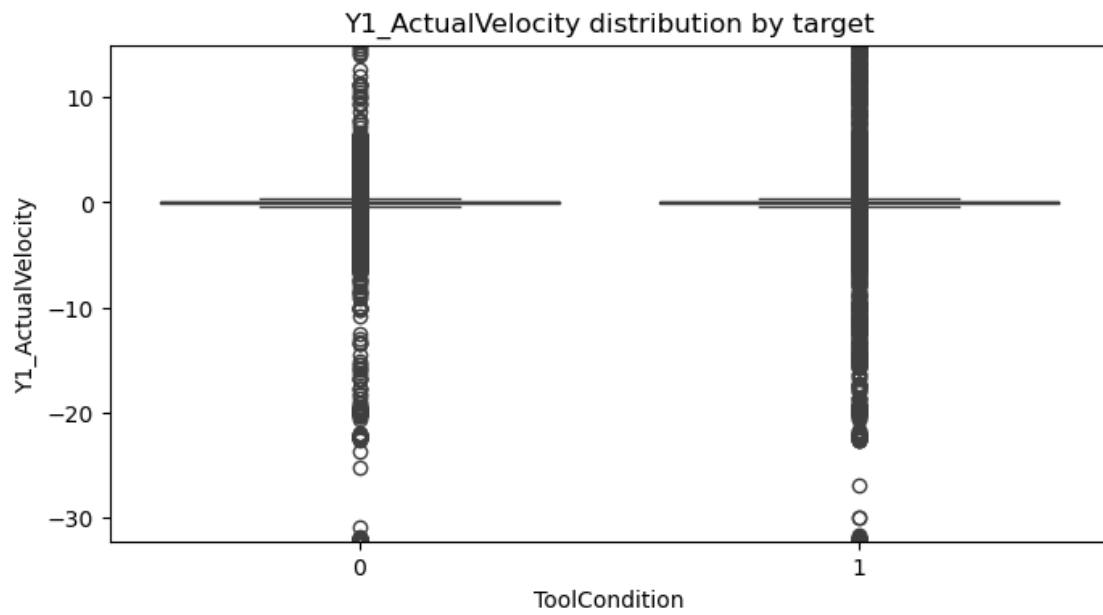
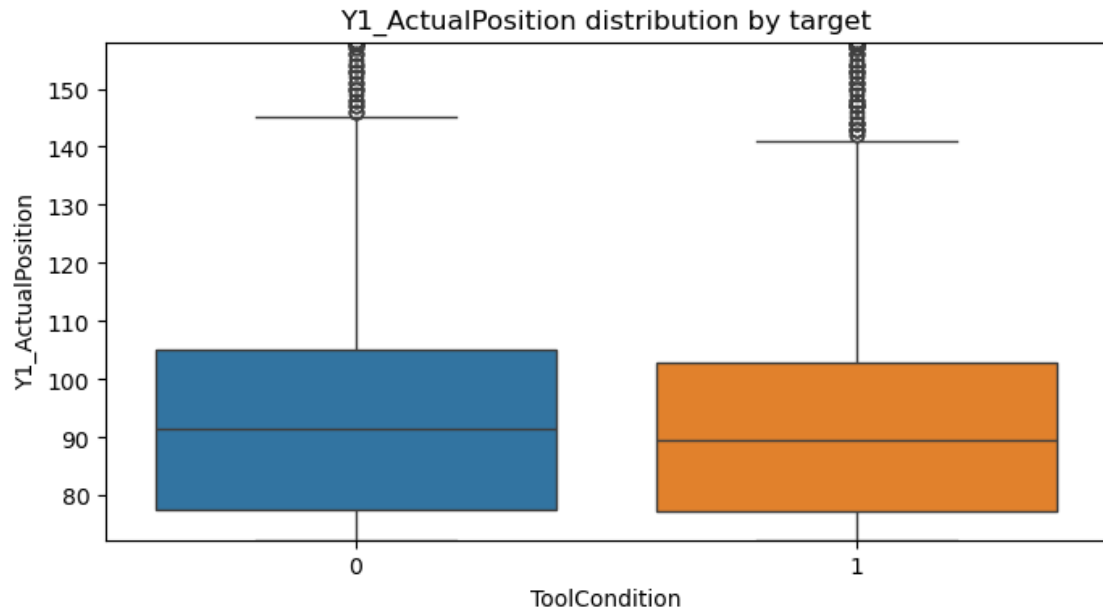


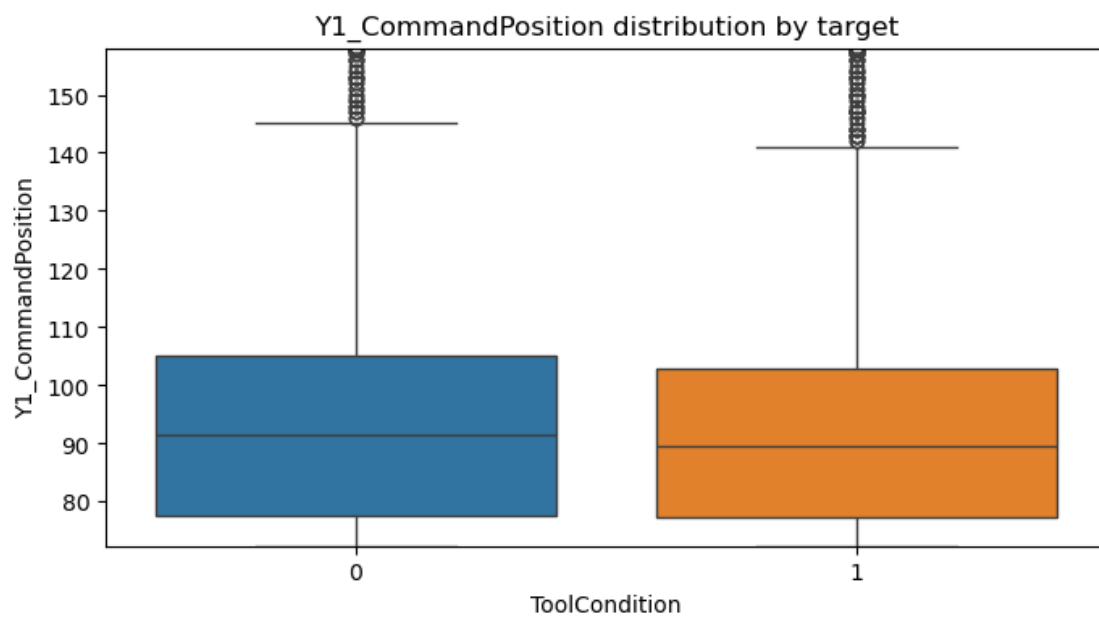
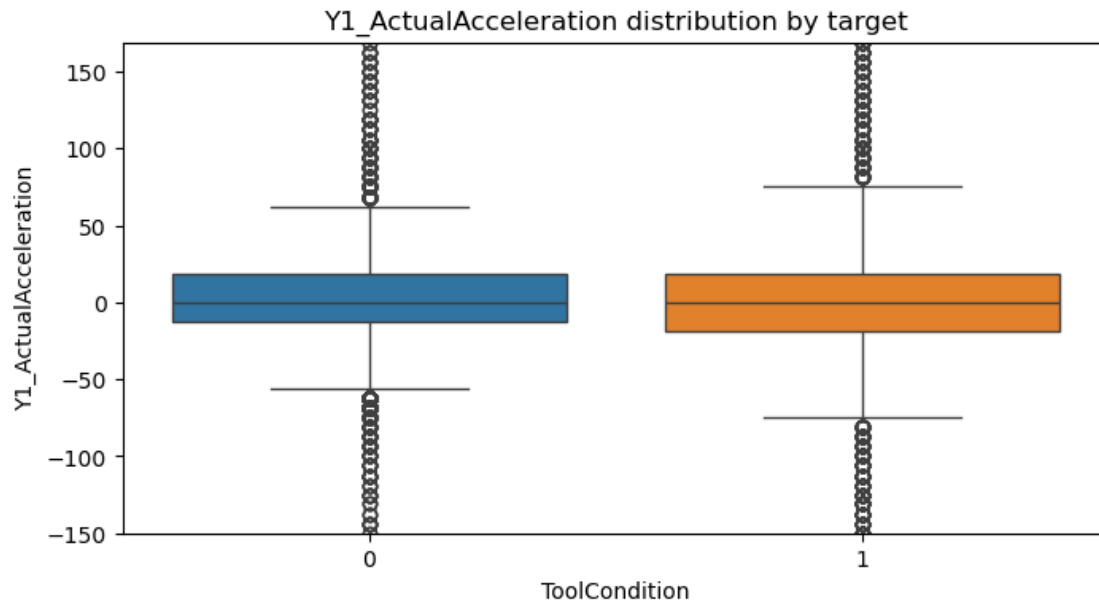


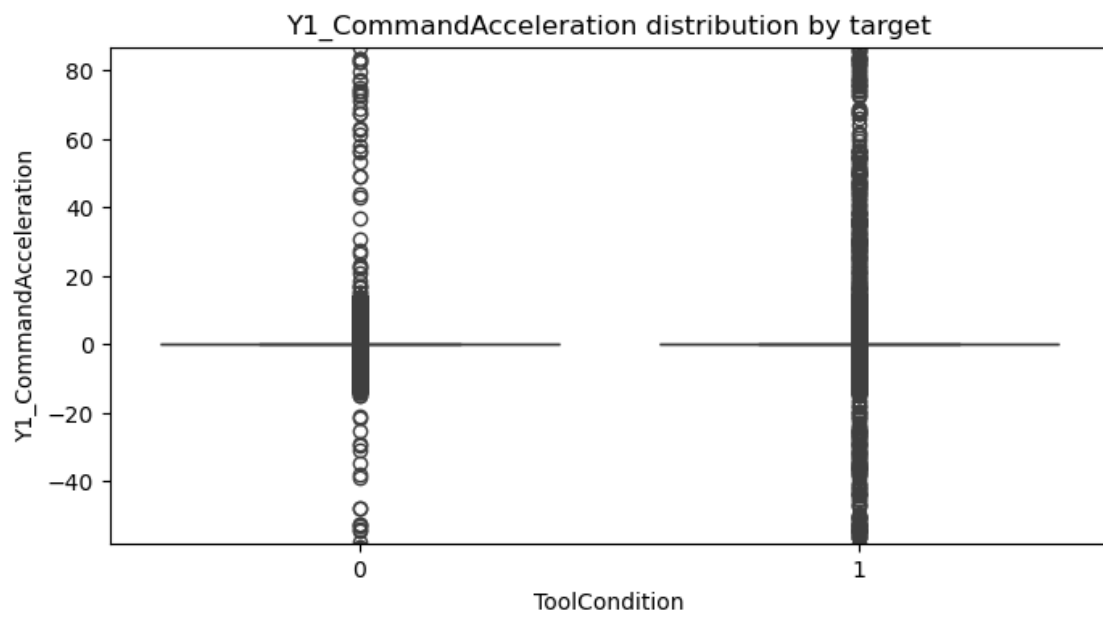
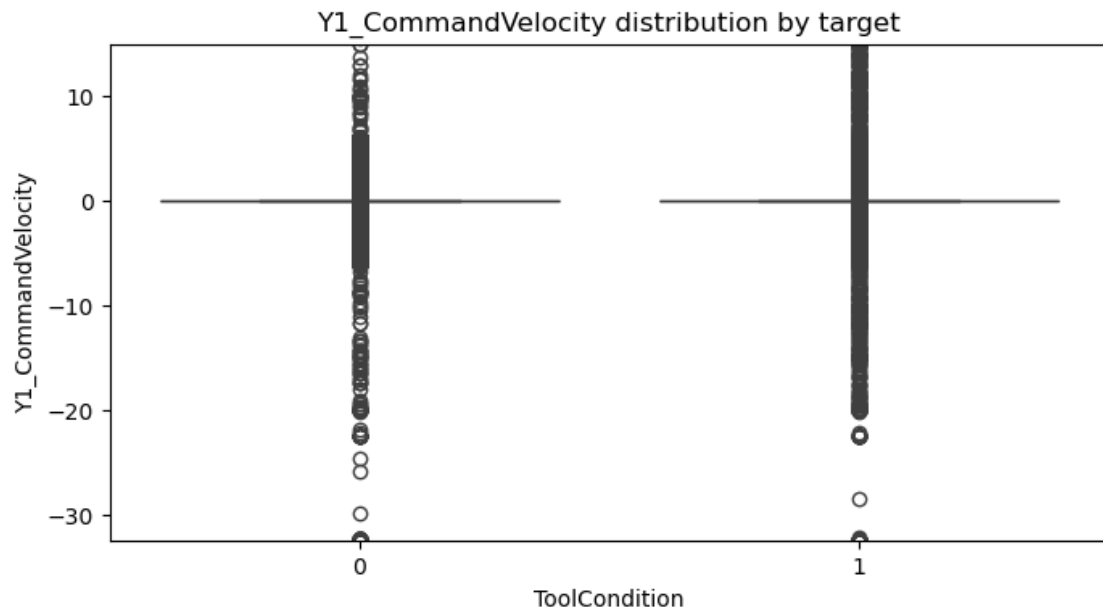


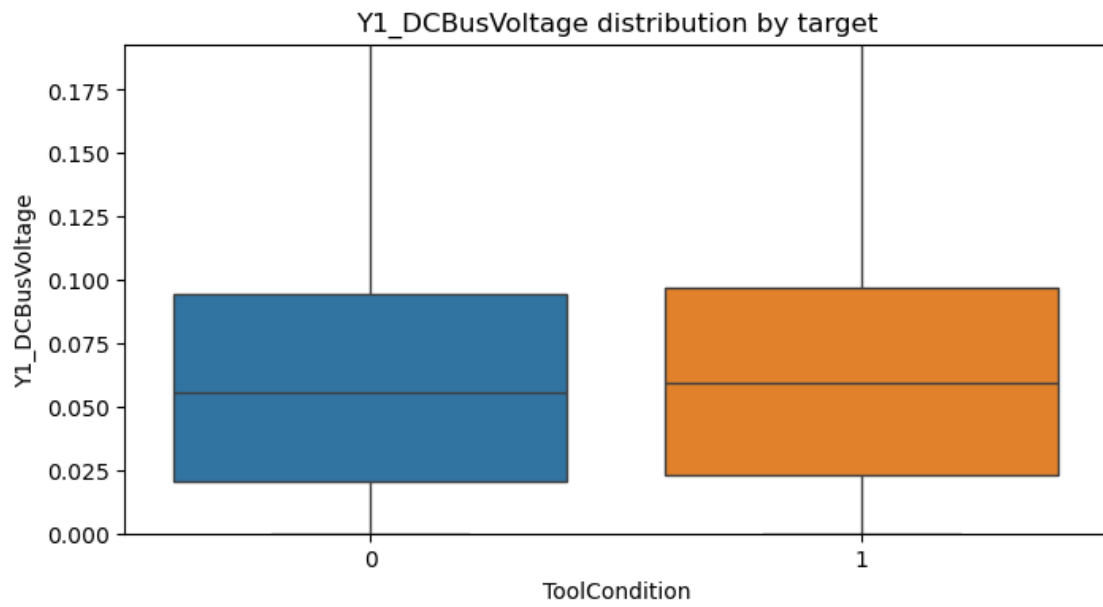
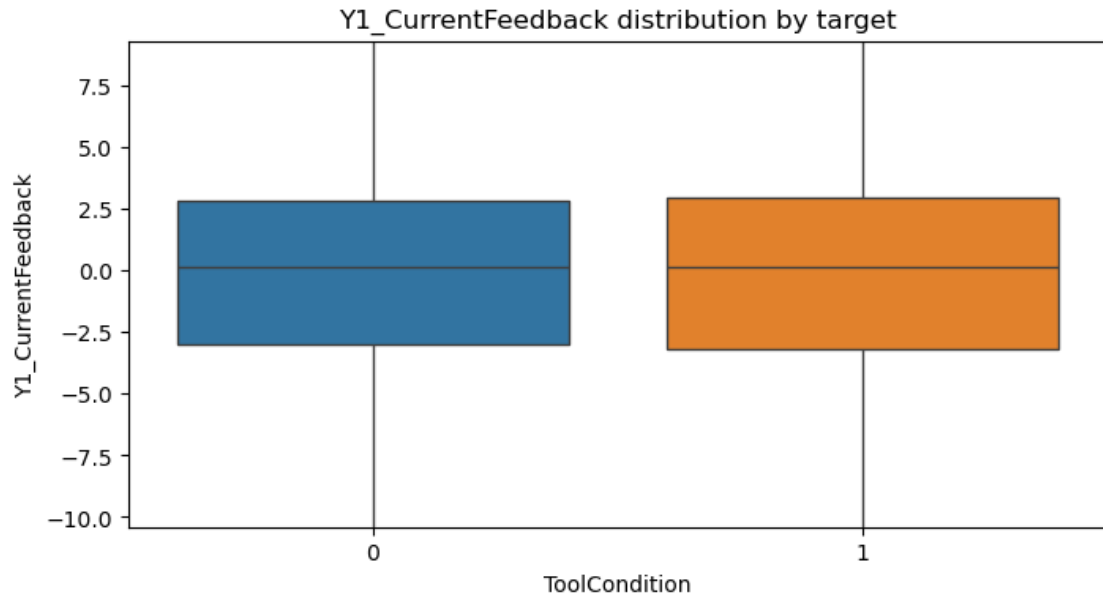


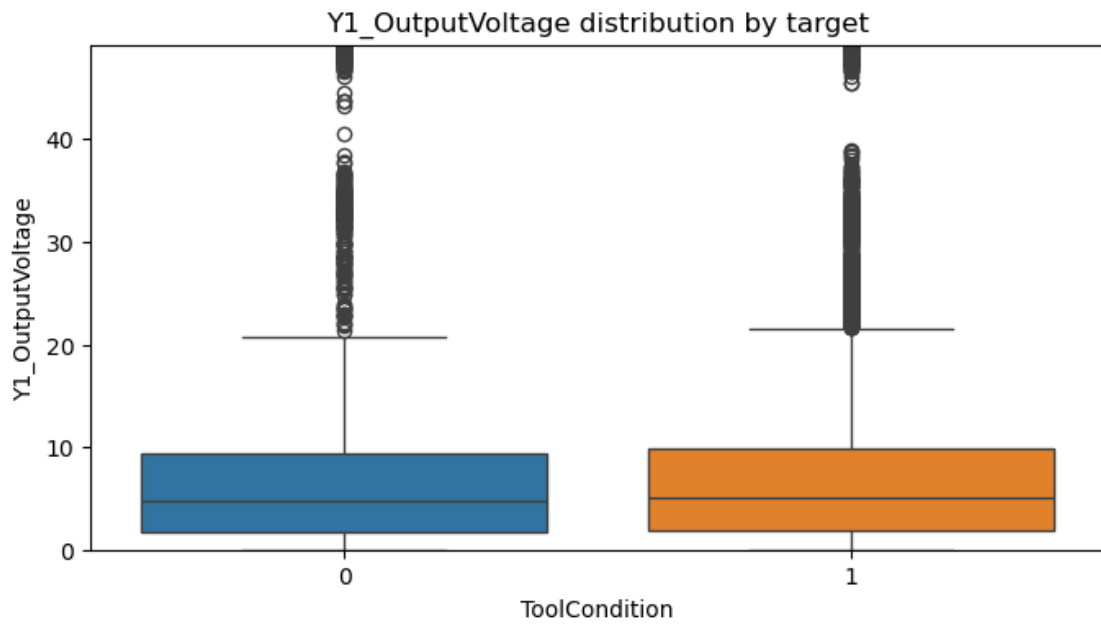
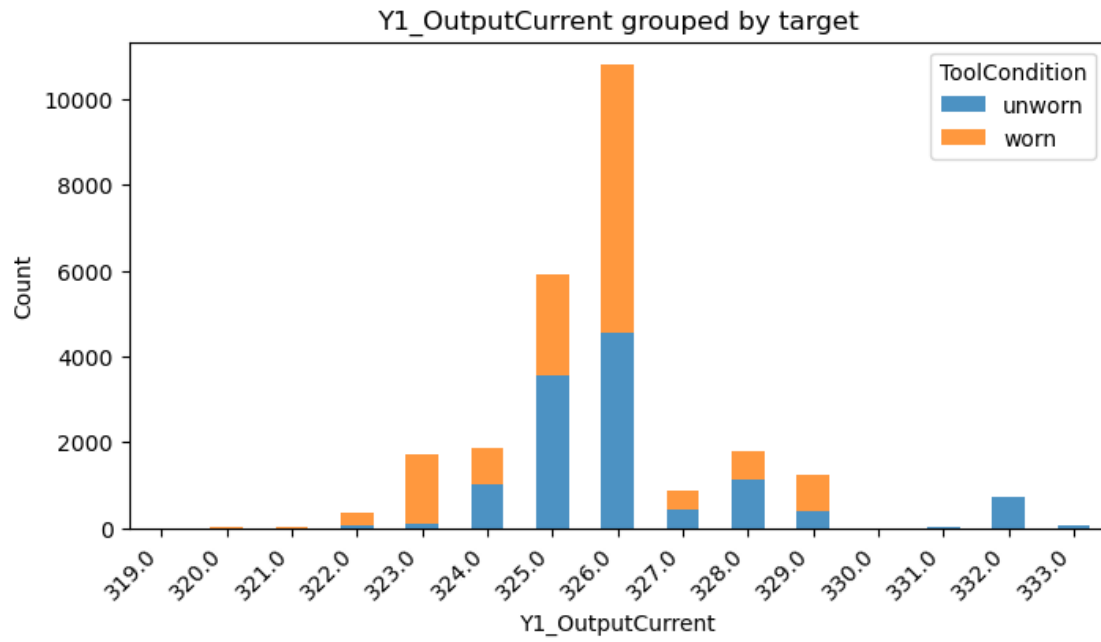


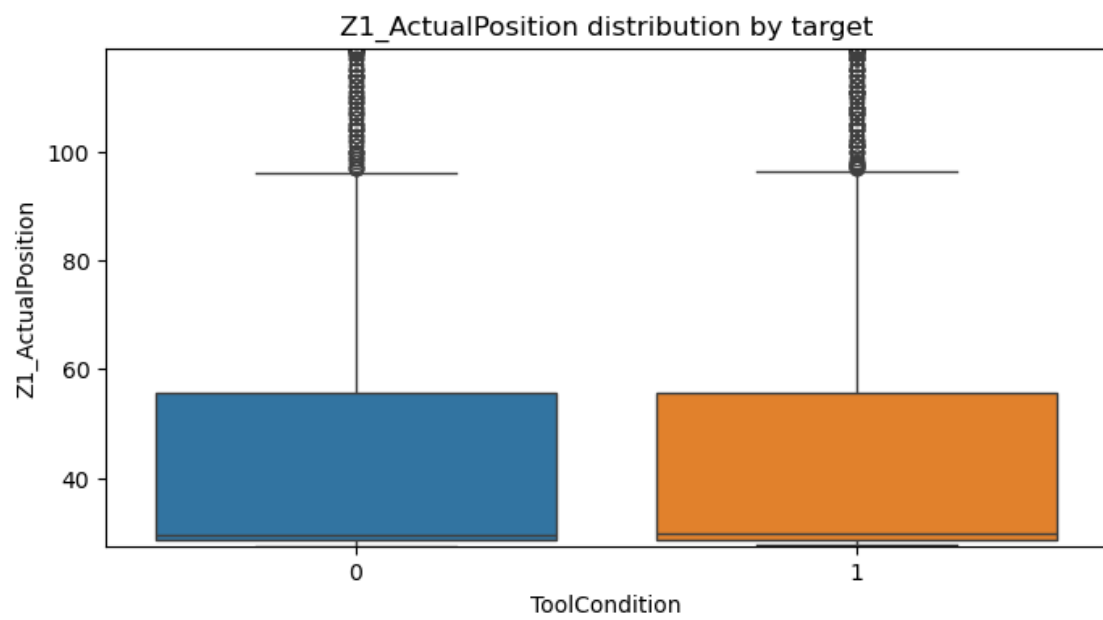
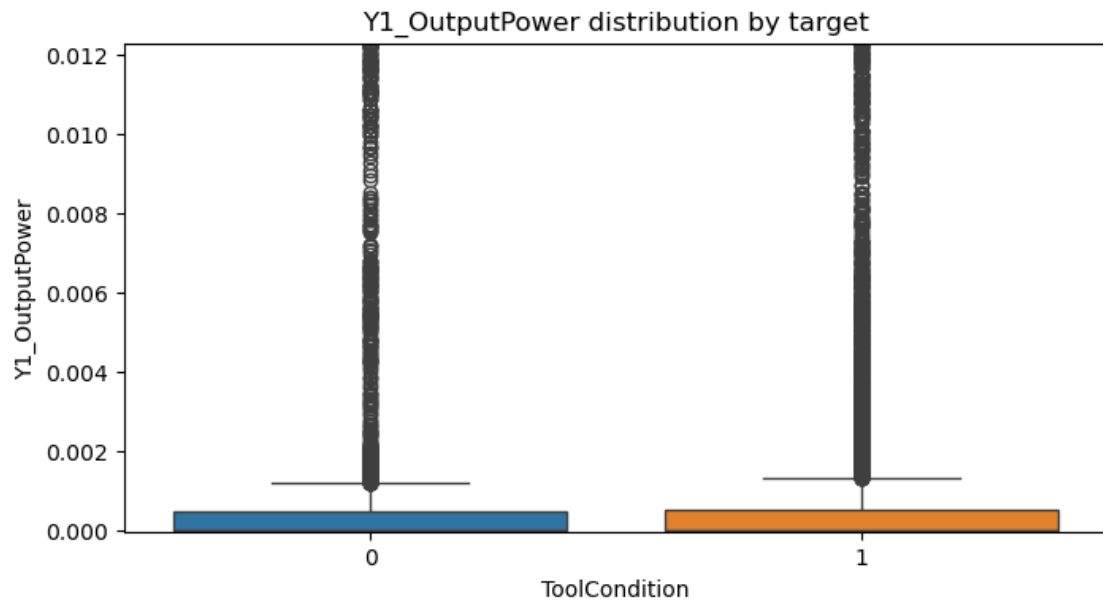


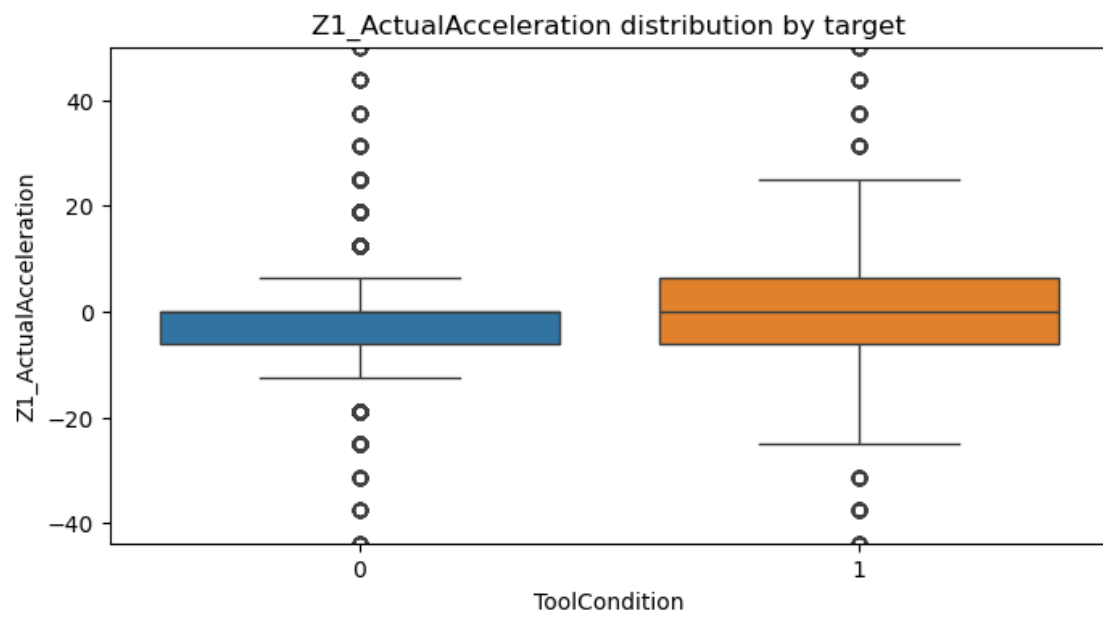
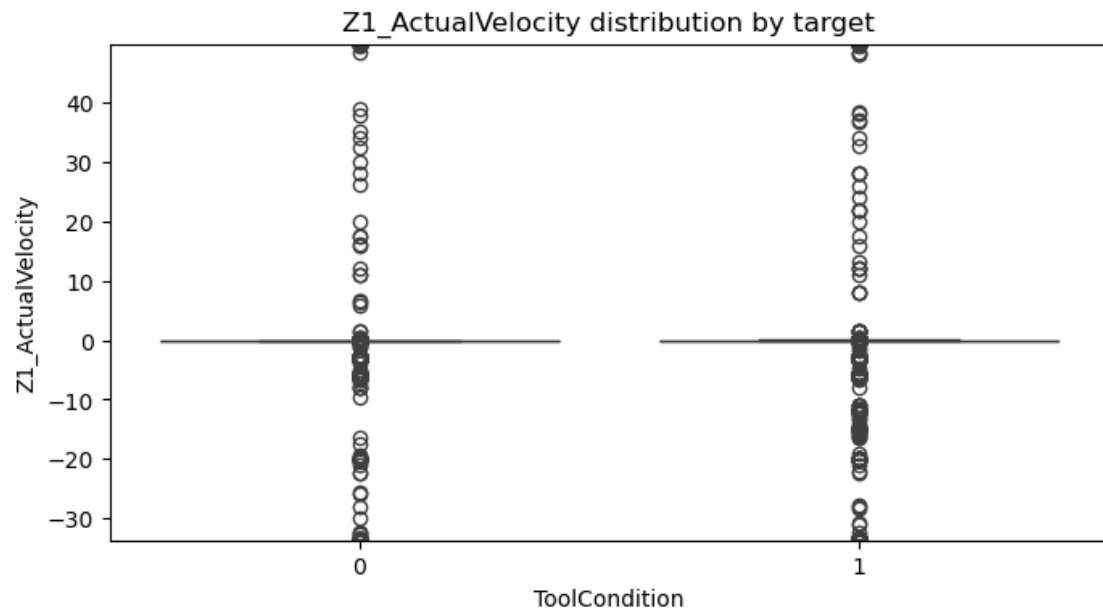


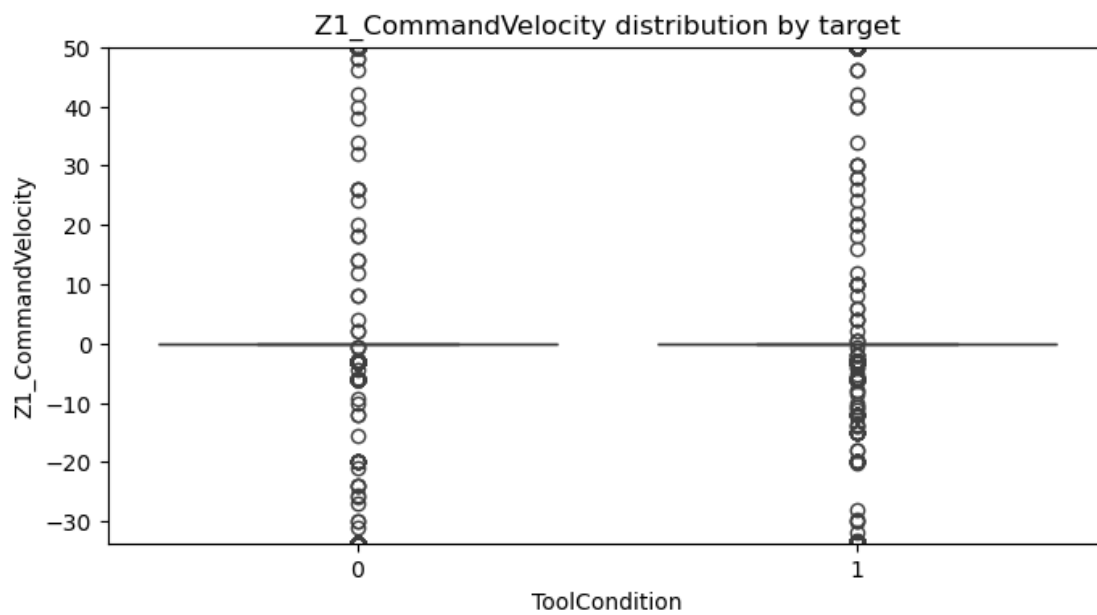
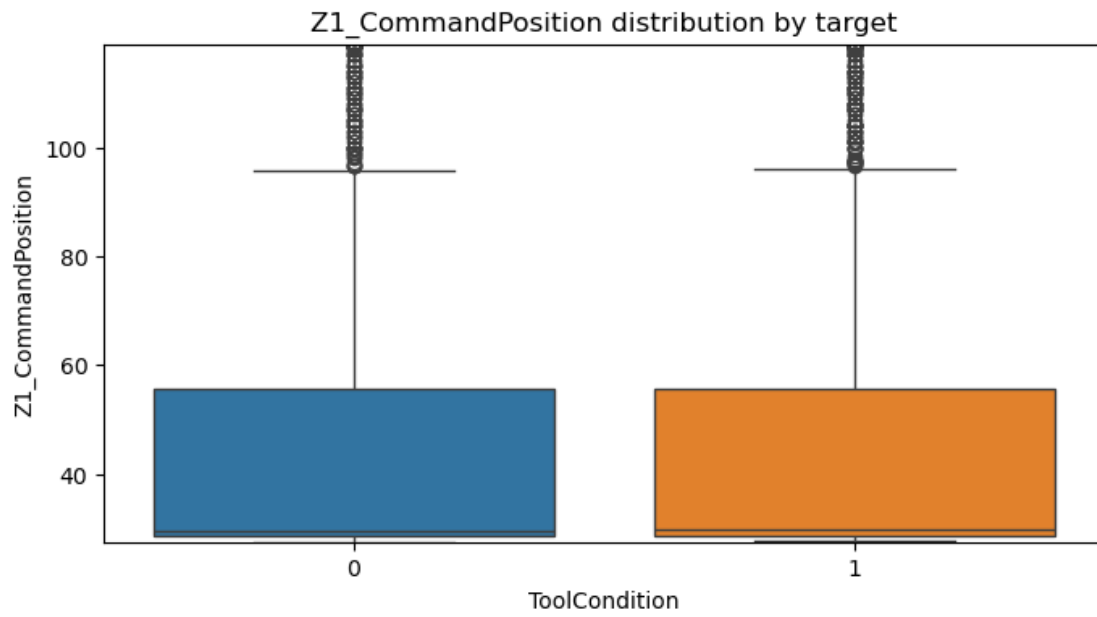


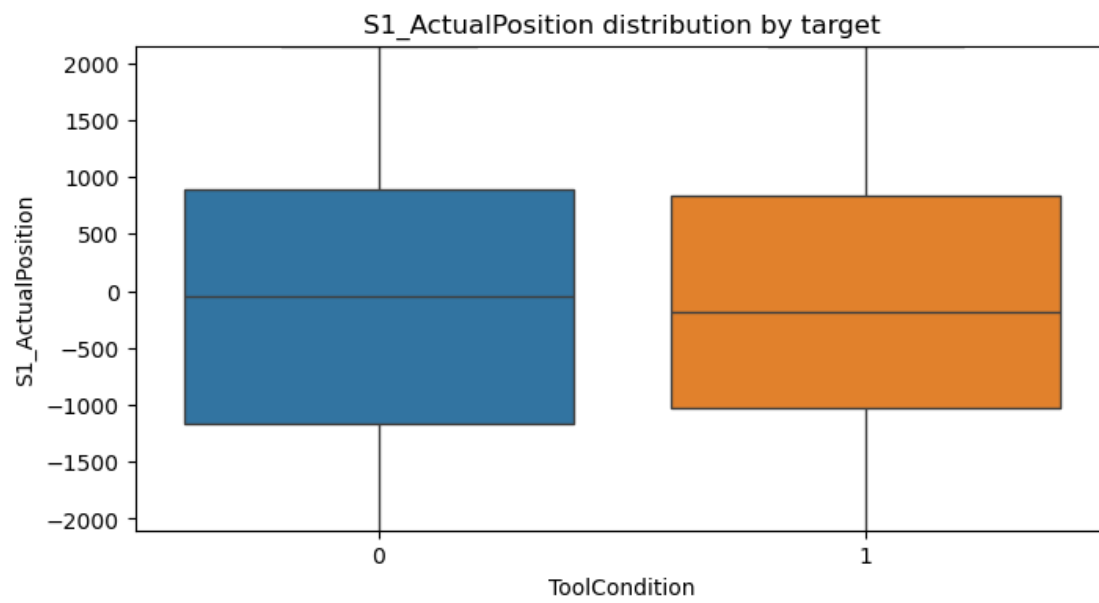
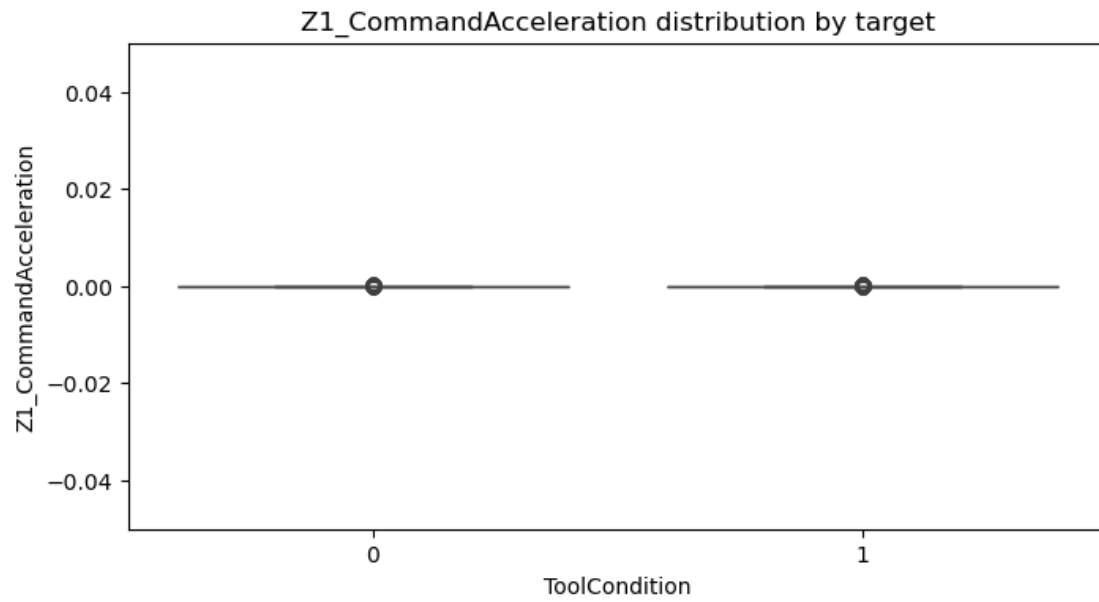


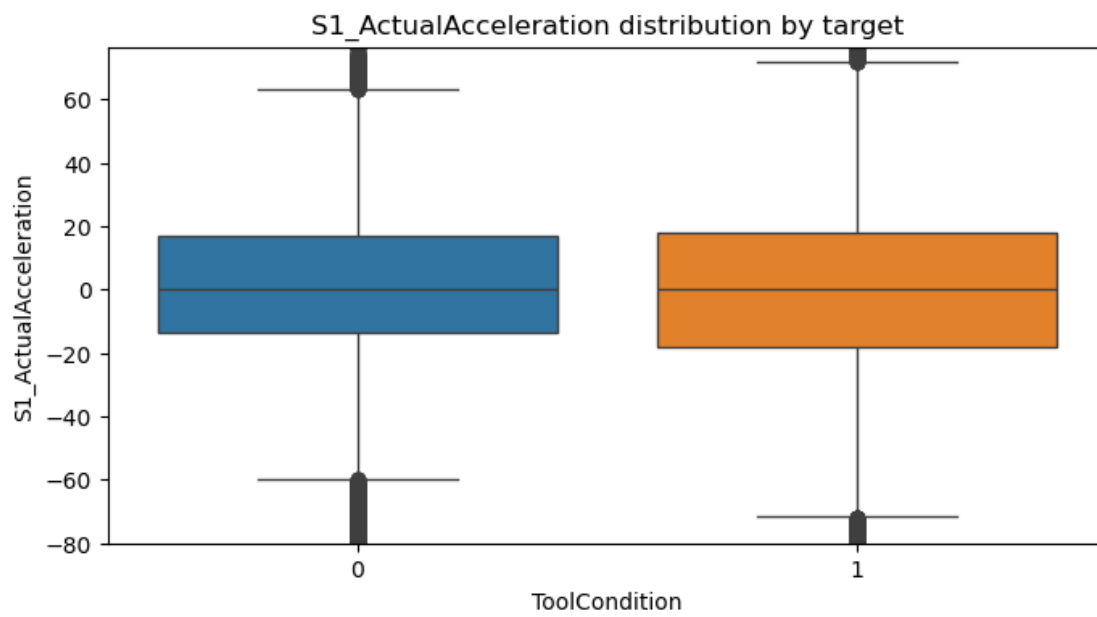
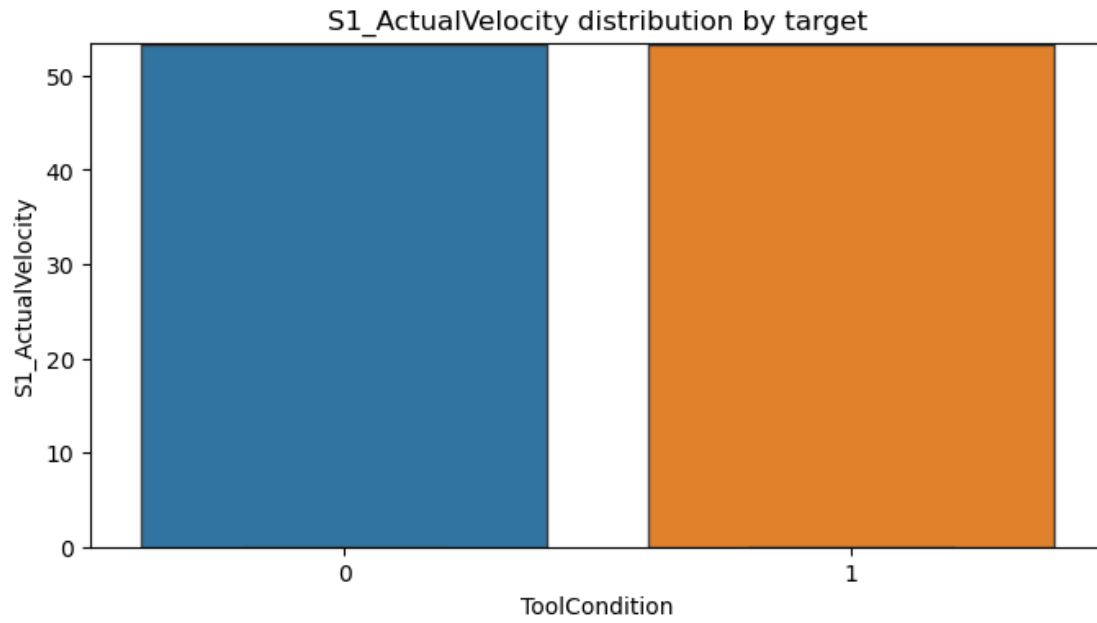


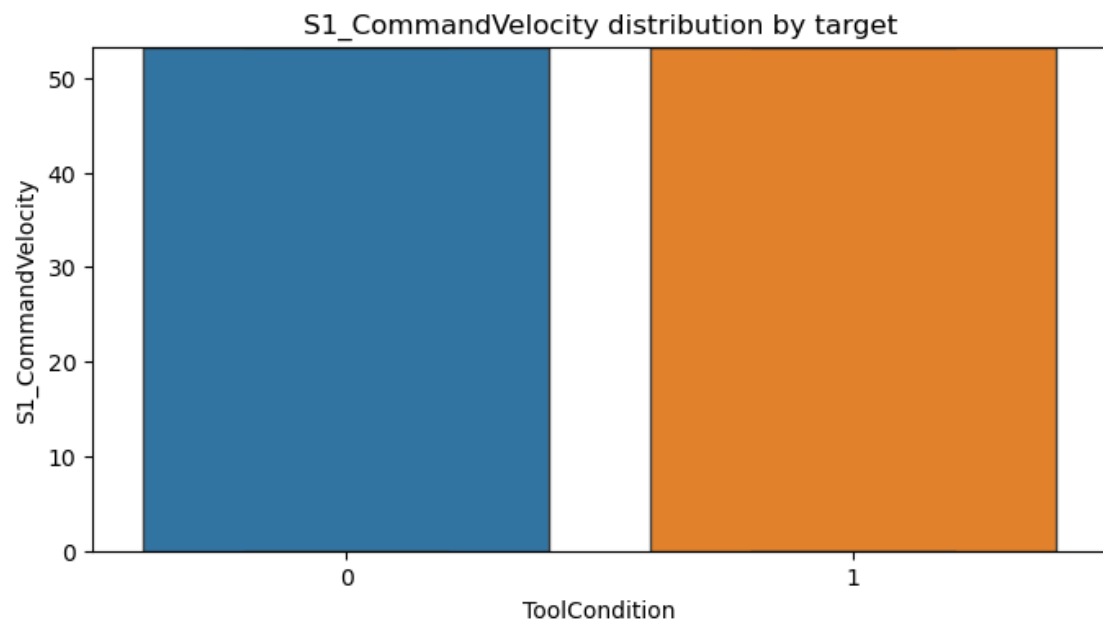
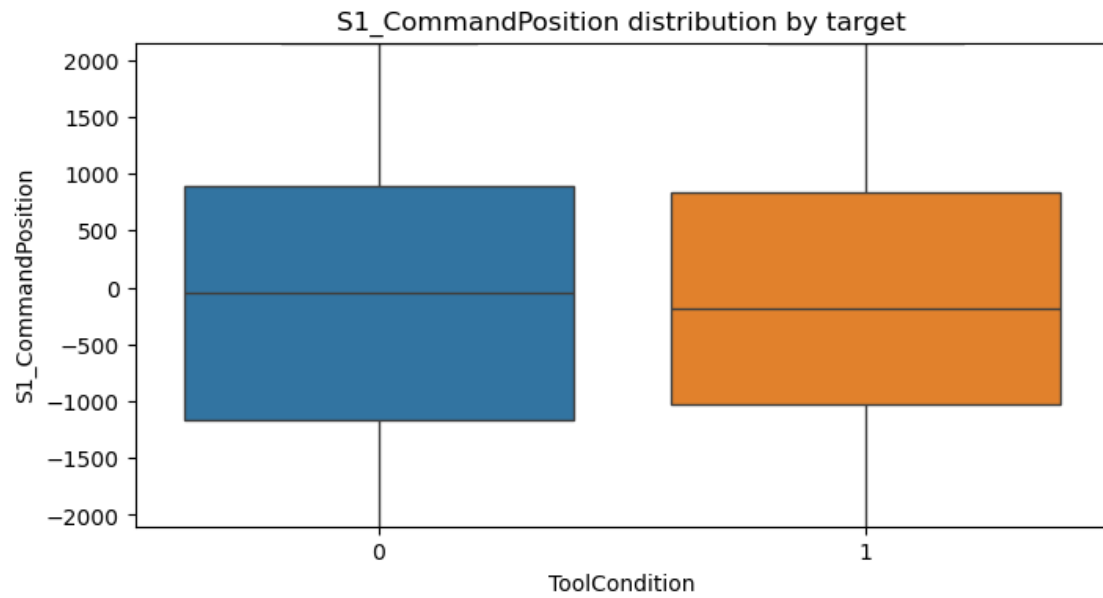


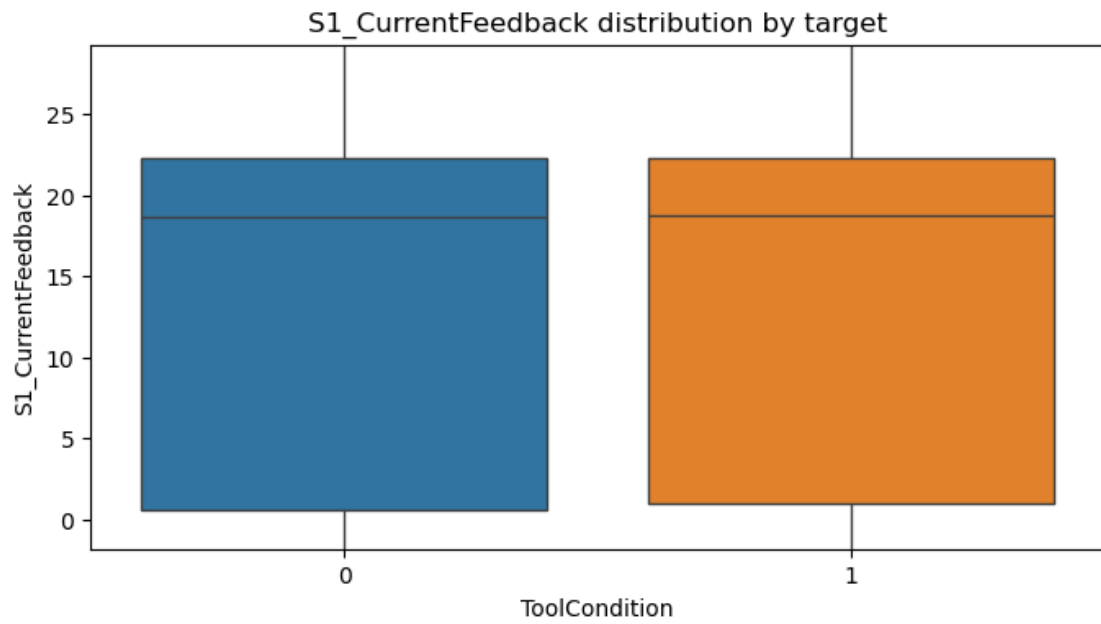
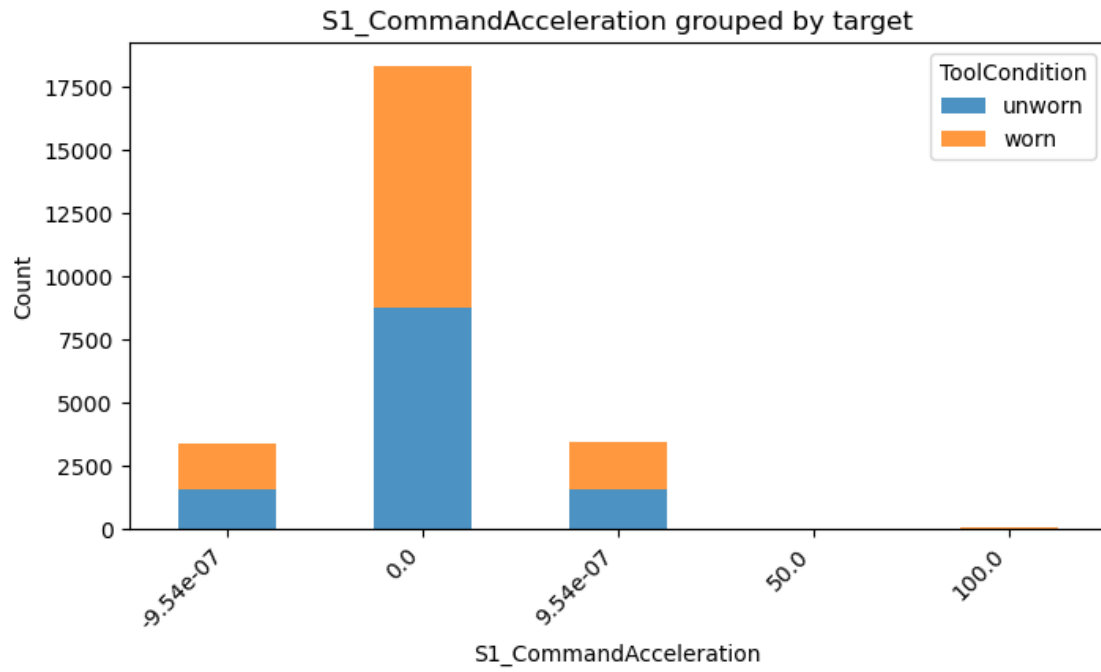


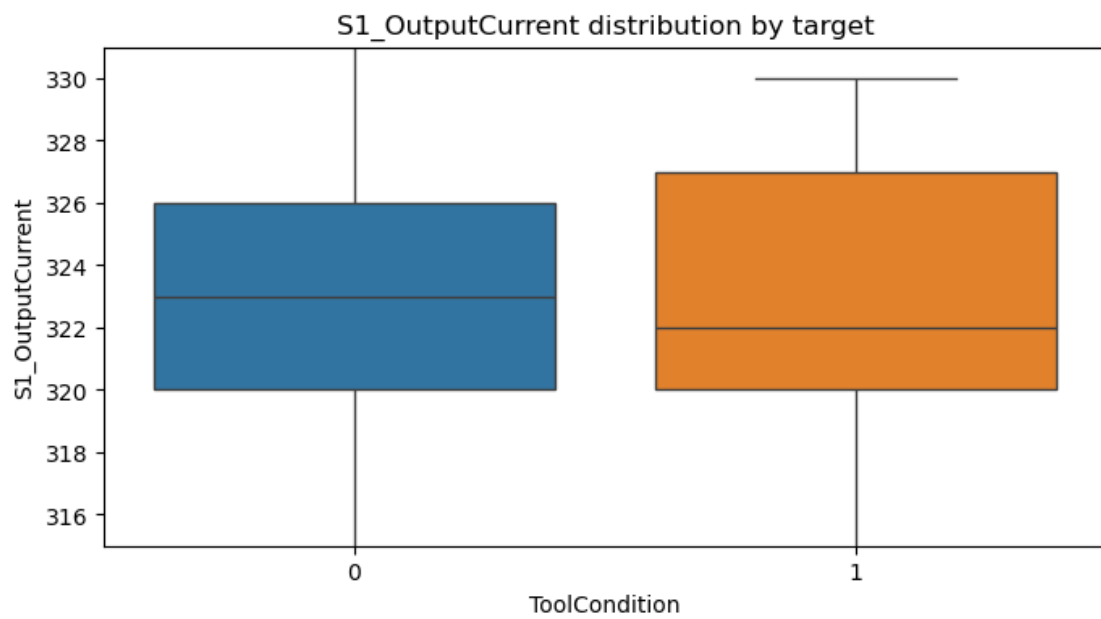
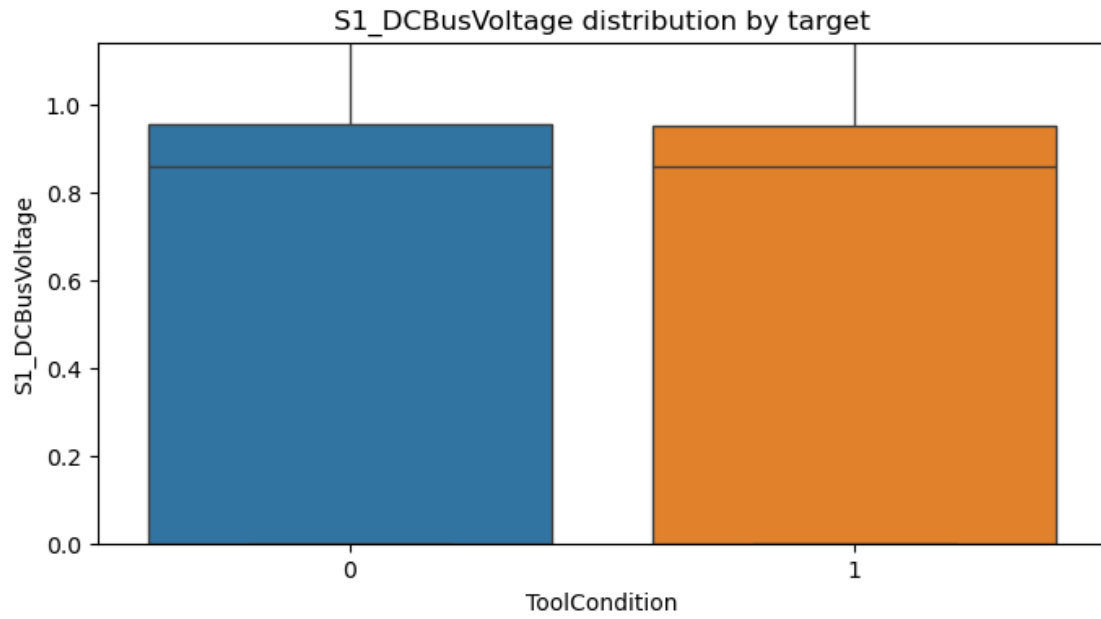


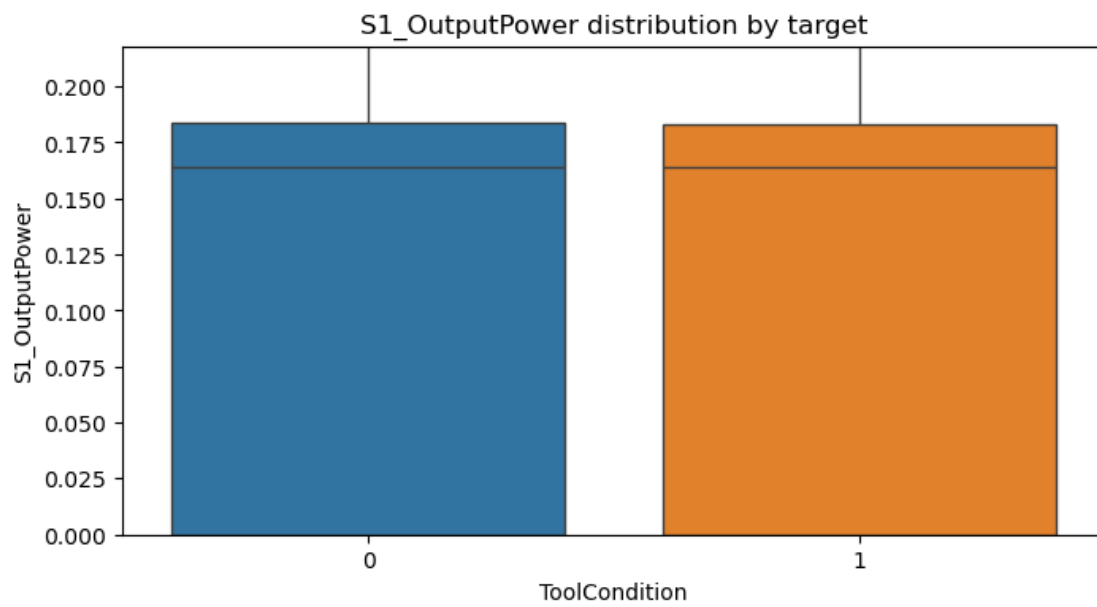
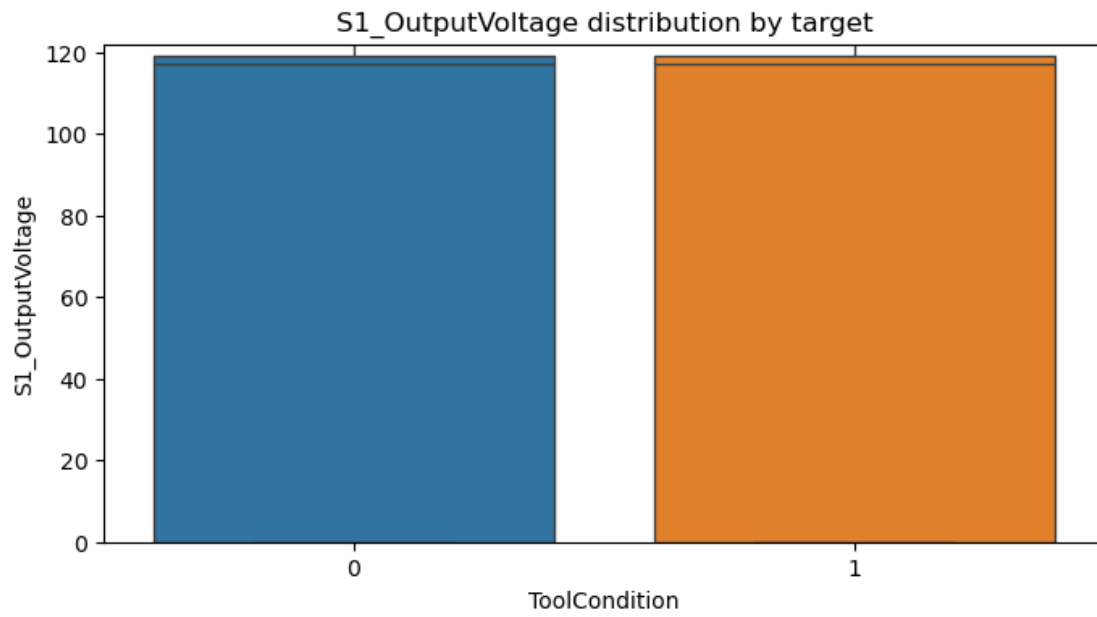


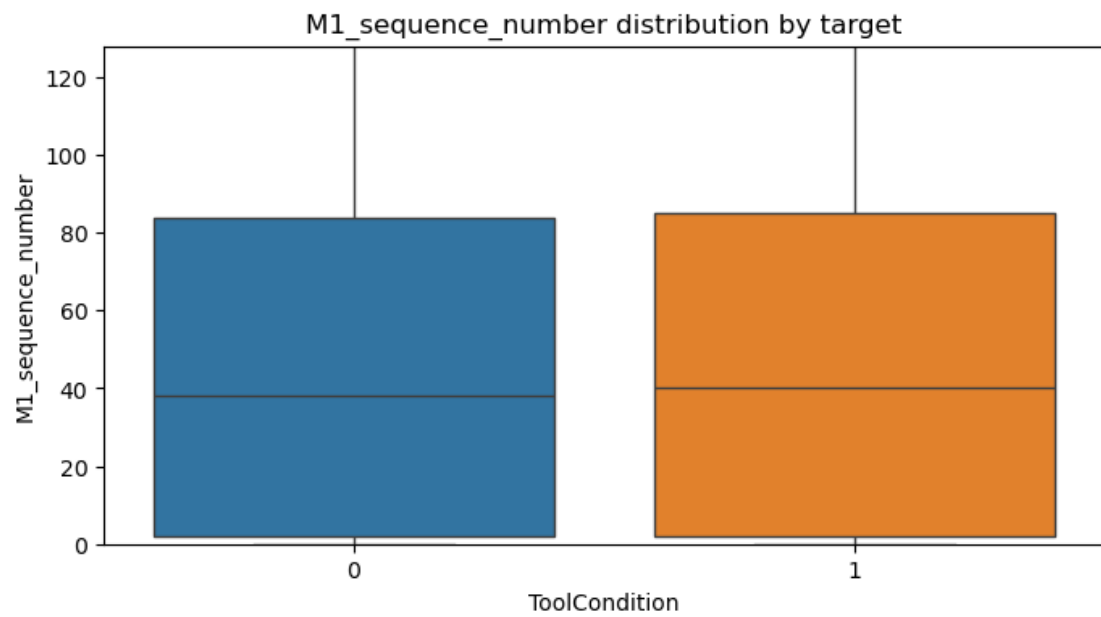
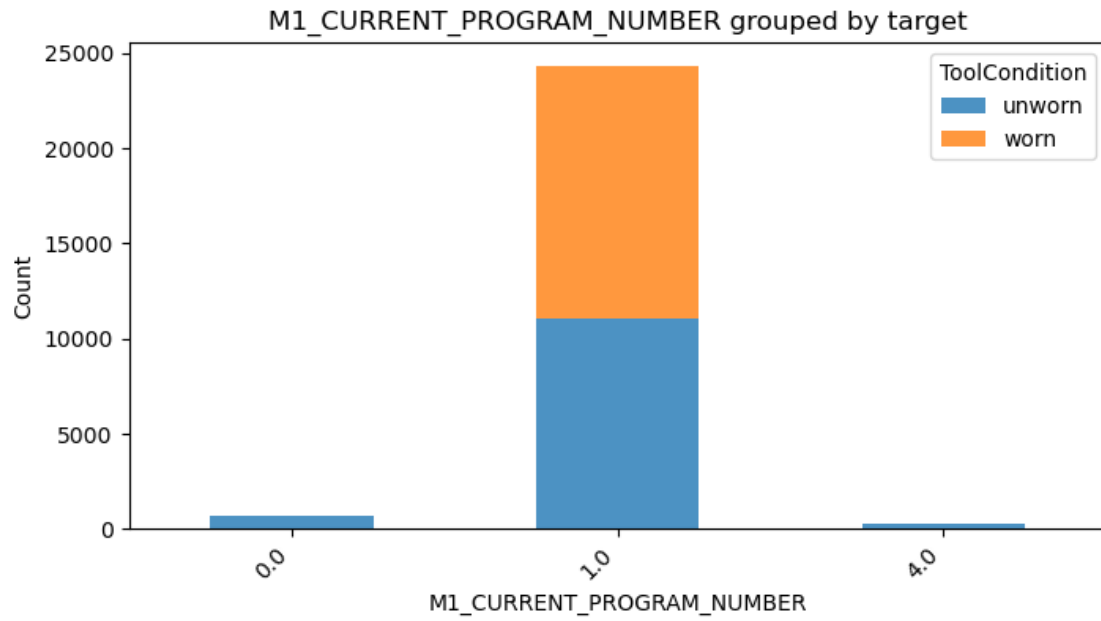


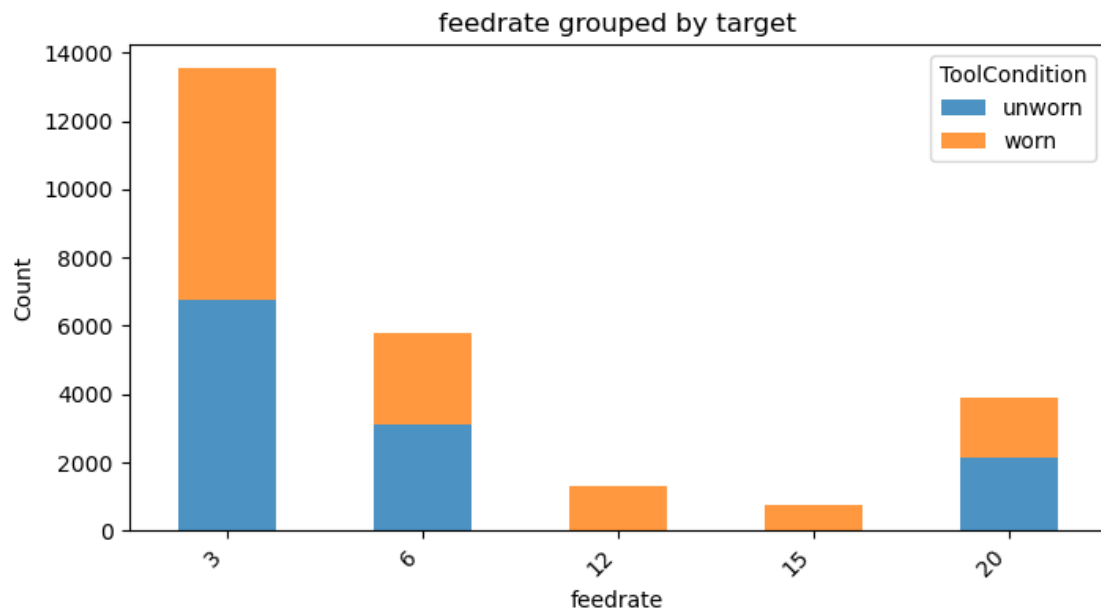
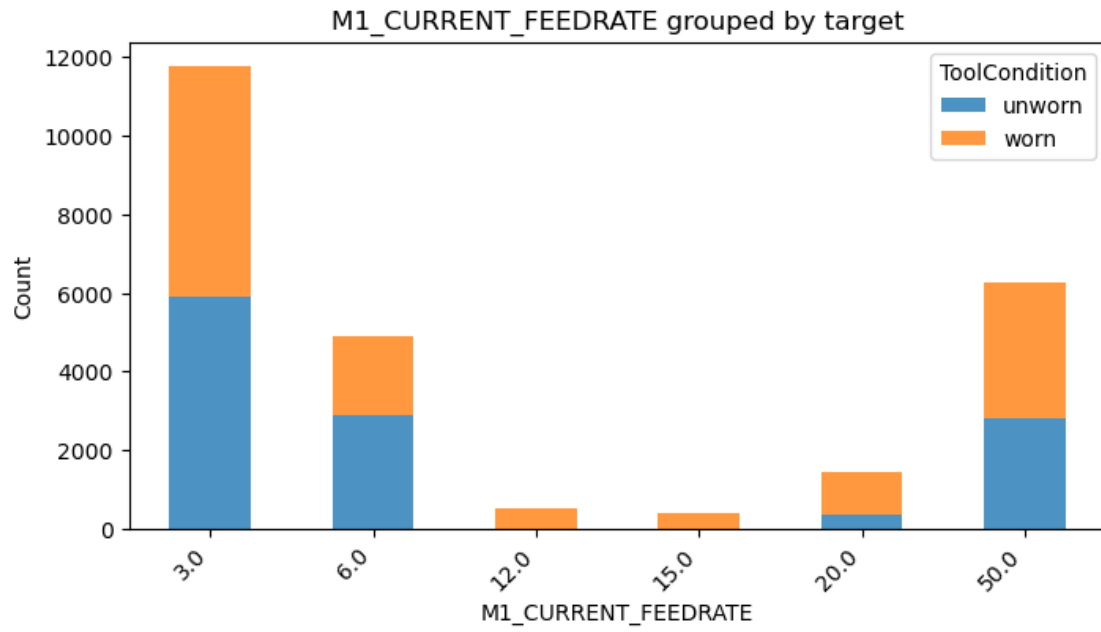


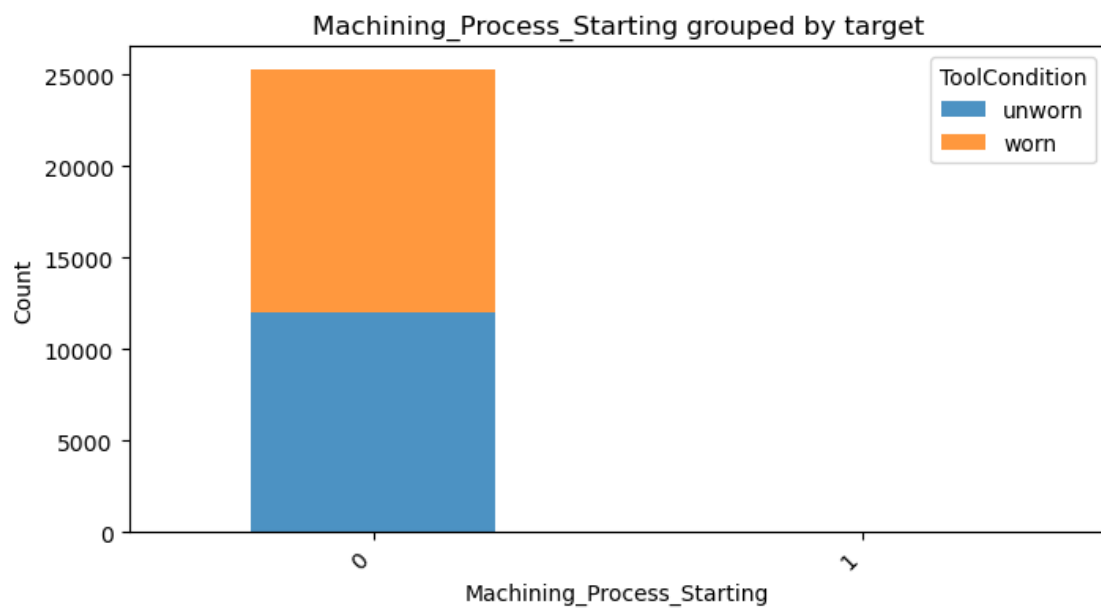
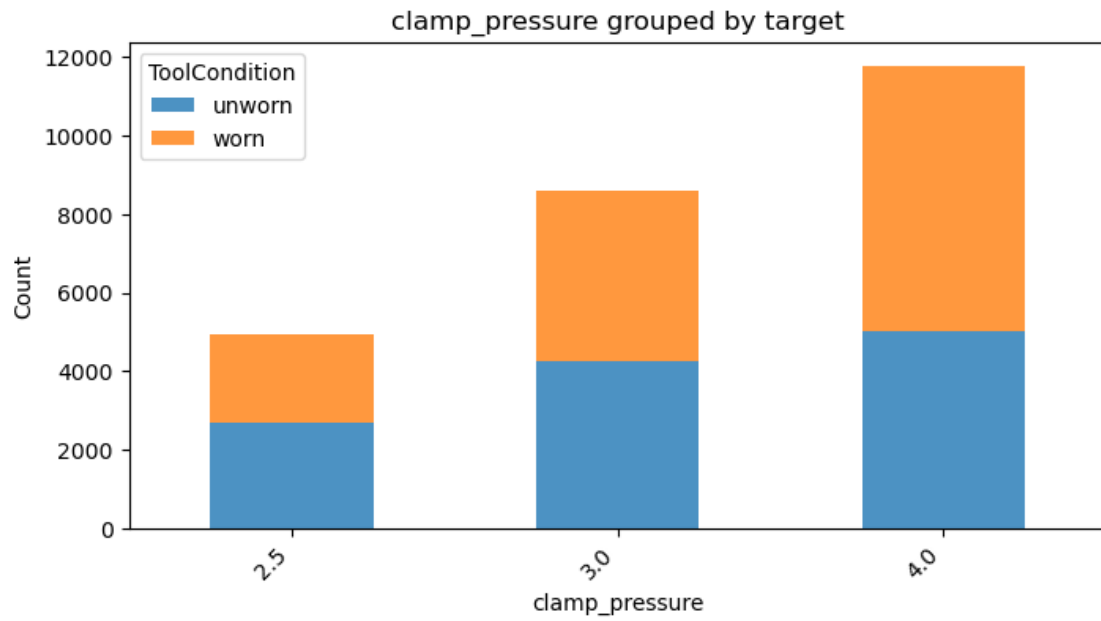


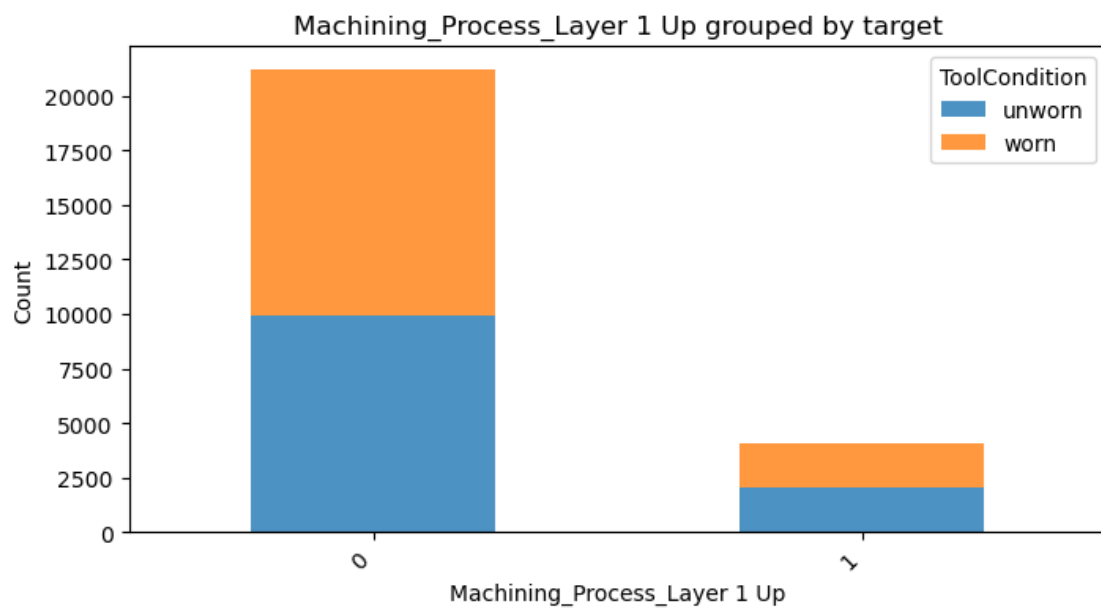
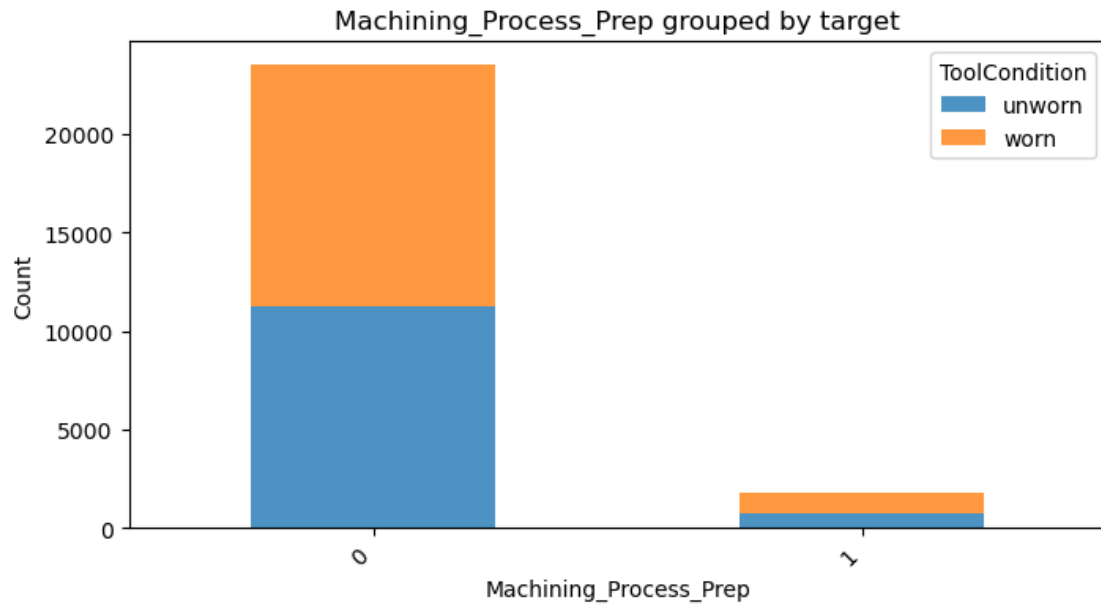


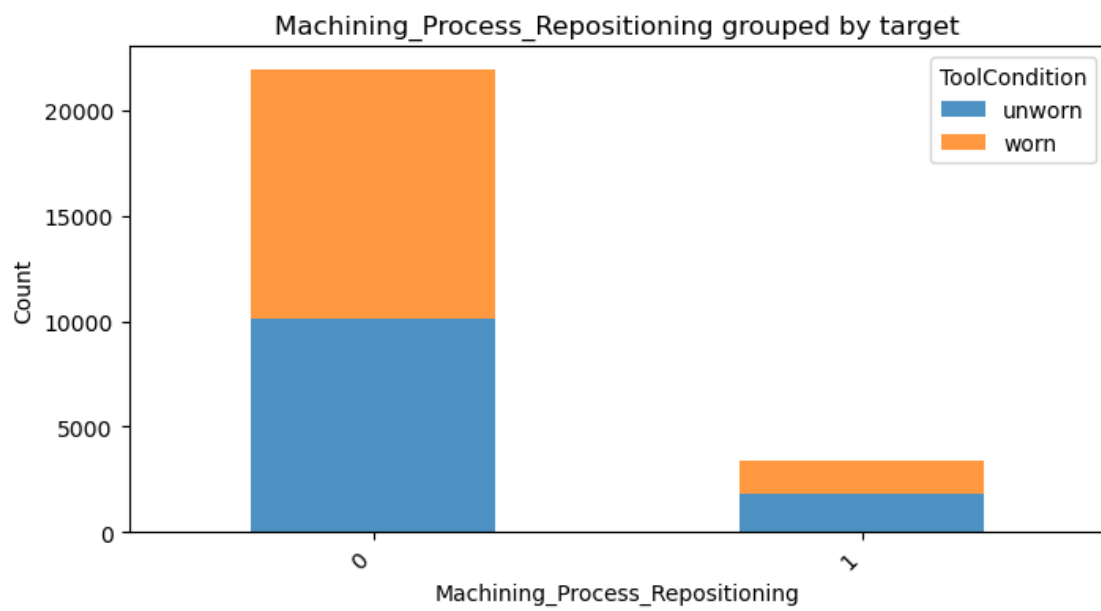
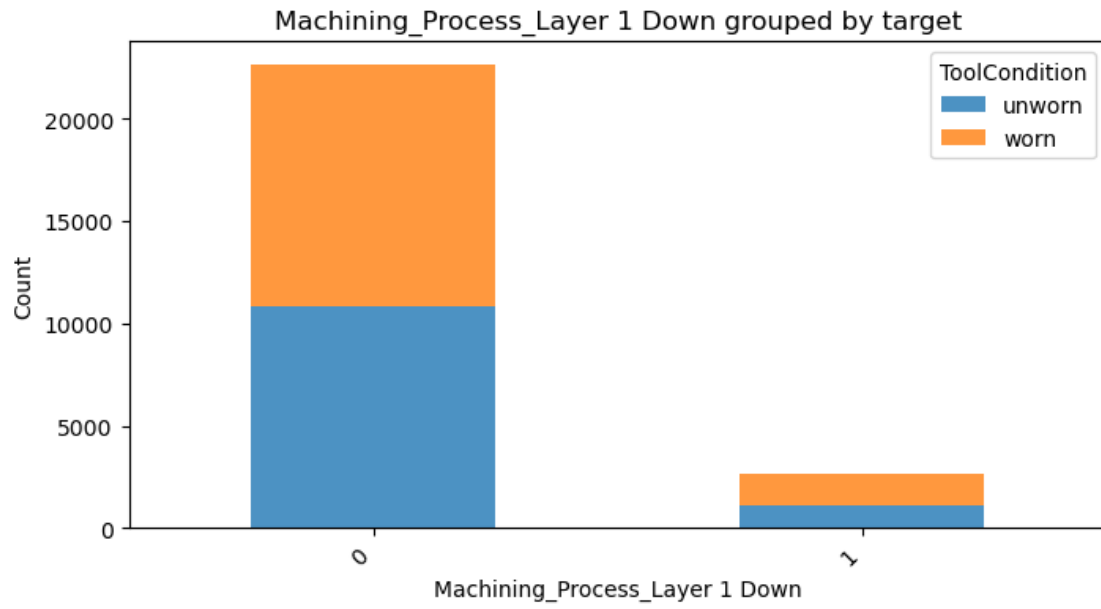


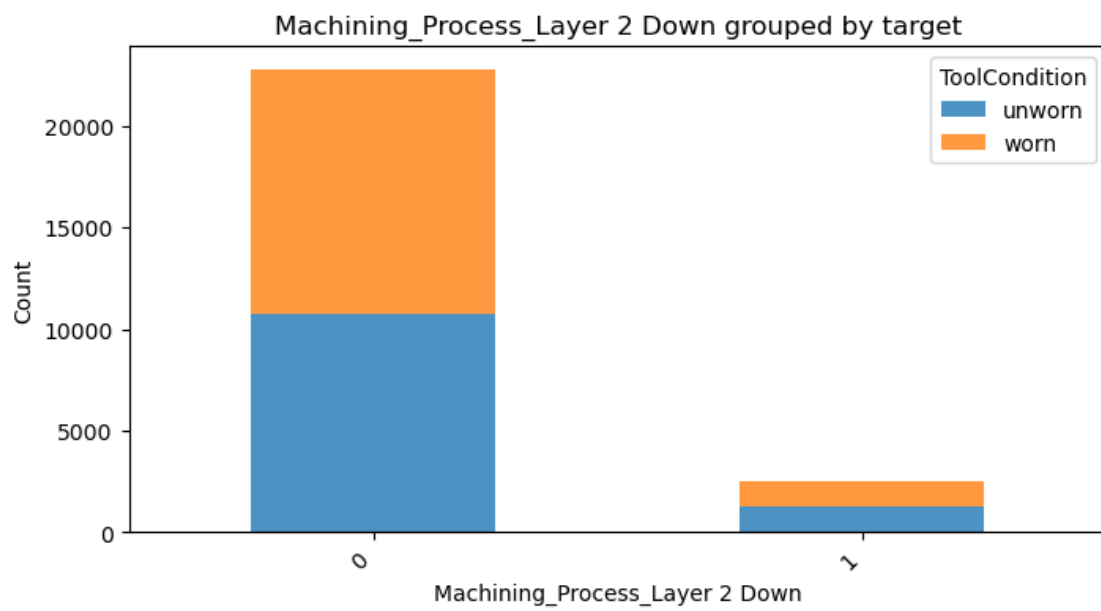
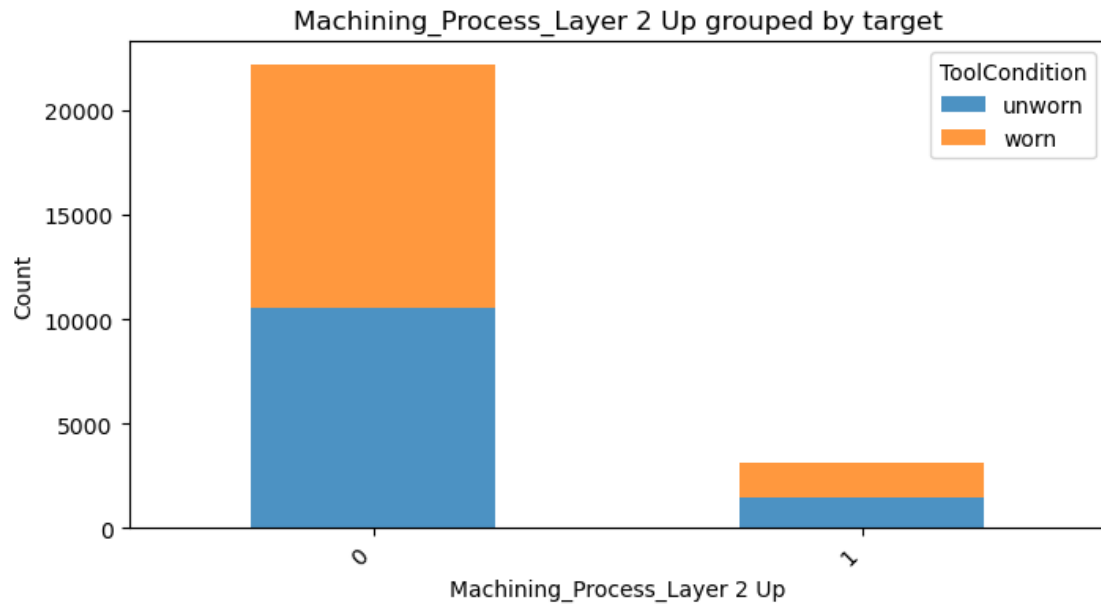


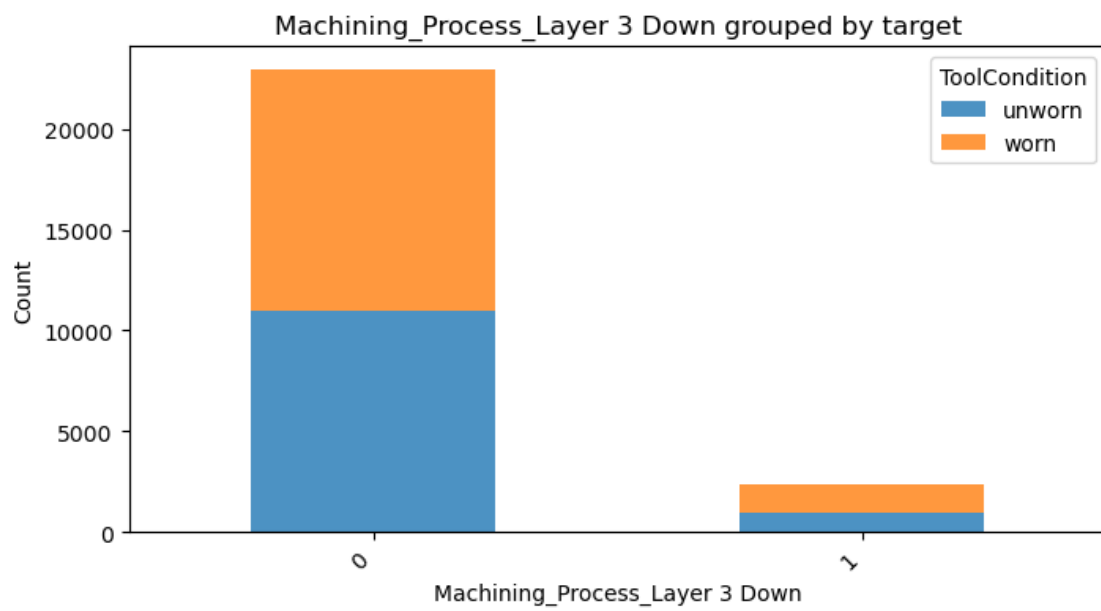
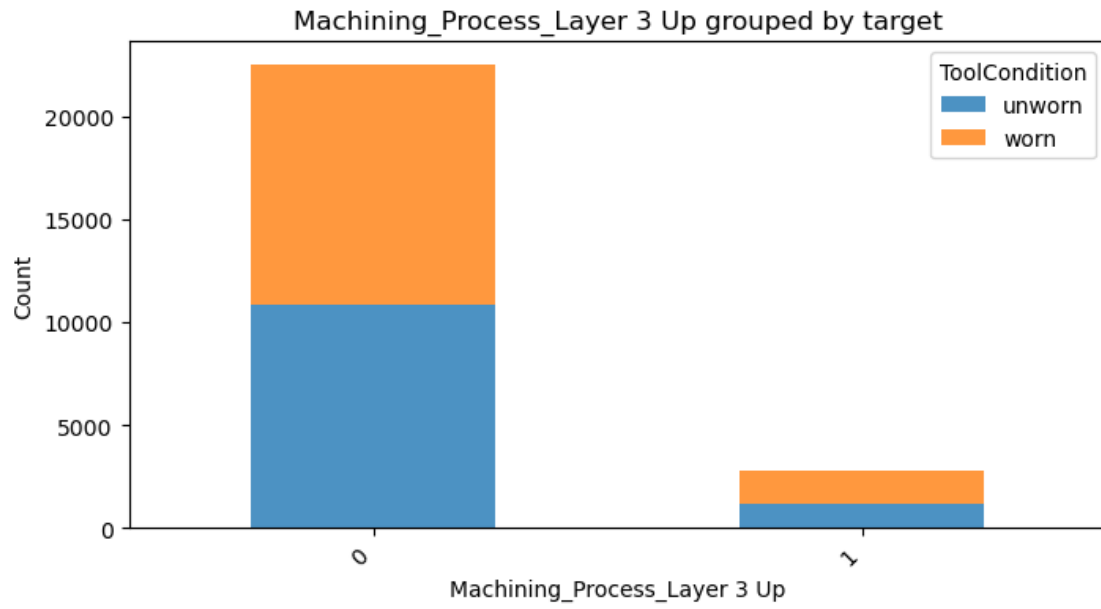


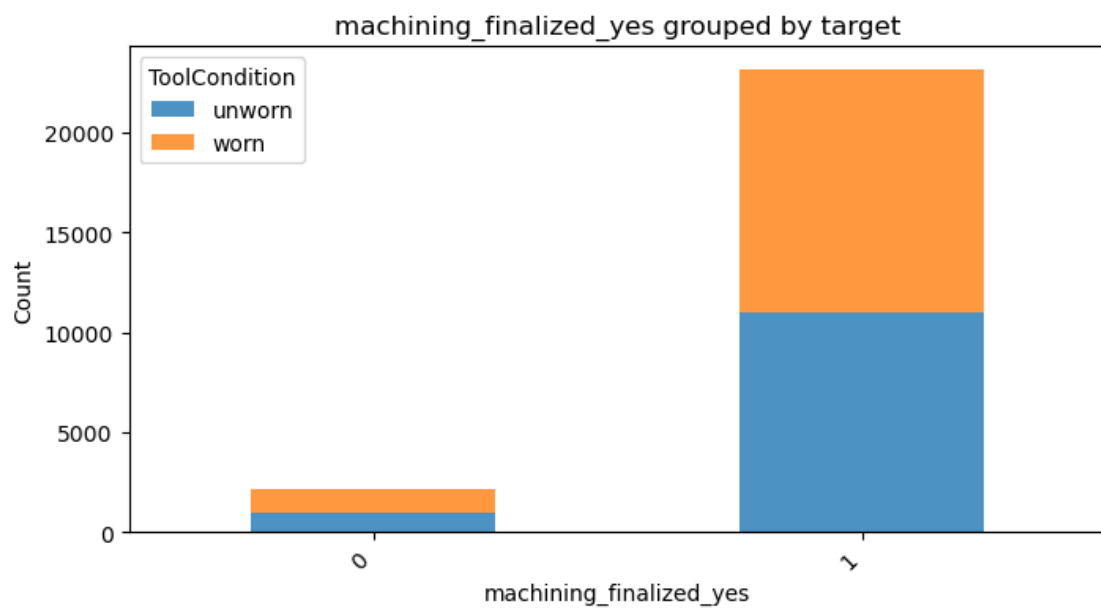
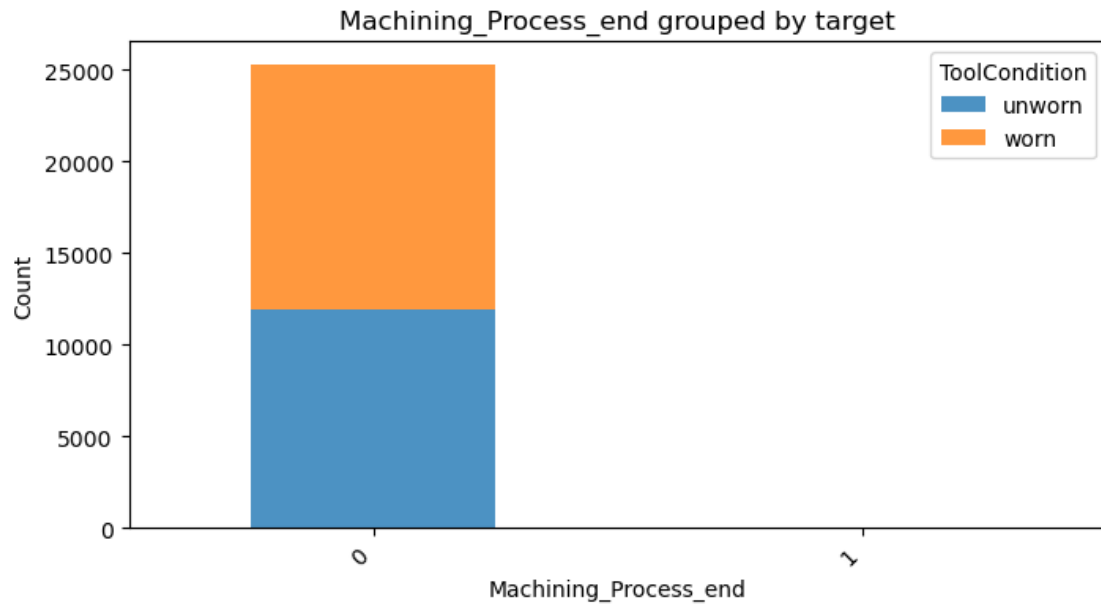


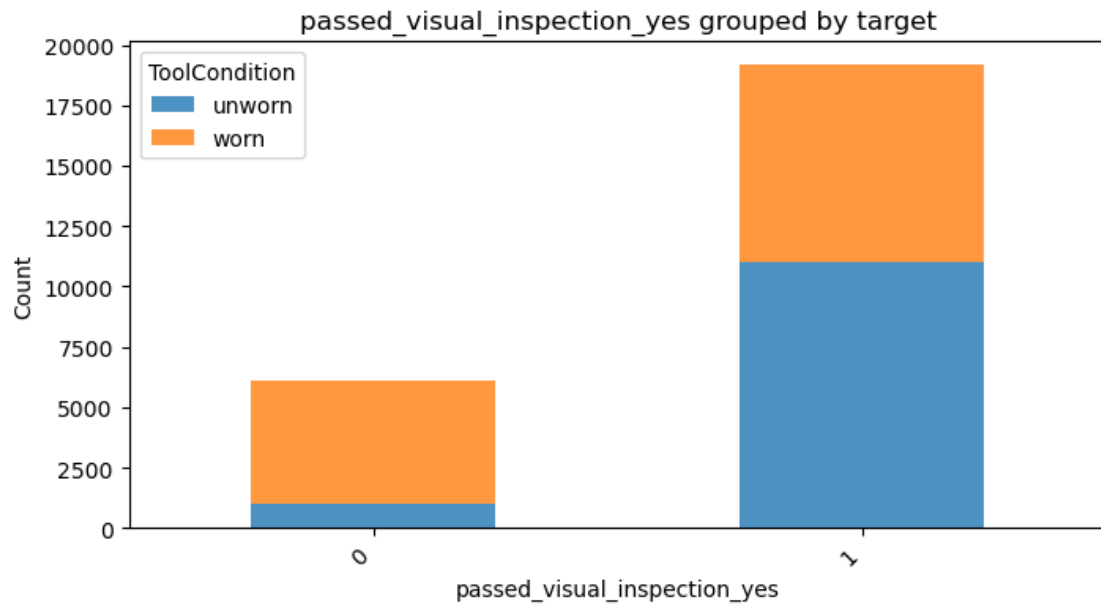










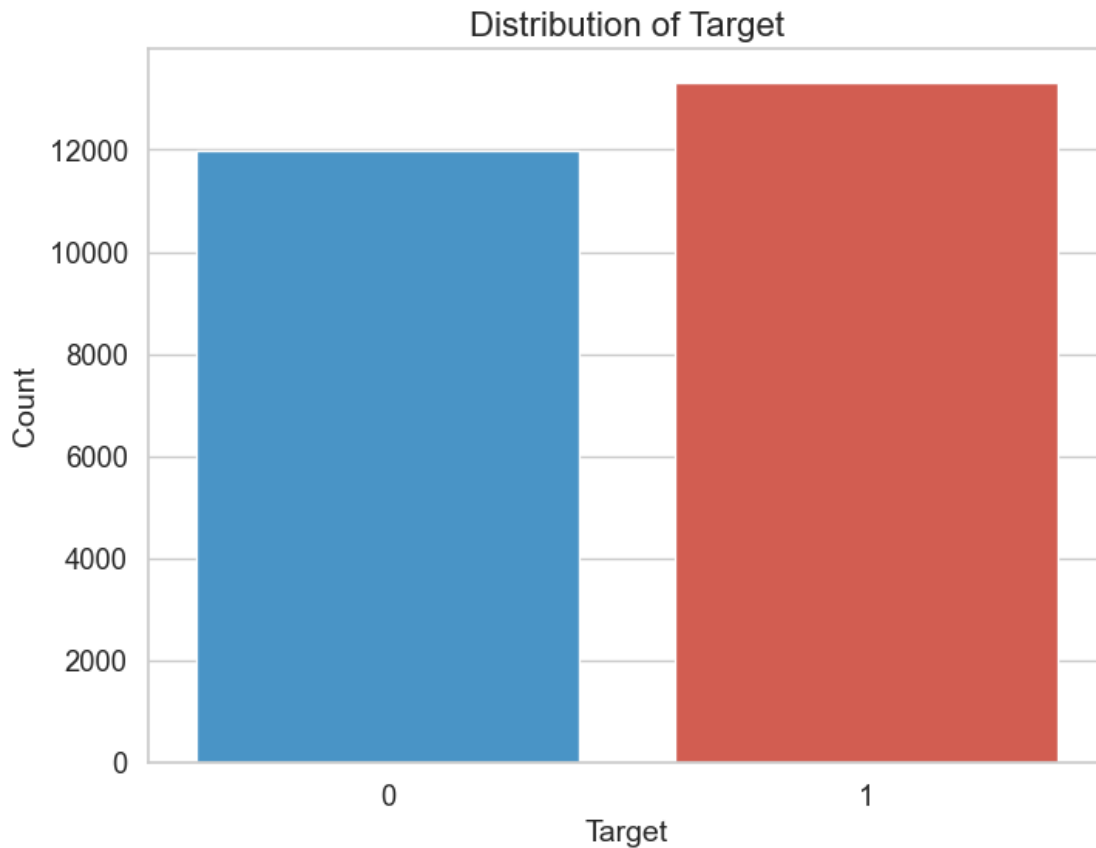


```
[26]: sns.set(style="whitegrid", palette="pastel", font_scale=1.2)
plt.figure(figsize=(8, 6))

sns.countplot(x='target', hue='target', data=tmp, palette=["#3498db", "#e74c3c"], legend=False)
```

```
plt.xlabel('Target', fontsize=14)
plt.ylabel('Count', fontsize=14)
plt.title('Distribution of Target', fontsize=16)

plt.show()
```



In our dataset, the target variables is nearly balanced, meaning that the classes are represented in roughly equal proportions, making it easier to train the model without underfitting or overfitting problems

1.7 4. Separate to train and test split

```
[27]: X = df.drop(['target'], axis = 1)
      y = df['target']

      X.shape, y.shape
```

```
[28]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25)
      X_train.shape, X_test.shape
```

1.8 5. Feature Engineering

```
[29]: def run_randomForest(X_train, X_test, y_train, y_test):  
  
    rf = RandomForestClassifier(n_estimators = 50, random_state = 42, max_depth=5)  
    rf.fit(X_train, y_train)  
  
    print('Train set')  
    pred = rf.predict_proba(X_train)  
    print('Random Forest roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))  
  
    print('Test set')  
    pred = rf.predict_proba(X_test)  
    print('Random Forest roc-auc: {}'.format(roc_auc_score(y_test, pred[:, 1])))
```

The function evaluates the performance of the model with default parameters and with different methods for processing NA values

Impute missing value with different methods

1.9 5.1 Random Sample Imputer

```
[30]: random_pipe = Pipeline(steps = [  
    ('random_pipe', RandomSampleImputer()),  
    ('ohe_enc', OneHotEncoder())  
)  
random_pipe.fit(X_train, y_train)  
  
random_X_train = random_pipe.transform(X_train)  
random_X_test = random_pipe.transform(X_test)
```

1.10 5.2 Categorical Imputer

```
[31]: categorical_pipe = Pipeline(steps = [  
    ('imputer_categ', CategoricalImputer(variables = ['passed_visual_inspection'],  
    imputation_method = 'frequent')),  
    ('ohe_enc', OneHotEncoder(variables = ['Machining_Process', 'machining_finalized', 'passed_visual_inspection'], drop_last = True))  
)  
categorical_pipe.fit(X_train, y_train)  
  
frequent_X_train = categorical_pipe.transform(X_train)  
frequent_X_test = categorical_pipe.transform(X_test)
```

1.11 5.3 Simple Imputer

```
[32]: simple_pipe = Pipeline(steps = [  
        ('imputer', CategoricalImputer(imputation_method = 'missing')),  
        ('ohe_enc', OneHotEncoder(drop_last = True))  
    ])  
simple_pipe.fit(X_train, y_train)  
  
simple_X_train = simple_pipe.transform(X_train)  
simple_X_test = simple_pipe.transform(X_test)
```

5.4 Results

```
[33]: # RandomSampleImputer  
run_randomForest(random_X_train, random_X_test, y_train, y_test)
```

```
[34]: # CategoricalImputer method frequent  
run_randomForest(frequent_X_train, frequent_X_test, y_train, y_test)
```

```
[35]: # CategoricalImputer method missing  
run_randomForest(simple_X_train, simple_X_test, y_train, y_test)
```

1.11.1 5.5 Analysis constant, duplicated and correlated features

```
[36]: select_pipe = Pipeline(steps = [  
        ('constant', DropConstantFeatures(tol = 0.998)),  
        ('duplicated', DropDuplicateFeatures()),  
        ('correlated', SmartCorrelatedSelection(selection_method = 'variance'))  
    ])  
select_pipe.fit(random_X_train)
```

```
[37]: train_X = select_pipe.fit_transform(random_X_train)  
test_X = select_pipe.transform(random_X_test)  
train_X.shape
```

```
[38]: train_X.shape, random_X_train.shape
```

Out of 58 variables, 35 variables remained, after the operation of selecting variables using methods (constant, duplicated, correlated) feature selection

Let's look at the performance of model after feature selecting

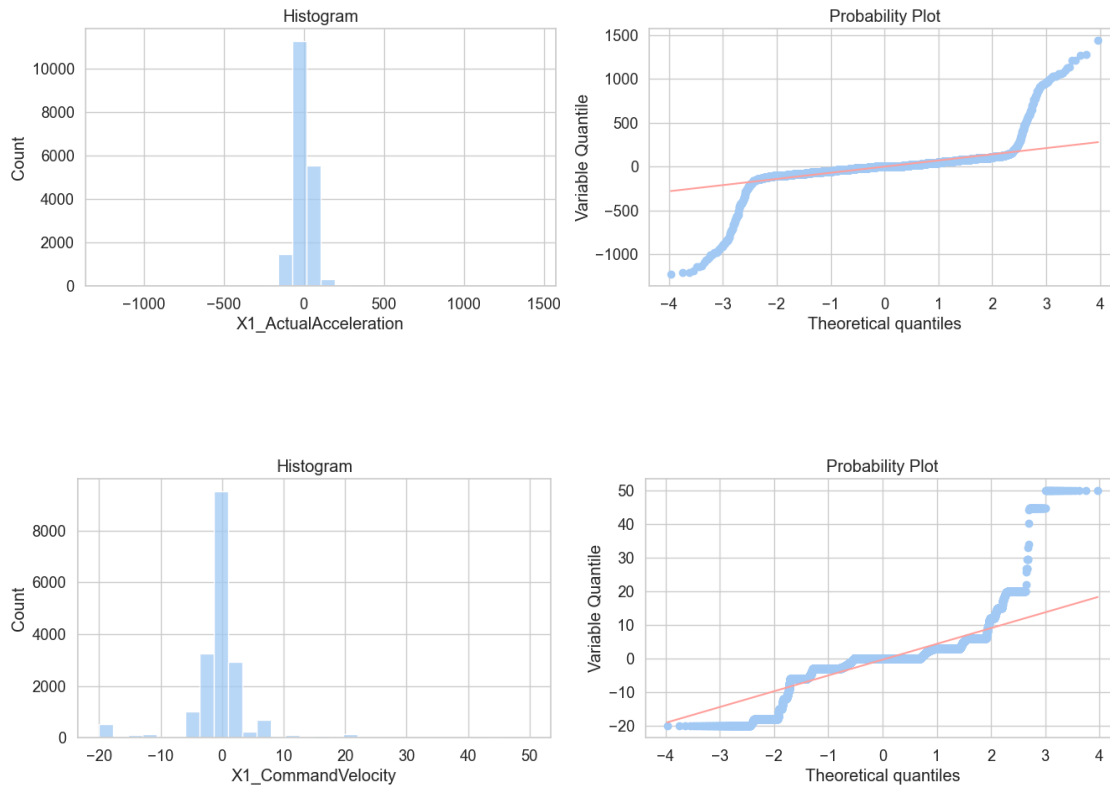
```
[39]: # Model performance after feature selection  
run_randomForest(train_X, test_X, y_train, y_test)
```

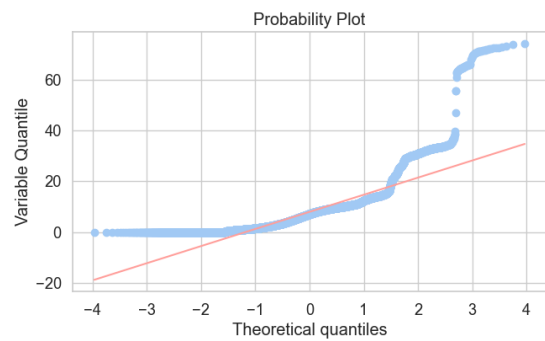
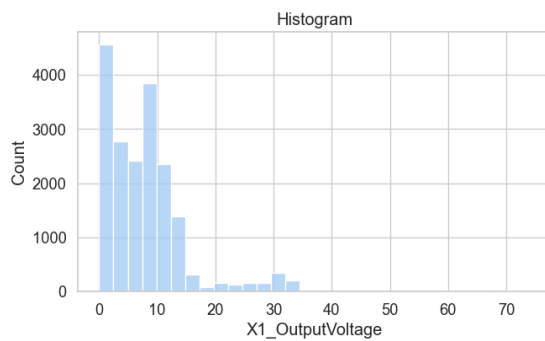
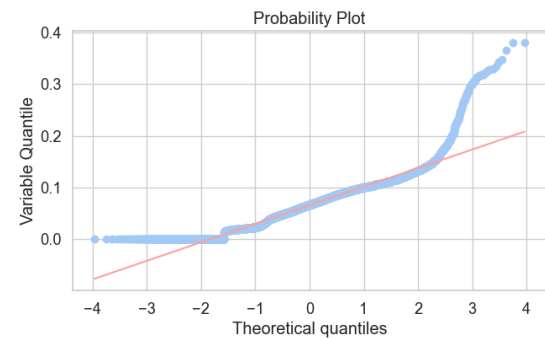
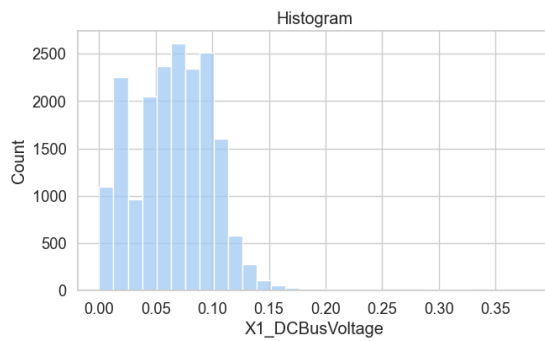
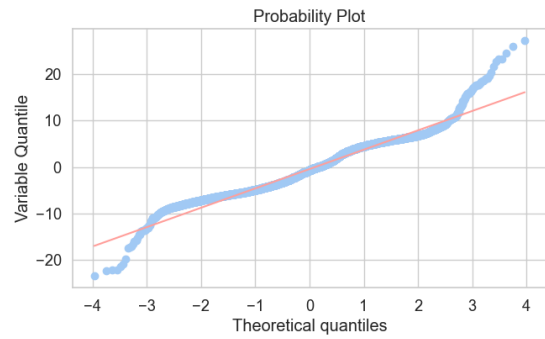
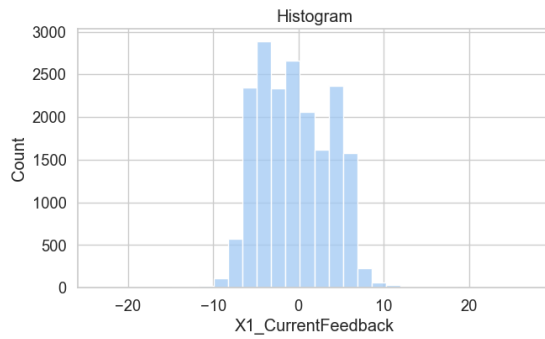
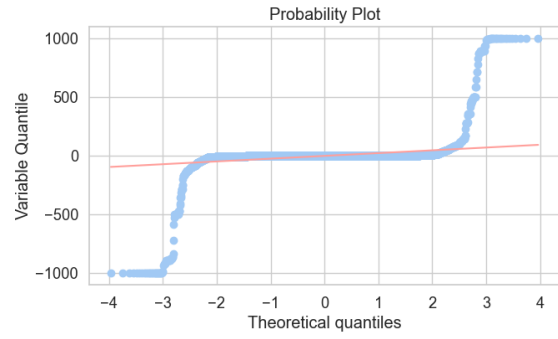
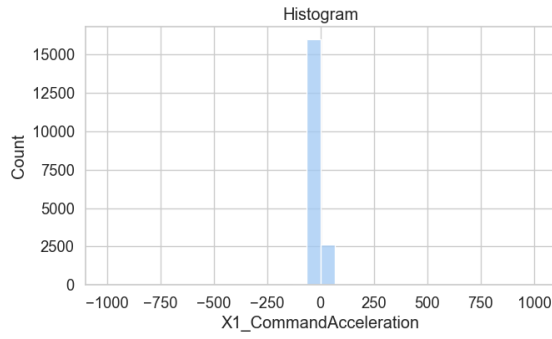
The results show that performance dropped by about 10% after variable selection, suggesting that overfitting

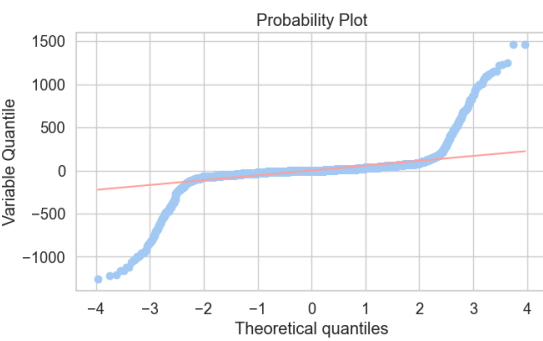
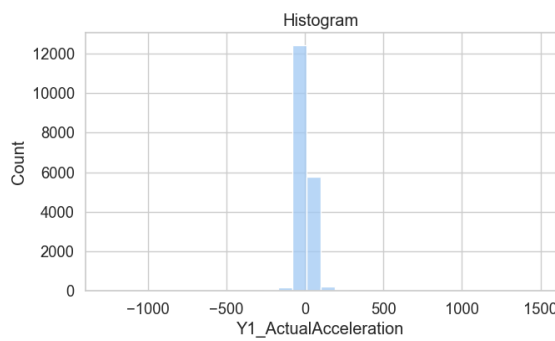
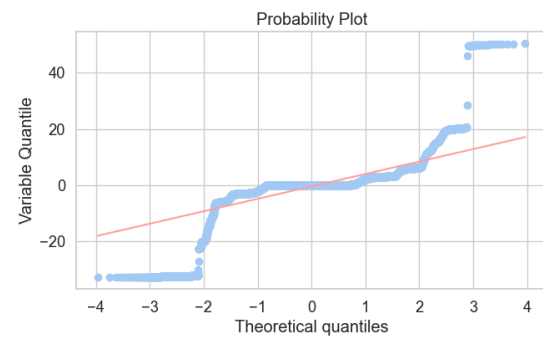
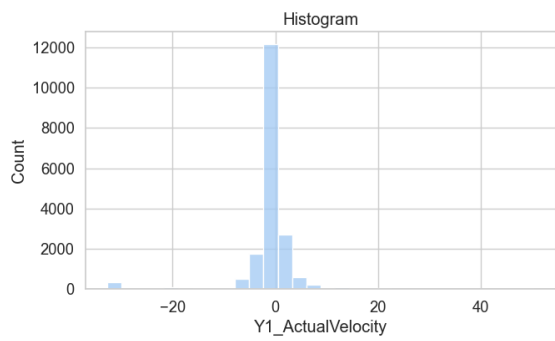
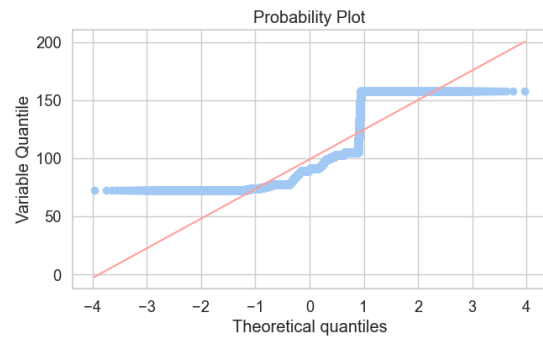
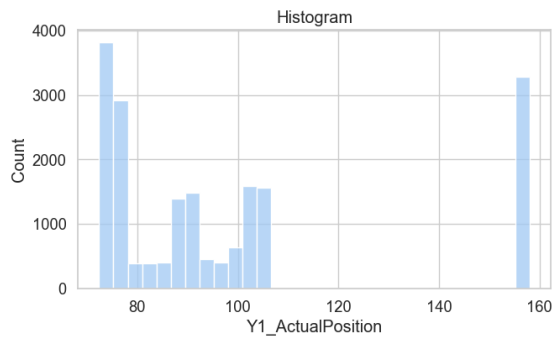
occurred before variable selection

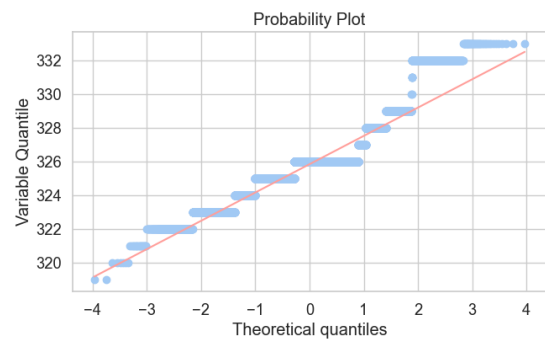
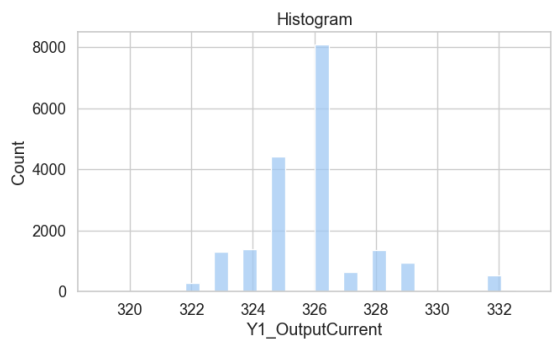
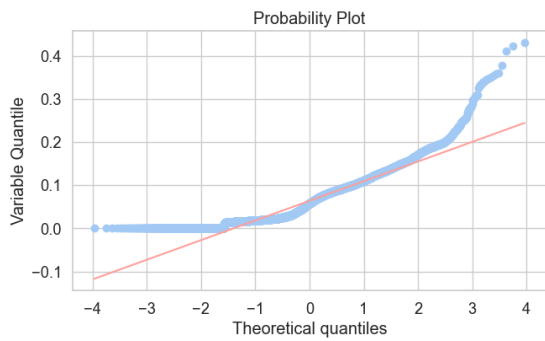
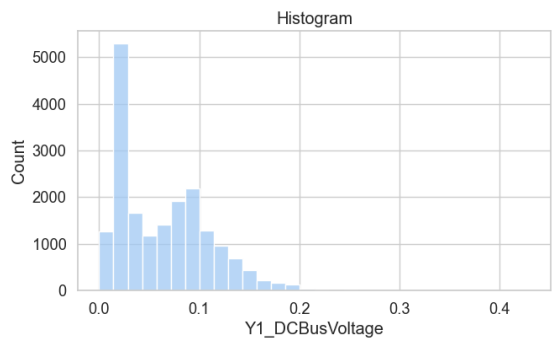
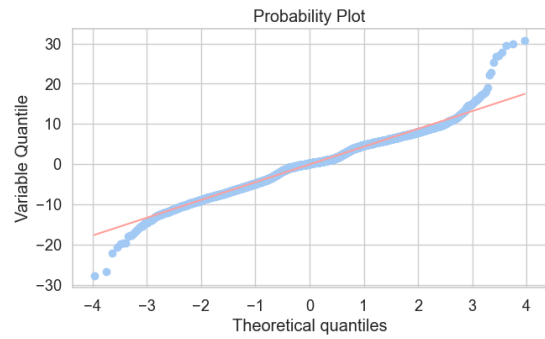
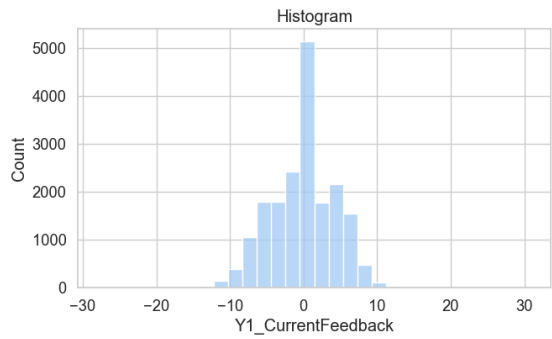
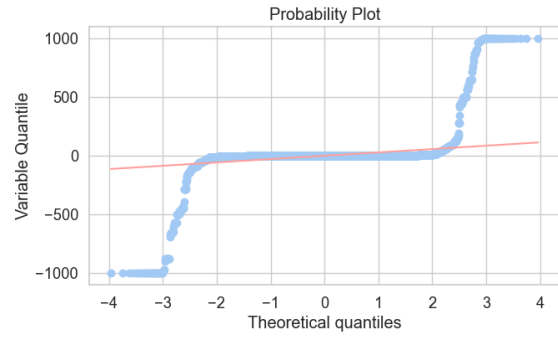
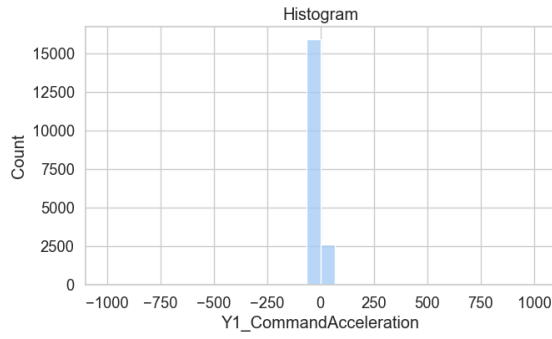
1.11.2 5.6 Analysis features after outlier handling

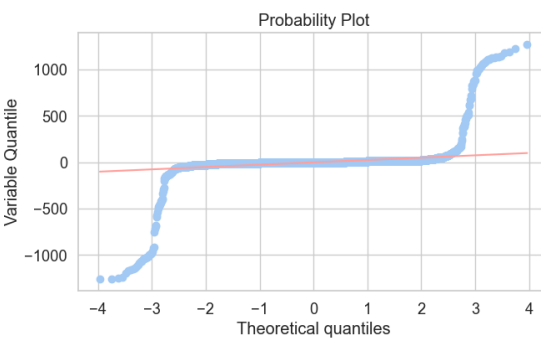
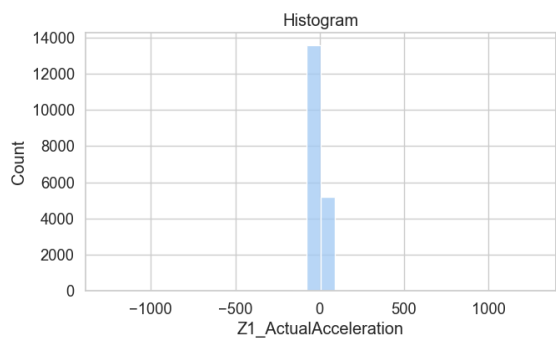
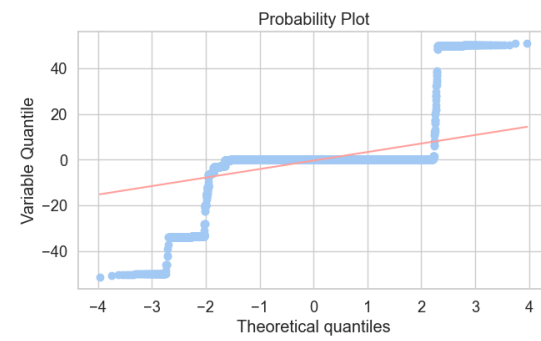
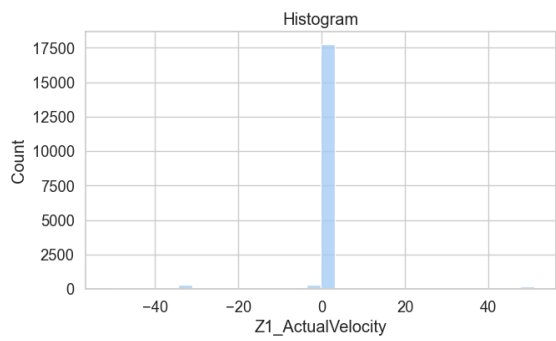
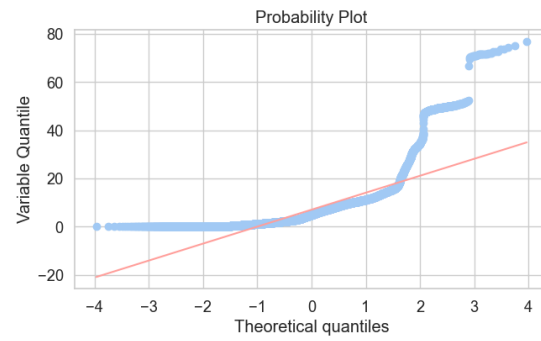
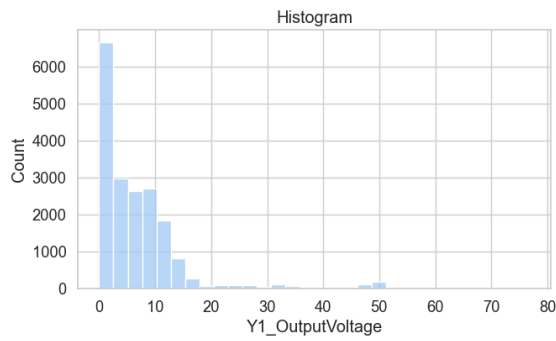
```
[40]: def diagnostic_plots_2(dataset, variable):  
  
    fig = plt.figure()  
    plt.figure(figsize = (16, 4))  
  
    plt.subplot(1, 2, 1)  
    sns.histplot(dataset[variable], bins = 30)  
    plt.title('Histogram')  
  
    plt.subplot(1, 2, 2)  
    stats.probplot(dataset[variable], dist = 'norm', plot = plt)  
    plt.ylabel('Variable Quantile')  
  
[41]: for col in train_X.columns:  
    diagnostic_plots_2(train_X, col)
```

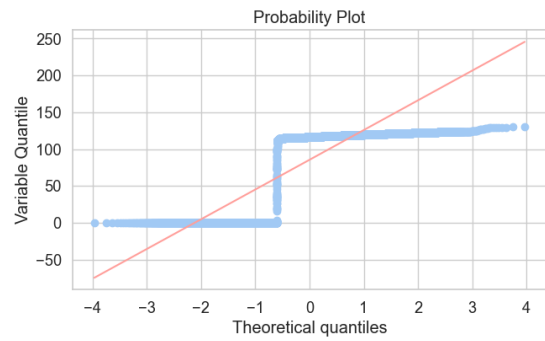
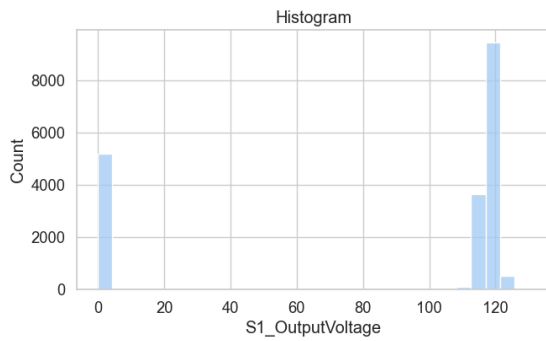
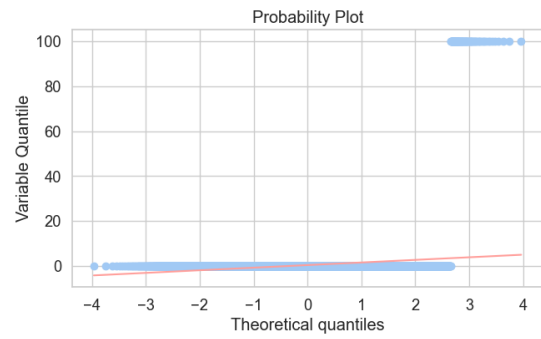
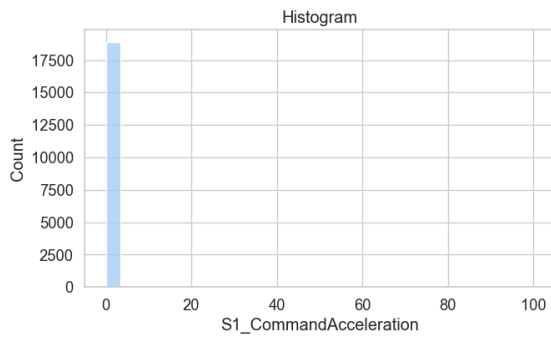
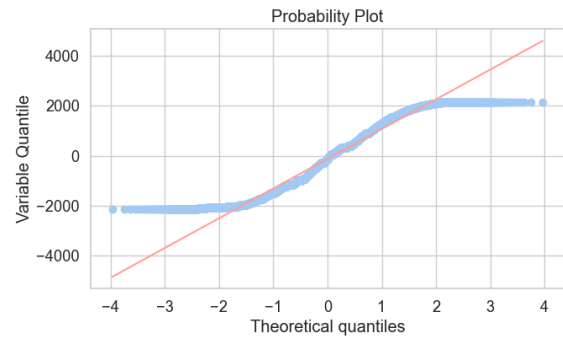
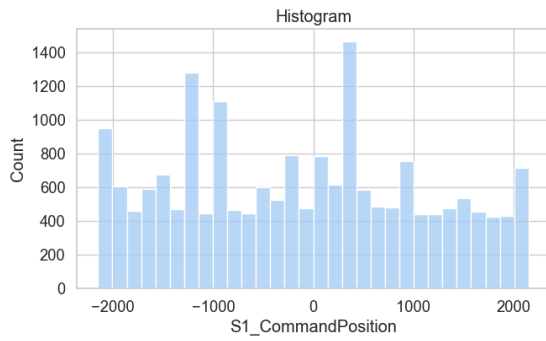
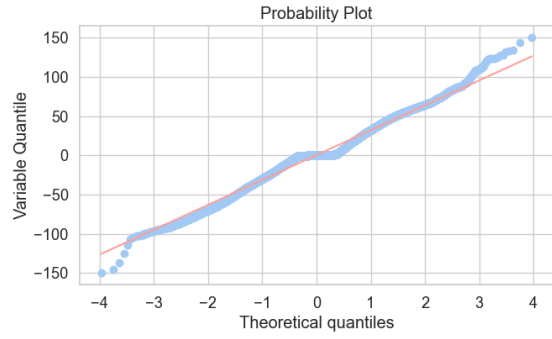
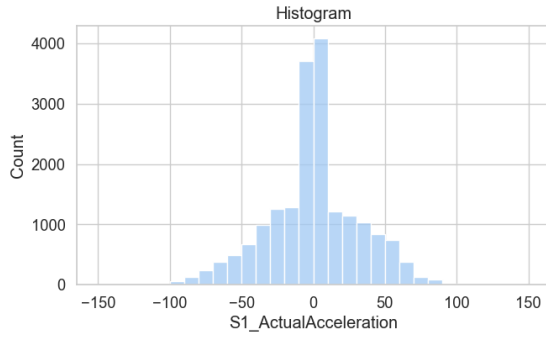


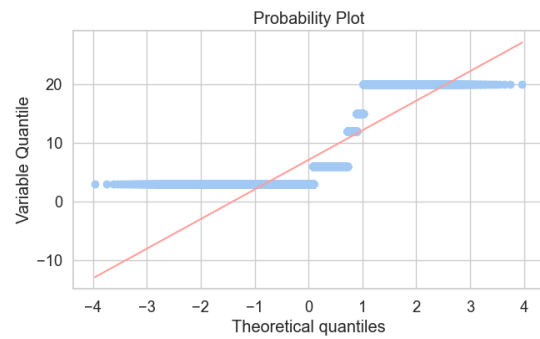
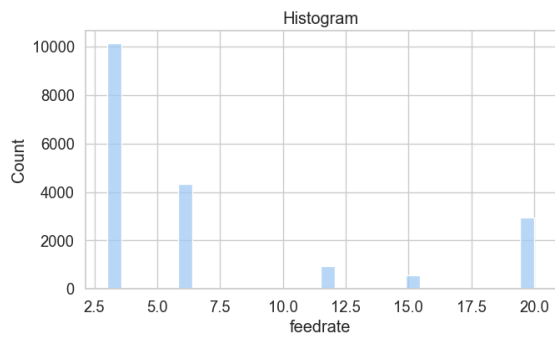
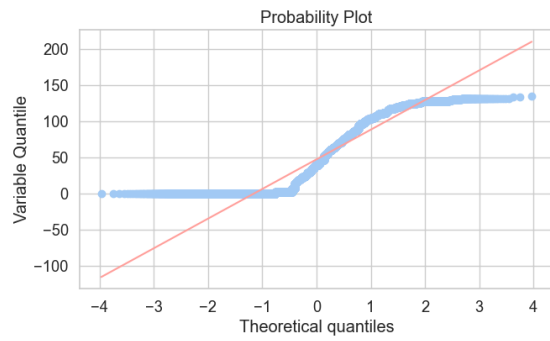
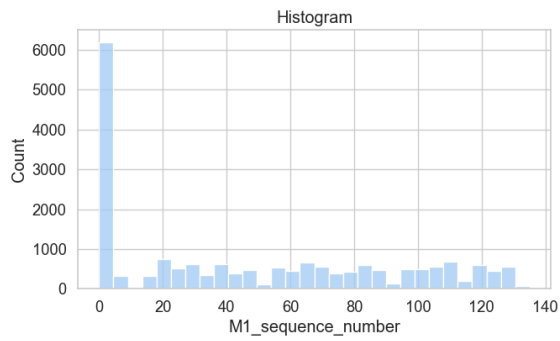
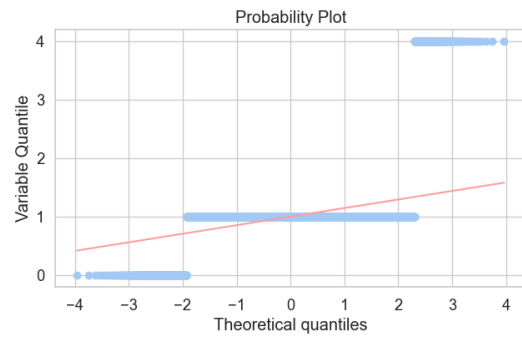
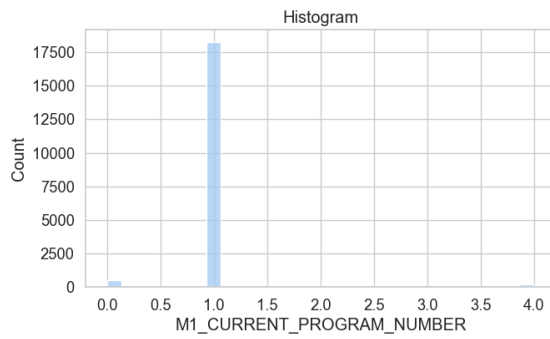


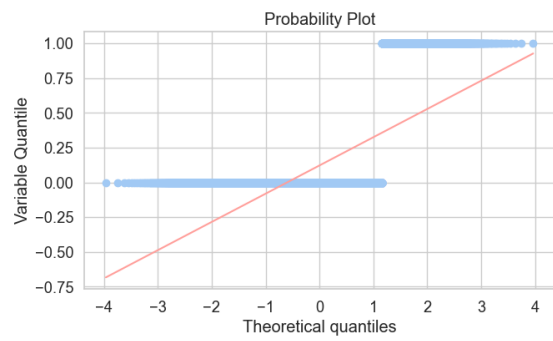
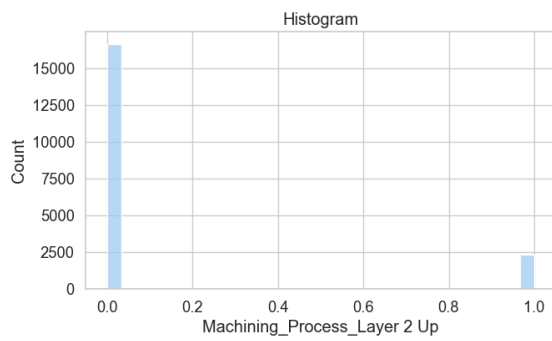
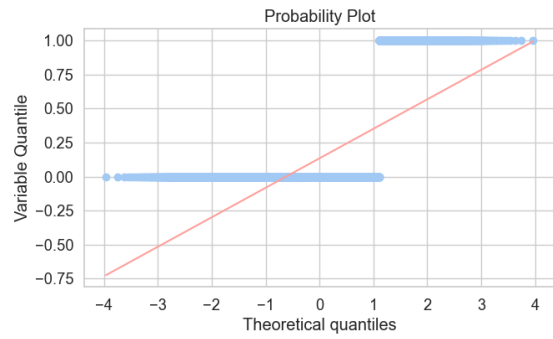
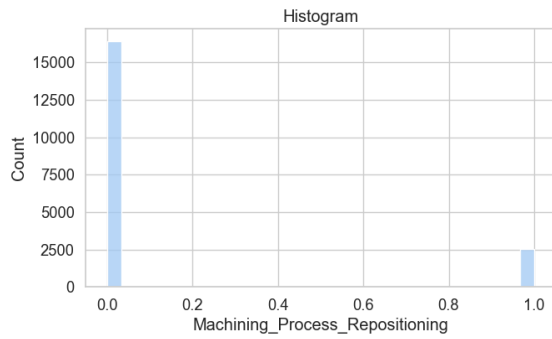
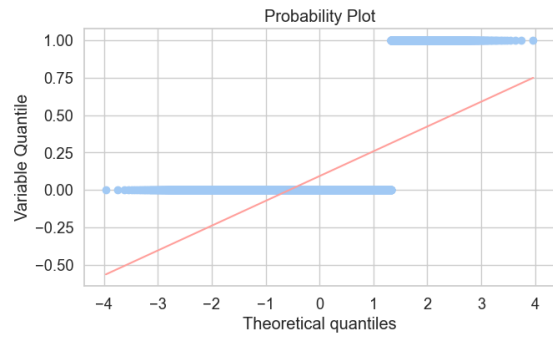
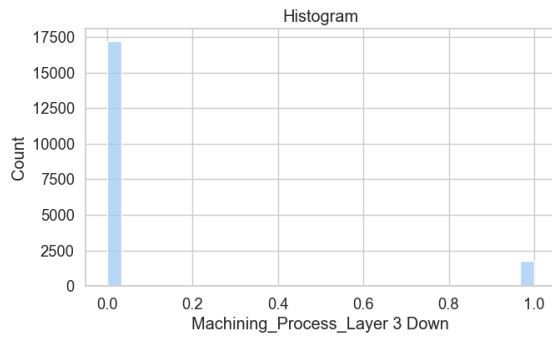
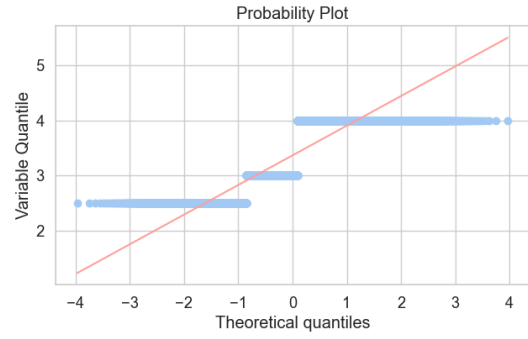
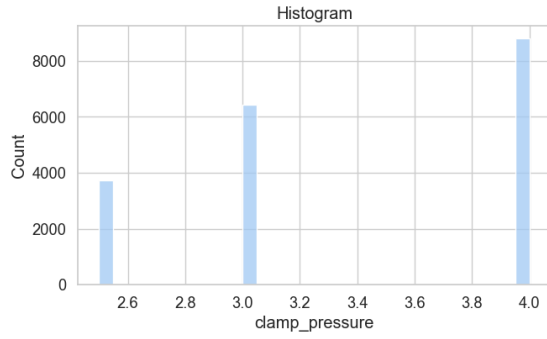


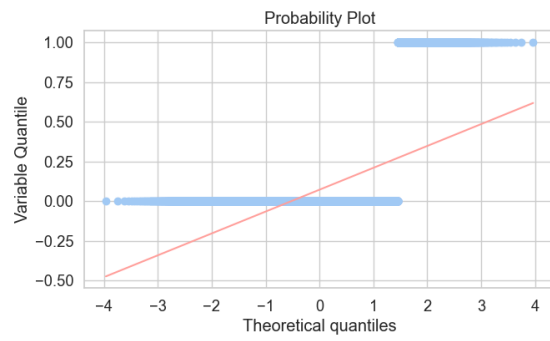
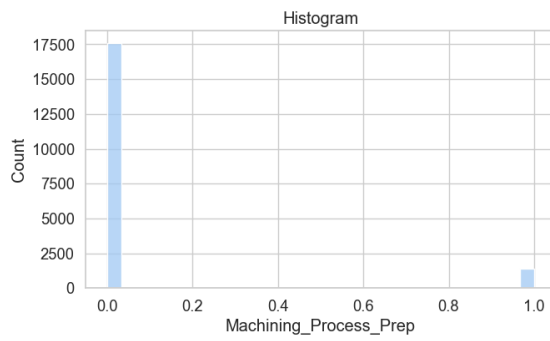
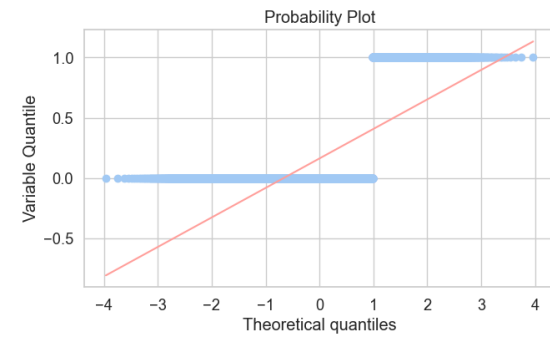
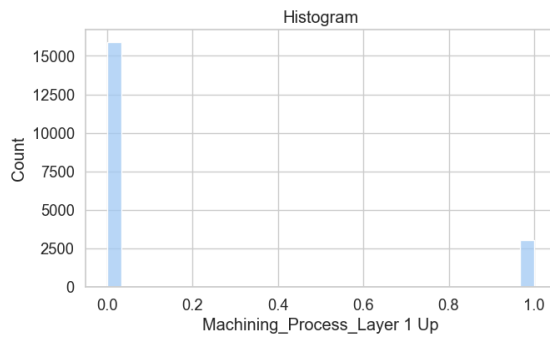
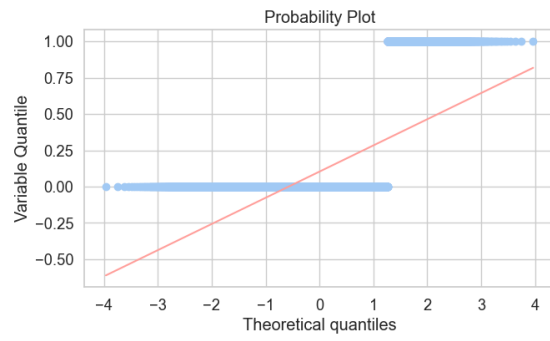
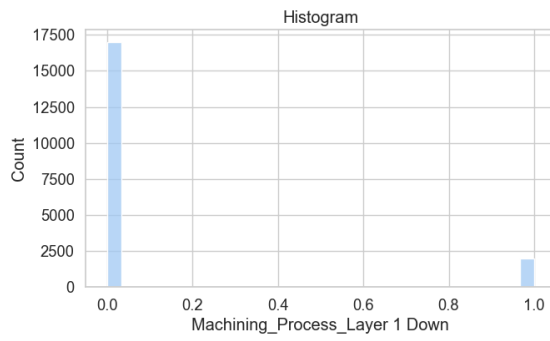


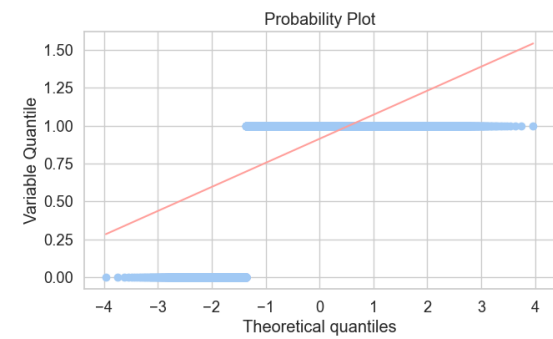
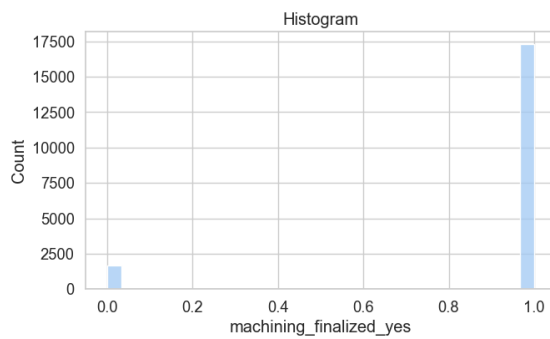
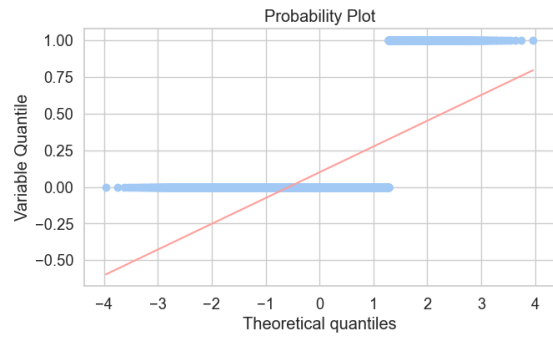
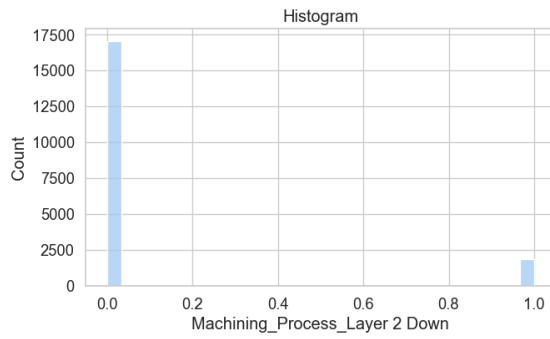
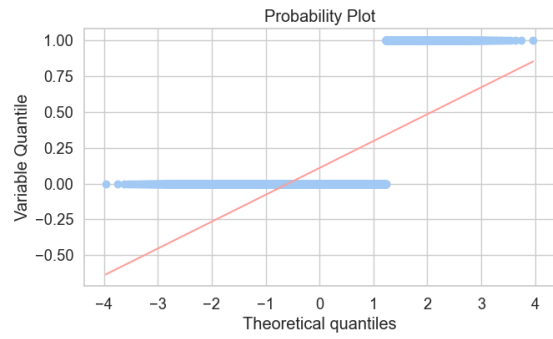
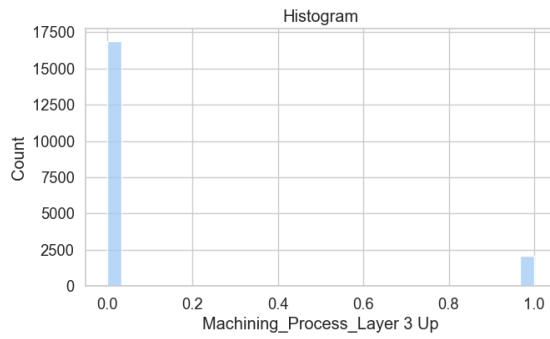
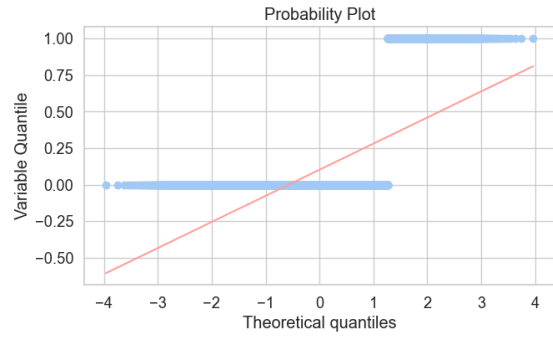
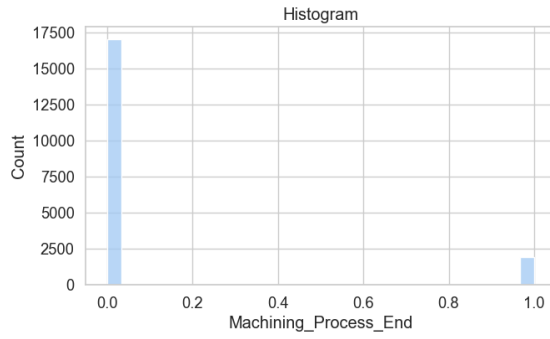


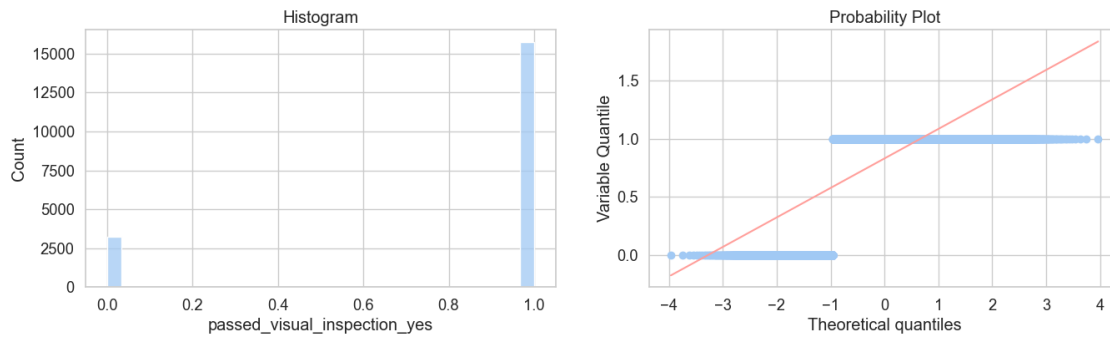












Select only variables with the float data type for discretization

```
[42]: float_vars = train_X.select_dtypes(include=['float'])
float_var_names = float_vars.columns.tolist()
```

```
[43]: len(float_var_names)
```

1.11.3 5.7 DecisionTreeDiscretiser

```
[44]: # Discretizator
treeDisc = DecisionTreeDiscretiser(
    cv = 10,
    scoring = 'accuracy',
    variables = float_var_names,
    regression = False,
    param_grid = {
        'max_depth': [2, 3, 4],
        'min_samples_leaf': [3, 4, 5]
    }
)
treeDisc.fit(train_X, y_train)
```

```
[45]: treeDisc.scores_dict_[float_var_names[0]]
```

```
[46]: train_t = treeDisc.fit_transform(train_X, y_train)
test_t = treeDisc.transform(test_X)
```

Build the graph of the Monotonicity of the relationship between variables X and Y (target)